



Escuela Técnica Superior
de Ingeniería Informática

Grado en Ingeniería Informática y Tecnologías Virtuales

Curso 2025-2026

Trabajo Fin de Grado

**PLATAFORMA PORTABLE SOBRE RASPBERRY PI
PARA AUDITORÍA DE REDES WPA-ENTERPRISE**

Autor: Alejandro Cañadas Fleury

Tutor: Jordi García Quintanilla

Co-Tutor: Raúl Martín Santamaría

Agradecimientos

Quiero dar las gracias en primer lugar a mis tutores, Jordi García Quintanilla y Raúl Martín Santamaría, por su apoyo, paciencia y guía durante todo el desarrollo del proyecto. A mis compañeros y amigos por las incontables horas de estudio y sobre todo por esas risas compartidas durante estos años de carrera que han hecho más liviano este arduo camino. A mi familia y entorno más cercano por su apoyo incondicional, en especial a mis padres Ana y Adalberto, a mi hermano Alonso y a mis abuelos Carmen, Rosario y Adalberto, que me han apoyado y han hecho posible que pueda llegar hasta aquí. También mi agradecimiento más sincero a Paula, por su paciencia, su comprensión y por estar siempre a mi lado. Por último y no menos importante, a la Universidad Loyola Andalucía por brindarme la oportunidad de realizar este trabajo y por proporcionarme los recursos necesarios para llevarlo a cabo. Sin todos ellos nada de esto habría sido posible.

Resumen

Las redes Wi-Fi WPA2/WPA3-Enterprise, sobre las que se apoya un servicio tan extendido como **eduroam**, basan toda su seguridad en una comprobación mutua: el usuario demuestra quién es y, a su vez, la red debe demostrar que es legítima antes de que se intercambie ninguna contraseña. Esa segunda comprobación, que recae sobre un certificado digital, suele quedar en manos del usuario final, que decide si confía o no en la red a la que se conecta. Este trabajo demuestra de forma empírica, sobre un laboratorio completamente aislado, que una red de este tipo puede ser suplantada mediante un ataque denominado *Evil Twin* cuando el usuario no verifica correctamente esa identidad.

Para ello se ha desarrollado, por un lado, una red legítima sobre una máquina virtual y, por otro, una plataforma atacante portátil, autónoma y de bajo coste sobre una Raspberry Pi 5 con una antena Wi-Fi comercial. La plataforma levanta una red gemela que imita a la legítima, engaña al dispositivo de la víctima y captura sus credenciales. Todo el proceso está automatizado, de modo que el equipo ejecuta el ataque sin intervención humana tras el encendido.

Los resultados confirman los tres comportamientos esperados del cliente: entrega de la contraseña en claro, *hash* crackeable o rechazo de la conexión gracias a la correcta configuración del dispositivo, y evidencian que la única protección realmente efectiva, la validación estricta del certificado del servidor, sigue siendo opcional pese a estar disponible desde hace años. El trabajo tiene, por tanto, una clara vocación defensiva y de concienciación.

Palabras clave:

- Redes Wi-Fi
- Ataque Evil Twin
- Métodos EAP (PEAP, GTC, MSCHAPv2)
- Raspberry Pi
- Auditoría de redes Wi-Fi

Índice de contenidos

Índice de tablas	XI
Índice de figuras	XIII
Índice de códigos	1
1. Introducción	3
1.1. Contexto y alcance	3
1.1.1. Historia de la red Wi-Fi	4
1.1.2. Eduroam como caso de uso masivo	8
1.2. Estructura del documento	12
2. Estado del arte	14
2.1. El problema y soluciones existentes	14
2.2. Limitaciones de lo existente	15
2.3. Motivación y alcance	16
3. Objetivos	18
3.1. Objetivo principal	18
3.2. Objetivos secundarios	19
3.2.1. Objetivos técnicos del proyecto	19
3.2.2. Objetivos formativos y de aprendizaje	20
3.3. Metodología	21
3.3.1. Propuesta y requisitos	21
3.3.2. Planificación y validación	22
4. Implementación o descripción informática	25
4.1. Visión general del laboratorio	26
4.2. Entorno y tecnologías utilizadas	28
4.2.1. Hardware	30
4.2.2. Software	31
4.3. Construcción de la red legítima	34
4.3.1. Preparación del anfitrión y la máquina virtual	35
4.3.2. Servidor de autenticación	36

4.3.3.	Punto de acceso con hostapd	37
4.3.4.	Servicios de red: DHCP, NAT y reenvío IP	39
4.3.5.	Automatización con systemd	40
4.3.6.	Validación de la red legítima	41
4.4.	Construcción de la plataforma atacante portátil	41
4.4.1.	PatataWiFiEnterprise y necesidad de modernización	42
4.4.2.	Arquitectura de dos radios	43
4.4.3.	Backend de captura: FreeRADIUS-WPE 3.2.5	44
4.4.4.	El Evil Twin: hostapd 2.6 y certificado del atacante	45
4.4.5.	Downgrade a EAP-GTC y fallback a MSCHAPv2	45
4.4.6.	Red de gestión WPA2-PSK	47
4.5.	Automatización y ciclo de vida desatendido	48
4.5.1.	Orquestación con systemd	48
4.5.2.	Instalación y reversión automatizadas	51
4.6.	Formato y almacenamiento de las credenciales capturadas	53
4.6.1.	Caso EAP-GTC: credencial en claro	54
4.6.2.	Caso de <i>downgrade</i> a MSCHAPv2: hash NetNTLMv1	54
4.6.3.	Recuperabilidad de MSCHAPv2 fuera de línea	55
4.7.	Decisiones de diseño y problemas críticos de montaje	56
4.7.1.	La imposibilidad de multi-BSSID	56
4.7.2.	FreeRADIUS: del 2.1.12 al paquete WPE 3.2.5 de Kali	57
4.7.3.	hostapd 2.6 frente a 2.10	57
4.7.4.	La colisión de direcciones IP entre las dos interfaces	58
4.7.5.	Sesiones tmux huérfanas y el script de arranque	58
4.7.6.	El retardo en la aparición de wlan1 tras el arranque	59
4.7.7.	radiusd ejecutándose como root	59
5.	Resultados, validación o experimentos	60
5.1.	Entorno experimental	60
5.1.1.	Disposición física de los equipos	61
5.1.2.	Procedimiento de prueba	61
5.1.3.	Criterios de validación y desenlaces posibles	62
5.2.	Comportamiento de los dispositivos	63
5.2.1.	Dispositivos Apple (iOS y macOS)	63
5.2.2.	Dispositivos Android	64
5.2.3.	Portátiles con Windows 11	66
5.2.4.	Lectores electrónicos Kindle	66
5.3.	Síntesis de los resultados	67
6.	Conclusiones y trabajos futuros	69
6.1.	Grado de cumplimiento de los objetivos	69
6.2.	Valoración personal	71

6.3. Concienciación y recomendaciones para las instituciones	72
6.4. Trabajos futuros	73
Bibliografía	74
Apéndices	79
A. Configuraciones y artefactos del laboratorio	81
A.1. Fase 1: red legítima	81
A.2. Fase 2: plataforma atacante	85
B. Referencia de comandos de operación	93
B.1. Comprobación del estado de los servicios	93
B.2. Verificación de los SSID activos	94
B.3. Monitorización del log de captura	94
B.4. Acceso remoto a la red de gestión	95
B.5. Acceso a la sesión tmux del orquestador	95
B.6. Recarga del driver de la radio atacante	96
B.7. Regeneración de los certificados del servidor RADIUS	96
C. Herramientas utilizadas en el proyecto	97
C.1. Herramientas de construcción y operación del laboratorio	97
C.2. Herramientas de redacción y control de versiones	98
C.3. Uso de inteligencia artificial	98

Índice de tablas

4.1. Tecnologías del laboratorio	29
4.2. Servicios systemd de la plataforma atacante	50
4.3. Formato del <i>hash</i> NetNTLMv1 capturado	55
5.1. Resumen de los dispositivos probados	68

Índice de figuras

1.1.	Línea temporal de la evolución de las redes Wi-Fi (1971–2028). . .	7
1.2.	Flujo de autenticación de eduroam en <i>roaming</i>	9
3.1.	Cronograma de las fases del proyecto	24
4.1.	Arquitectura general del laboratorio	26
4.2.	Cadena de certificados de la red legítima	36
4.3.	Flujo de autenticación 802.1X/EAP de la red legítima	39
4.4.	Las dos radios de la plataforma atacante	44
4.5.	Flujo del método EAP en el <i>Evil Twin</i>	47
4.6.	Cadena de arranque desatendido de la plataforma atacante	49

Índice de códigos

4.1.	Bloque principal de <code>hostapd.conf</code>	38
4.2.	<code>eap-gtc-downgrade.patch</code>	46
4.3.	Aplicación idempotente de un parche en <code>install.sh</code>	51
4.4.	Estructura de la captura en el caso EAP-GTC.	54
A.1.	<code>/etc/hostapd/hostapd.conf</code> (AP legítimo WPA2-Enterprise)	81
A.2.	Fragmento de <code>/etc/freeradius/3.0/mods-available/eap</code>	82
A.3.	Fragmento de <code>/etc/freeradius/3.0/clients.conf</code>	83
A.4.	Entrada de <code>/etc/freeradius/3.0/users</code> (usuario de prueba)	83
A.5.	<code>/etc/dnsmasq.conf</code> (DHCP + DNS de la red legítima)	83
A.6.	<code>/etc/systemd/system/wlan-prepare.service</code>	84
A.7.	<code>/etc/systemd/system/hostapd.service.d/override.conf</code>	84
A.8.	<code>/etc/systemd/system/wlan-ip.service</code>	84
A.9.	<code>/etc/sysctl.d/99-tfg-forward.conf</code>	85
A.10.	<code>hostapd-mgmt/mgmt.conf</code> (red de gestión)	85
A.11.	<code>scripts/tfg-cleanup.sh</code>	86
A.12.	<code>scripts/tfg-mgmt.sh</code>	86
A.13.	<code>scripts/tfg-attack.sh</code>	87
A.14.	<code>systemd/tfg-cleanup.service</code>	88
A.15.	<code>systemd/tfg-mgmt.service</code>	88
A.16.	<code>systemd/tfg-attack.service</code>	88
A.17.	<code>freeradius-wpe/radiusd.conf.patch</code>	89
A.18.	<code>freeradius-wpe/eap.patch</code>	89
A.19.	<code>freeradius-wpe/eap-gtc-downgrade.patch</code>	90
A.20.	<code>patatawifi-patches/patatawifi.conf.patch</code>	90
A.21.	<code>patatawifi-patches/patatawifi-virtual.conf.patch</code>	92
A.22.	<code>patatawifi-patches/hostapd-2.6-config.patch</code>	92
B.1.	Comprobación del estado de la cadena de servicios	93
B.2.	Relanzamiento manual de la cadena de servicios	94
B.3.	Verificación de las dos redes al aire	94
B.4.	Seguimiento del log de captura de FreeRADIUS-WPE	94
B.5.	Vaciado del log de captura entre sesiones	95
B.6.	Acceso SSH por la red de gestión fuera de banda	95
B.7.	Conexión a la sesión tmux del Evil Twin	95
B.8.	Recarga manual del driver <code>mt76x2u</code>	96
B.9.	Regeneración de los certificados con <code>make bootstrap</code>	96

1

Introducción

La seguridad de las redes Wi-Fi de tipo WPA-Enterprise, y muy en particular de **eduroam**, constituye el eje sobre el que se articula este trabajo, cuyo propósito es demostrar de forma práctica hasta qué punto un ataque *Evil Twin* puede comprometer este tipo de despliegues. Antes de abordar esa demostración, conviene situar el terreno sobre el que se construye todo lo demás. Este capítulo introductorio enmarca el contexto tecnológico del que parte el proyecto y delimita su alcance, repasando la evolución de la tecnología Wi-Fi y el papel de **eduroam** como caso de uso masivo en el ámbito académico.

1.1. Contexto y alcance

La tecnología Wi-Fi se ha convertido en un pilar fundamental de la economía digital y de la sociedad moderna, permitiendo la movilidad y conectividad de miles de millones de dispositivos en todo el mundo. Sin embargo, la historia de la red Wi-Fi, y más concretamente, la pregunta de cuándo surgió esta tecnología tiene una respuesta compleja, ya que sus bases intelectuales y técnicas se forjaron a lo largo de varias décadas y de forma paralela gracias a académicos, reguladores gubernamentales e ingenieros. En este capítulo se recoge ese contexto: la evolución de la tecnología Wi-Fi y **eduroam** como caso de uso masivo.

1.1.1. Historia de la red Wi-Fi

Antes de centrar el trabajo en la seguridad de las redes Wi-Fi, conviene situar esta tecnología en su contexto histórico para entender de dónde viene y por qué ha llegado a ser una infraestructura indispensable. El recorrido cronológico que sigue parte de sus precursores y del nacimiento del estándar IEEE 802.11, atraviesa la consolidación de la Wi-Fi Alliance y la adopción masiva, y llega hasta los estándares de alto rendimiento más recientes, cerrando con la evolución de los mecanismos de seguridad que han acompañado a cada generación.

Los precursores y el nacimiento del estándar (1971 - 1997)

El verdadero inicio de la comunicación inalámbrica de datos se remonta a 1971 con el proyecto ALOHAnet en la Universidad de Hawái [1, 2]. Liderado por el informático Norman Abramson, este experimento buscaba conectar las instalaciones informáticas de las islas hawaianas utilizando radio UHF, esto, dio lugar a, la que es considerada, la primera red inalámbrica de paquetes de datos del mundo. ALOHAnet introdujo el protocolo ALOHA, un método de acceso aleatorio que es el ancestro directo del mecanismo CSMA/CA (del inglés, *Carrier Sense Multiple Access with Collision Avoidance*), el cual sigue siendo la lógica fundamental que utiliza cada punto de acceso Wi-Fi moderno para decidir cuándo transmitir datos y cuándo esperar [1].

Un hito regulativo crítico ocurrió en 1985, cuando la Comisión Federal de Comunicaciones de EE. UU. (FCC) tomó una decisión histórica: abrir las bandas ISM (del inglés, *Industrial, Scientific and Medical*), incluida la frecuencia de 2.4 GHz, para uso sin licencia [1, 3]. Esta decisión democratizó las ondas de radio y desató una extraordinaria ola de innovación, permitiendo que cualquier persona pudiera construir y operar dispositivos en estas frecuencias sin necesidad de ningún tipo de permiso especial.

En la década de 1990, las innovaciones tecnológicas permitieron lo que se considera el nacimiento del estándar ya que comenzaron a aparecer las piezas necesarias para el Wi-Fi tal como lo conocemos hoy en día. En 1991, la corporación NCR y AT&T inventaron en los Países Bajos un proyecto destinado inicialmente a sistemas de cajas registradoras, lanzando así los primeros productos inalámbricos bajo el nombre de WaveLAN, los cuales tenían unas velocidades de aproximadamente 1 a 2 Mbps [4].

Paralelamente, un equipo en la Organización de Investigación Científica e Industrial del Commonwealth (CSIRO) en Australia, liderado por el Dr. John O'Sullivan, resolvió un grave problema técnico: cuando las señales de radio rebotaban y llegaban en distintos momentos se creaba una interferencia por trayectos múltiples. Para resolverlo, desarrollaron una brillante técnica matemática utilizan-

do Transformadas Rápidas de Fourier, que patentaron en 1996. Esta técnica se convirtió en la base fundamental de la forma de onda OFDM (del inglés, *Orthogonal Frequency-Division Multiplexing*) utilizada en todos los estándares Wi-Fi modernos [1].

Finalmente, impulsados por ingenieros como Vic Hayes (conocido como el “padre del Wi-Fi”) y Bruce Tuch, el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) publicó el primer estándar formal IEEE 802.11 en 1997 [4, 1, 2]. Este estándar inicial ofrecía velocidades de hasta 2 Mbps, estableciendo el marco de trabajo sobre el cual se construiría todo lo demás.

La Alianza Wi-Fi y la adopción masiva (1999 - 2003)

El año 1999 fue un punto de inflexión. Aunque el estándar ya existía, la industria necesitaba asegurar que los equipos de distintos fabricantes funcionaran entre sí. Para ello, varias empresas tecnológicas fundaron la Wireless Ethernet Compatibility Alliance (WECA) [5, 2]. En el año 2000, la agencia Interbrand impulsó el cambio de nombre de esta tecnología a Wi-Fi, y, posteriormente, en 2002, la organización pasó a llamarse Wi-Fi Alliance [5, 1].

En 1999 se lanzaron dos estándares cruciales: el 802.11b, que operaba en la banda de 2.4 GHz con velocidades de hasta 11 Mbps, y el 802.11a, que operaba en la banda de 5 GHz alcanzando los 54 Mbps [1, 4]. En el ámbito comercial, el gran avance llegó cuando Apple adoptó el Wi-Fi para su serie de portátiles iBook en 1999, bajo la marca AirPort, lo que hizo que se convirtiera en el primer producto de consumo masivo que podía ofrecer este tipo de conectividad [4]. El estándar 802.11b democratizó el acceso, llevando el Wi-Fi a aeropuertos y cafeterías. Tras esta absorción de forma global, en 2003 se ratificó el estándar 802.11g, el cual combinó la velocidad de 54 Mbps del 802.11a con la popular banda de 2.4 GHz del 802.11b [1, 3].

Los estándares de alto rendimiento (2009 - Actualidad)

Conforme pasaba el tiempo, había más demanda de ancho de banda, lo cual llevó a la evolución de las tecnologías. En 2009 llegó el estándar 802.11n (Wi-Fi 4). Esto fue un hito monumental, ya que introdujo la tecnología MIMO (del inglés, *Multiple-Input Multiple-Output*) [1, 2, 3]. Esta tecnología permitía que, utilizando múltiples antenas, las velocidades se dispararan hasta 600 Mbps, haciendo del Wi-Fi una alternativa ya mucho más comparable a las conexiones por cable. En 2013, el 802.11ac (Wi-Fi 5) refinó este enfoque operando exclusivamente en la banda de 5 GHz con canales más anchos y tecnología MU-MIMO (del inglés, *Multi-User Multiple-Input Multiple-Output*), permitiendo a un punto de acceso transmitir a varios clientes simultáneamente y permitiendo que las velocidades

teóricas subieran a más de 3 Gbps [1, 4].

Con la explosión y aumento de dispositivos conectados (IoT (del inglés, *Internet of Things*), *smartphones*, portátiles), el cuello de botella dejó de ser la velocidad máxima y pasó a ser la eficiencia en entornos densos. En 2019, se introdujo el 802.11ax (Wi-Fi 6), que adoptó una tecnología de las redes celulares llamada OFDMA (del inglés, *Orthogonal Frequency-Division Multiple Access*), que permite dividir un canal para atender a múltiples usuarios simultáneamente con latencias reducidas [6, 1, 2]. En 2021, la extensión Wi-Fi 6E otorgó acceso a la nueva banda de 6 GHz. Hasta entonces todo el tráfico se concentraba en las bandas de 2.4 y 5 GHz que, al estar compartidas por un número cada vez mayor de dispositivos, sufrían una fuerte congestión e interferencias que limitaban el rendimiento real. Al tratarse de una banda nueva y reservada a los equipos más modernos, los 6 GHz aportan un espectro limpio, es decir, libre de esa saturación, multiplicando así el espectro disponible [1, 4].

El avance más reciente es el 802.11be, o Wi-Fi 7, ratificado formalmente en 2024 [7, 1]. Su innovación clave es la Operación Multienlace (MLO, del inglés *Multi-Link Operation*), que permite a los dispositivos conectarse simultáneamente a través de múltiples bandas (2.4, 5 y 6 GHz), logrando unas velocidades teóricas de hasta 46 Gbps y latencias predecibles, ideales para los usos cotidianos cada vez más exigentes, como las videollamadas en alta definición, el *streaming* de vídeo sin cortes o los hogares con multitud de dispositivos conectados a la vez [7, 1]. Mientras tanto, el IEEE ya está trabajando en el estándar 802.11bn (Wi-Fi 8), que está proyectado para 2028 y está enfocado en mejoras de fiabilidad, como reducción de latencia, pérdida de paquetes y rendimiento en condiciones difíciles [1, 7, 8].

En poco más de 20 años, el Wi-Fi ha pasado de ser un proyecto de conectividad esporádico usado solamente en algunos lugares, a ser un gigante económico global y una tecnología indispensable para muchos. Según estimaciones del año 2018, el valor económico global del Wi-Fi rozó los 2 trillones de dólares, soportando a más de 13.000 millones de dispositivos en todo el planeta (con una media de casi dos dispositivos por cada persona en el mundo) y gestionando más de la mitad de todo el tráfico de Internet a nivel global [2].

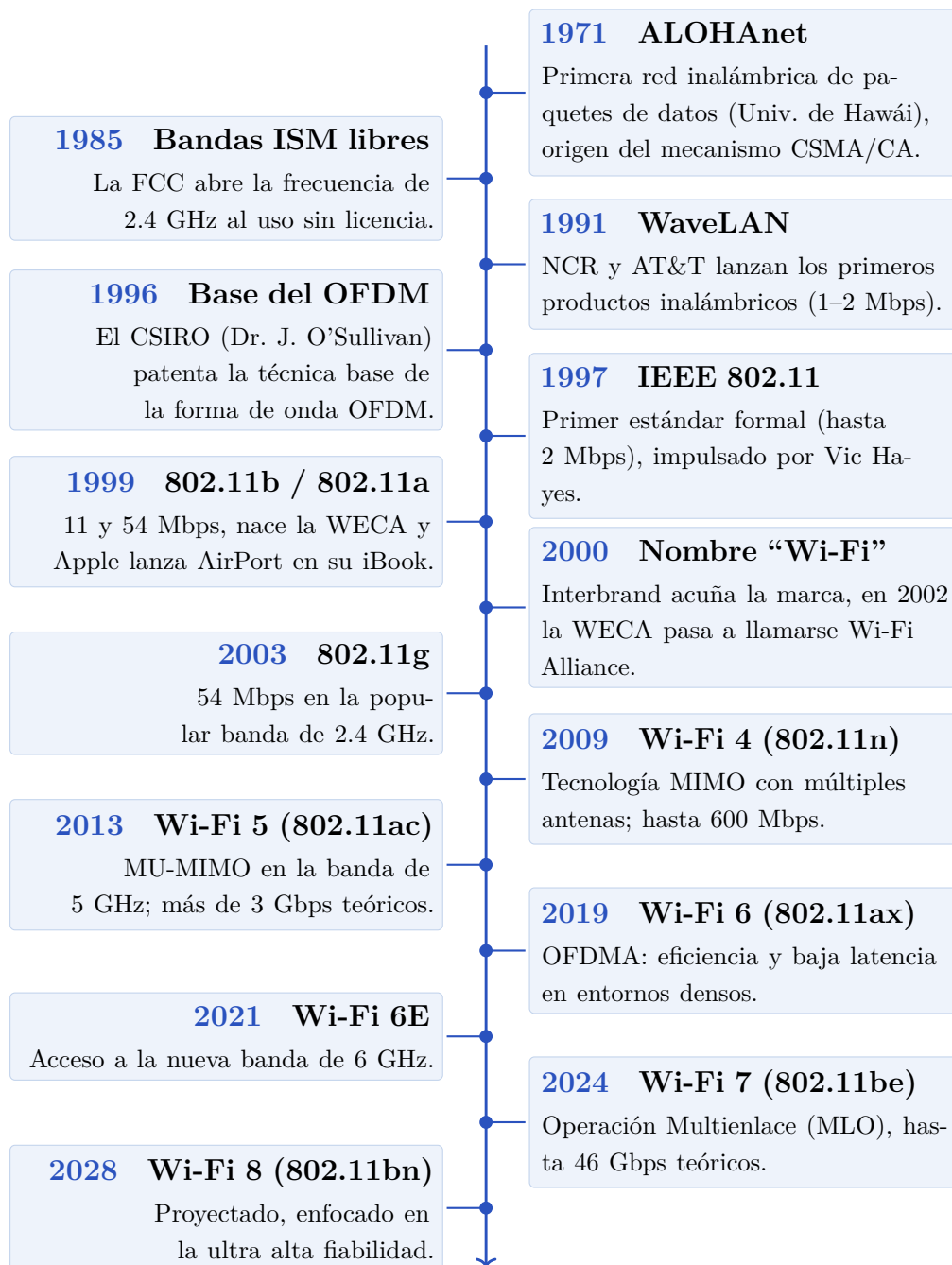


Figura 1.1: Línea temporal de la evolución de las redes Wi-Fi (1971–2028).

La evolución de la seguridad en redes Wi-Fi

A medida que la adopción crecía y más gente hacía uso de ella, la seguridad se convirtió en un desafío crítico. El estándar original del año 1997 utilizaba un protocolo de cifrado llamado WEP (del inglés, *Wired Equivalent Privacy*), que resultó ser muy vulnerable [9, 10]: en 2001, varios investigadores probaron

que WEP podía descifrarse con facilidad mediante herramientas automatizadas. Para evitar el colapso de la tecnología, la Wi-Fi Alliance introdujo como medida temporal el WPA (del inglés, *Wi-Fi Protected Access*) en 2003 y, poco después, el WPA2 en 2004 (basado en el estándar IEEE 802.11i), el cual requería cifrado AES (del inglés, *Advanced Encryption Standard*). Este se mantuvo como el estándar de seguridad dominante durante más de 15 años [9, 10, 11], hasta que en 2018 la industria evolucionó hacia el WPA3. Dado que WPA2 ya empleaba cifrado AES, las mejoras de WPA3 no se centran en el algoritmo de cifrado, y además difieren según el modo de operación. En las redes domésticas (WPA3-Personal, basadas en una contraseña compartida) la novedad principal es el protocolo SAE (del inglés, *Simultaneous Authentication of Equals*), que reemplaza el *handshake* de WPA2 (vulnerable a ataques de diccionario fuera de línea sobre el material capturado) y protege frente a la fuerza bruta aunque la contraseña sea débil [10, 1]. En las redes corporativas (WPA3-Enterprise), que son las que conciernen a este trabajo, no interviene SAE: se conserva la autenticación 802.1X/EAP contra un servidor RADIUS (del inglés, *Remote Authentication Dial-In User Service*) y el mismo modelo de confianza basado en la validación del certificado del servidor que ya empleaba WPA2-Enterprise [10]. Por ello, el ataque *Evil Twin* desarrollado en esta memoria sigue siendo igual de válido frente a redes WPA3-Enterprise, ya que el ataque no se dirige contra el cifrado ni contra el *handshake*: se aprovecha de la confianza indebida del cliente en un certificado ilegítimo.

1.1.2. Eduroam como caso de uso masivo

Repasada la evolución de la tecnología Wi-Fi y de su seguridad, conviene aterrizarla en un despliegue real y a gran escala que sirva de hilo conductor al resto del trabajo: **eduroam**. A continuación se presenta qué es esta iniciativa y cuál es su impacto global, cómo se configura y se sostiene sobre una jerarquía federada de servidores RADIUS, y se desciende finalmente al caso concreto de la configuración en la Universidad Loyola Andalucía.

Definición e impacto global de eduroam

eduroam (del acrónimo en inglés *educational roaming*) es una iniciativa internacional creada originalmente bajo el amparo de TERENA (ahora conocida como GÉANT), la cual facilita la movilidad de investigadoras, estudiantes y personal académico [12]. Su objetivo principal es proporcionar un espacio Wi-Fi común que permita el acceso inalámbrico a Internet de forma sencilla, segura y transparente cuando un usuario se desplaza a otra institución asociada al proyecto. De forma que la premisa fundamental es que el usuario pueda conectarse a la red de cualquier institución que visite utilizando exactamente las mismas credenciales (usuario y contraseña) de su organización de origen, como si estuviera en su propia oficina o

campus [13].

A nivel técnico, **eduroam** es una infraestructura basada estrictamente en el estándar IEEE 802.1X (que proporciona un control de acceso a la red basado en puertos, es decir, el acceso se autoriza en el propio punto de conexión y el tráfico de datos del usuario permanece bloqueado hasta que la autenticación tiene éxito) y en los protocolos de seguridad WPA2-Enterprise y WPA3-Enterprise [14, 12, 15]. En lugar de depender de contraseñas compartidas, la red utiliza una jerarquía global de servidores RADIUS interconectados que enrutan de forma segura las peticiones de autenticación [14, 12]. Cuando un usuario visita una institución externa, el punto de acceso (AP, del inglés *Access Point*) local captura su petición y, a través de esta jerarquía RADIUS, la redirige hasta el servidor de autenticación de su institución de origen, que es la única entidad que verifica y valida las credenciales reales [12, 16].

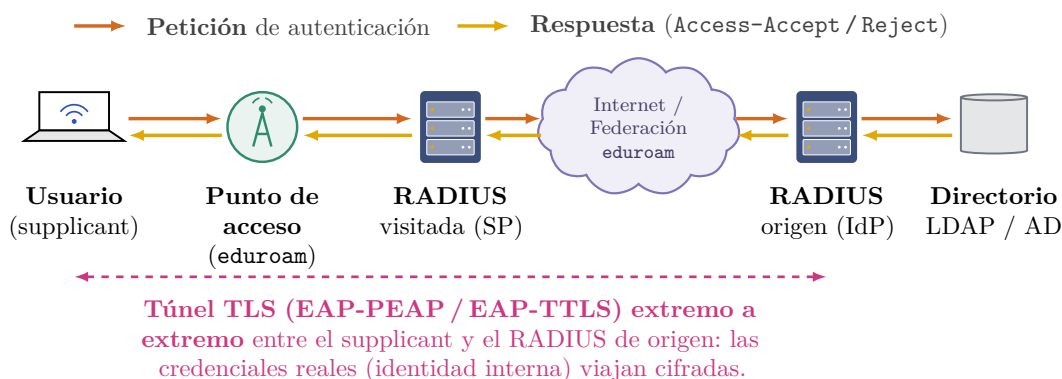


Figura 1.2: Flujo de autenticación de **eduroam** cuando un usuario se conecta desde una institución ajena a la suya. El punto de acceso de la institución visitada (proveedor de servicio, SP) emite el SSID **eduroam** con WPA2/WPA3-Enterprise (802.1X) y reenvía la petición a través de la jerarquía federada de servidores RADIUS hasta el servidor de la institución de origen del usuario (proveedor de identidad, IdP), única entidad que valida las credenciales contra su directorio (LDAP/Active Directory). La identidad externa (**anonymous@dominio**) viaja en claro y solo sirve para enrutar la petición, las credenciales reales (identidad interna) viajan cifradas dentro de un túnel TLS extremo a extremo entre el *supplicant* y el IdP. Las flechas naranjas representan la petición y las amarillas la respuesta (Access-Accept / Reject).

A nivel de impacto y uso, **eduroam** ha experimentado un crecimiento mas que exponencial y se ha llegado a consolidar como un pilar indispensable para la movilidad académica a nivel global. Los últimos datos nos muestran que el servicio está desplegado en más de 100 países y territorios, dando cobertura a decenas de miles de ubicaciones [17]. Durante el año 2024 el sistema obtuvo una

cifra récord de 8.400 millones de autenticaciones nacionales e internacionales, superando ampliamente el récord anterior de 7.500 millones registrado en 2023 [17]. En España, este servicio está coordinado por RedIRIS y está ampliamente extendido por todo el país, estando disponible en más de 120 instituciones, entre las cuales se incluyen alrededor de 70 universidades, así como en cientos de organismos de investigación [13].

Configuración y arquitectura federada

Como ya se ha anticipado, **eduroam** se sostiene sobre una jerarquía federada de servidores RADIUS; dentro de ese esquema de confianza y enrutamiento, las instituciones desempeñan dos roles: el de proveedor de identidad (IdP, del inglés *Identity Provider*), que es la institución a la que pertenece el usuario originalmente, y el de proveedor de servicio (SP, del inglés *Service Provider*), que es la institución que se visita [14, 12, 16].

Para que un usuario pueda conectarse en distintos países o instituciones, **eduroam** emplea túneles EAP (del inglés, *Extensible Authentication Protocol*), como EAP-TTLS (del inglés, *Tunneled Transport Layer Security*) o EAP-PEAP, los cuales dividen la información en dos fases [14, 16]:

1. *Identidad externa*: contiene únicamente el dominio del usuario (por ejemplo, `anonymous@universidad.es`). Esta información viaja en claro y sirve exclusivamente para que el punto de acceso (AP) y el servidor RADIUS local sepan a qué institución redirigir la autenticación [16].
2. *Identidad interna*: una vez que el servidor RADIUS de la institución de origen del usuario responde, se establece un túnel seguro y cifrado entre el cliente y el servidor de autenticación; dentro de este túnel viajan las credenciales reales (usuario y contraseña) [14, 16].

De este modo, la institución que visita el usuario (SP) nunca puede tener acceso a las credenciales del usuario, lo que preserva su privacidad y delega así la responsabilidad de la autenticación exclusivamente en la institución a la que este usuario pertenece (IdP) [16, 12].

Para que una organización configure y despliegue **eduroam** localmente, debe seguir estos tres pasos fundamentales:

1. *Instalar un servidor RADIUS* y conectarlo al sistema de gestión de identidades de la organización (que suele ser un directorio activo o servidor LDAP (del inglés, *Lightweight Directory Access Protocol*)) para validar las credenciales de sus usuarios [12, 16].

2. *Configurar los puntos de acceso (APs)*: los APs de la red deben configurarse para emitir el SSID (del inglés, *Service Set Identifier*) **eduroam** utilizando el protocolo de seguridad WPA2-Enterprise o WPA3-Enterprise, y para redirigir las peticiones de autenticación al servidor RADIUS local [12, 16].
3. *Federar el servidor RADIUS*: conectar el servidor RADIUS local a los servidores *proxy* de nivel nacional e internacional de **eduroam**, lo que permite que las peticiones de autenticación puedan ser redirigidas a cualquier parte del mundo [12, 16].

Configuración específica en la Universidad Loyola Andalucía

Accediendo al portal de **eduroam** para el configuration assistant tool (CAT) [18], se puede descargar un perfil de configuración específico para la Universidad Loyola Andalucía, disponible tanto para personal de la universidad como para alumnos. Se ofrecen instaladores para Windows, Apple, Linux, Chrome OS y Android, y configura el *supplicant* del cliente con tres elementos clave para mitigar ataques de tipo *Evil Twin*: el anclaje de la autoridad de certificación institucional propia (CN (del inglés, *Common Name*) Wifi Universidad Loyola, autofirmada y emitida exclusivamente para este servicio), la verificación estricta del nombre del servidor RADIUS legítimo (wifiul.eduroamuloyola.es), y el uso de identidad externa anónima (anonymous@al.uloyola.es) para proteger la privacidad del usuario durante la fase pública del *handshake* TLS (del inglés, *Transport Layer Security*). La configuración resultante implementa WPA2-Enterprise con método EAP-PEAP y autenticación interna MSCHAPv2 [12].

Tras la descarga y análisis del *script* de instalación de la herramienta **eduroam** Configuration Assistant Tool (CAT) para sistemas Linux utilizado por la Universidad Loyola Andalucía, se detallan los parámetros técnicos exactos bajo los cuales la red está configurada para el perfil de “Alumno”.

Esta configuración es la que los usuarios de la Universidad Loyola Andalucía deben tener para operar de forma segura y correcta en la red **eduroam**:

- *SSID*: **eduroam**.
- *Dominio del usuario (REALM)*: los usuarios (alumnos) utilizan el sufijo @al.uloyola.es.
- *Identidad anónima (outer identity)*: configurada de forma predeterminada como anonymous@al.uloyola.es, lo que asegura que el tráfico de enrutamiento inicial no revele el nombre de usuario real.
- *Protocolo de autenticación EAP*:

- *Método externo (fase 1)*: PEAP (del inglés, *Protected Extensible Authentication Protocol*).
 - *Método interno (fase 2)*: MSCHAPv2 (del inglés, *Microsoft Challenge-Handshake Authentication Protocol version 2*).
- *Servidor de autenticación institucional*: wifiul.eduroamuloyola.es.
 - *Certificado de autoridad certificadora (CA)*: la configuración legítima inyecta un certificado raíz público específico (incluido en el instalador) para validar el servidor wifiul.eduroamuloyola.es.

1.2. Estructura del documento

Esta memoria se organiza en seis capítulos y tres apéndices, que siguen el recorrido natural del trabajo desde el planteamiento del problema hasta las conclusiones.

El presente Capítulo 1 sitúa el contexto y el alcance del trabajo, repasa la evolución de las redes Wi-Fi y de su seguridad y presenta **eduroam** como el caso de uso masivo sobre el que gira todo el proyecto.

El Capítulo 2 recoge el estado del arte. Expone el problema de fondo, esto es, la autenticación basada en el certificado del servidor y el riesgo que aparece cuando el cliente no lo valida, repasa las soluciones y herramientas existentes con sus limitaciones, y deriva de ese contraste la motivación y el alcance de la propuesta.

El Capítulo 3 fija los objetivos del trabajo, tanto el principal como los secundarios de carácter técnico y formativo, y describe la metodología seguida, con la propuesta de solución, los requisitos del montaje y la planificación por fases junto con sus criterios de validación.

El Capítulo 4 detalla la implementación del laboratorio. Parte de una visión general del entorno y de las tecnologías empleadas y describe después, paso a paso, la construcción de la red legítima y de la plataforma atacante portátil sobre la Raspberry Pi 5.

El Capítulo 5 presenta los resultados y la validación experimental. Describe el entorno de pruebas y el mecanismo que conduce al dispositivo víctima hacia el gemelo, recoge el comportamiento observado dispositivo a dispositivo y lo reúne en una tabla resumen que permite una lectura transversal.

El Capítulo 6 cierra la memoria con las conclusiones. Valora el grado de cumplimiento de los objetivos, recoge la valoración personal del autor, plantea la

labor de concienciación y las recomendaciones para las instituciones y apunta las líneas de trabajo futuro.

Por último, los apéndices reúnen el material de apoyo, esto es, las configuraciones y artefactos íntegros del laboratorio (Apéndice A), una referencia de los comandos de operación más habituales (Apéndice B) y la relación de herramientas utilizadas en el proyecto (Apéndice C).

2

Estado del arte

Antes de abordar la construcción del laboratorio, conviene situar este trabajo dentro de lo que ya se sabe y se ha hecho sobre el ataque *Evil Twin* contra redes WPA2/WPA3-Enterprise. Este capítulo recorre primero el problema de fondo (la autenticación mutua basada en certificado y el riesgo que aparece cuando el cliente no lo valida) junto a las soluciones y herramientas existentes y los resultados que obtienen, a continuación expone las limitaciones que impiden que esas medidas basten para cerrar el problema. De ese contraste se deriva, por último, la motivación y el alcance de la propuesta, centrada en el factor humano y en lo accesible que resulta el ataque.

2.1. El problema y soluciones existentes

La seguridad de las redes WPA2/WPA3-Enterprise (basadas en el estándar 802.1X y el protocolo EAP) depende de la autenticación mutua [19]. A diferencia de las redes personales, aquí el servidor de autenticación (AS, del inglés *Authentication Server*) debe demostrar su identidad al cliente (*supplicant*) mediante un certificado digital antes de que el cliente envíe sus credenciales [19]. El problema radica en que si el *supplicant* no valida correctamente este certificado, un atacante puede hacerse pasar por el servidor de autenticación legítimo para que el cliente le entregue sus credenciales: levanta un Punto de Acceso falso (*Rogue AP* o *Evil Twin*), establece un túnel TLS fraudulento (EAP-PEAP o EAP-TTLS) y captura dichas credenciales (ya sea en texto claro mediante métodos como PAP (del inglés, *Password Authentication Protocol*)/GTC o capturando el desafío-respuesta en

MSCHAPv2) [19, 20].

Para analizar y mitigar este problema, la comunidad científica y técnica ha desarrollado diversos trabajos y herramientas:

- Análisis experimentales del ataque en entornos reales: Investigaciones como las de Palamà et al. o el proyecto EvilRoam han demostrado la viabilidad práctica de este ataque [21, 19]. En un entorno universitario real, Palamà et al. lograron robar credenciales a más de un tercio de los usuarios (17 de 37 estudiantes de ingeniería) de forma completamente pasiva, simplemente porque sus dispositivos estaban mal configurados [21]. EvilRoam obtuvo resultados similares utilizando hardware económico (Raspberry Pi) y software libre (FreeRADIUS modificado), logrando que casi un tercio de los dispositivos que intentaron conectarse al *Evil Twin* completaran la autenticación y expusieran sus datos.
- Soluciones a nivel de red e infraestructura: Se han propuesto multitud de sistemas académicos para detectar *Evil Twins*. Estos incluyen enfoques basados en anomalías de tráfico (midiendo el tiempo de llegada de los paquetes) [22], sistemas basados en la ubicación que se apoyan en el RSSI (del inglés, *Received Signal Strength Indicator*), el valor que indica la potencia con la que un dispositivo recibe la señal de un punto de acceso: dado que un *Evil Twin* emite desde una posición física distinta a la del punto de acceso legítimo, su nivel de señal difiere del esperado y puede delatarlo [23]; y enfoques basados en *fingerprinting*, una técnica que identifica de forma unívoca cada equipo a partir de rasgos físicos difíciles de imitar, que permiten distinguir el hardware legítimo del impostor [24]. También se han propuesto sistemas criptográficos del lado del cliente, como el *Robust Certificate Management System* (RCMS), que introduce un código de verificación adicional para el certificado del servidor [20].
- Herramientas de configuración automática (La solución oficial): Para evitar que el usuario configure la red manualmente, la federación eduroam desarrolló la herramienta eduroam Configuration Assistant Tool (CAT) (y aplicaciones móviles derivadas como geteduroam). Esta herramienta instala un perfil cerrado en el dispositivo que garantiza que el *supplicant* confíe únicamente en el certificado legítimo de la institución y fuerza el uso de identidades anónimas para evitar el rastreo [25, 26, 27].

2.2. Limitaciones de lo existente

A pesar de los desarrollos mencionados, el problema persiste en la actualidad debido a graves inconvenientes tanto en la adopción de las medidas como en la usabilidad de los sistemas operativos:

- Falta de obligatoriedad en el uso de herramientas seguras: La principal debilidad de soluciones como **eduroam** CAT es que su uso no es obligatorio. Muchos usuarios prefieren configurar la red de forma manual [20, 19] o directamente no la llegan a configurar nunca. A esto se suma que diversas instituciones proporcionan guías de configuración obsoletas e inseguras que instruyen explícitamente a los usuarios a seleccionar la opción “No validar certificado” o a dejar la identidad anónima en blanco [21, 19].
- Vulnerabilidades en la interfaz de los sistemas operativos: La literatura demuestra que los sistemas operativos móviles fallan en la protección del usuario.
 - En Android, a pesar de que las versiones recientes intentan obligar a especificar un dominio de certificado, gran parte del ecosistema y de las capas de personalización de los fabricantes aún permiten elegir la opción “No validar” [21, 19].
 - En iOS (Apple), el sistema es más restrictivo y no permite configuraciones inseguras por defecto, pero delega la validación del certificado al usuario. Cuando un iPhone se conecta a un *Evil Twin*, muestra una advertencia con el certificado falso. Puesto que los usuarios carecen de conocimientos criptográficos para distinguir un certificado legítimo de uno falsificado, frecuentemente aceptan el certificado fraudulento por inercia o inexperiencia, comprometiendo su seguridad [21, 19].
- Inviabilidad de las soluciones académicas: Los métodos de detección basados en anomalías de tráfico, sensores de RSSI o *fingerprinting* suelen ser poco fiables. Las redes Wi-Fi tienen variaciones naturales de retardo y potencia, lo que genera falsos positivos. Además, soluciones como RCMS (Robust Certificate Management System, que es un sistema propuesto específicamente para prevenir este tipo de ataques) requieren modificaciones en el software de los dispositivos cliente, lo cual es logísticamente inviable para una red global de acceso público [20].

2.3. Motivación y alcance

La relevancia de este trabajo reside en la demostración empírica de que la barrera de entrada para comprometer una red WPA Enterprise es alarmantemente baja. Mientras que los trabajos académicos suelen enfocarse en soluciones algorítmicas complejas [20], este enfoque ataca el verdadero eslabón débil: el factor humano y la accesibilidad del ataque.

- Demostración de la facilidad técnica: Este trabajo pretende solucionar la falta de concienciación demostrando que cualquier persona con conocimientos

técnicos básicos y un presupuesto muy reducido puede comprometer redes “seguras”. El uso de una Raspberry Pi 5 y una antena comercial (Alfa AWUS036ACM) junto con repositorios de código abierto de fácil acceso (como PatataWiFiEnterprise, que automatiza el uso de `hostapd-mana` y FreeRADIUS-WPE [28, 29]) demuestra que no se requiere equipamiento de ciberespionaje avanzado [19]. Todo está a disposición del público general.

- El enfoque en la concienciación por encima de la teoría: La principal aportación es evidenciar que la seguridad teórica de WPA2/WPA3-Enterprise es inútil si el *supplicant* (el usuario final) toma decisiones erróneas. Al montar un laboratorio aislado y simular la intercepción de credenciales, es posible aterrizar un concepto abstracto en un peligro tangible. Esto justifica la necesidad de que las instituciones (como las universidades) apliquen políticas estrictas de seguridad (como obligar al uso de `eduroam` CAT) [25, 19] y dejen de responsabilizar al usuario final sobre decisiones técnicas criptográficas [19].
- Aportación educativa: Finalmente, esta simulación va a proporcionar un entorno seguro y muy similar a una red `eduroam` corriente (laboratorio aislado con máquinas virtuales) que servirá como material educativo. Permite visualizar en tiempo real cómo un atacante puede extraer un *hash* NetNTLMv1 de MSCHAPv2 [30] o contraseñas en texto claro mediante métodos degradados (GTC, del inglés *Generic Token Card*) [19], lo que resulta vital para educar tanto a administradores de red como a usuarios.

3

Objetivos

Justificada ya la motivación del trabajo en el capítulo anterior, conviene fijar de forma precisa qué se persigue con él y cómo se ha llegado hasta el resultado. Este capítulo concreta primero el objetivo principal, demostrar de forma empírica que una red WPA2-Enterprise del tipo **eduroam** puede comprometerse mediante un *Evil Twin* sobre una plataforma portátil, autónoma y de bajo coste cuando el cliente no valida el certificado, desglosa después los objetivos secundarios, separados en los técnicos y los formativos, y cierra con la metodología seguida, desde la propuesta y sus requisitos hasta la planificación y la validación experimental.

3.1. Objetivo principal

El objetivo principal de este trabajo es demostrar de forma empírica, sobre un laboratorio completamente aislado, que una red Wi-Fi WPA2-Enterprise del tipo **eduroam** puede ser comprometida mediante un ataque *Evil Twin* cuando el dispositivo cliente no valida correctamente el certificado del servidor RADIUS. El trabajo no busca una vulnerabilidad nueva, pues el protocolo WPA2/WPA3-Enterprise es criptográficamente sólido; lo que evidencia es que toda su seguridad acaba dependiendo de una única decisión que casi siempre queda en manos del usuario final: confiar o no en el certificado que le presenta la red.

Para que esa demostración resulte significativa, el ataque se materializa sobre una plataforma portátil, autónoma y de bajo coste: una Raspberry Pi 5 con una antena Wi-Fi comercial, capaz de funcionar alimentada por una batería

externa dentro de, por poner un ejemplo, una mochila, sin necesidad de un equipo supervisado ni de intervención constante una vez encendida. Con ello se pretende mostrar que el coste y la dificultad de montar este ataque son hoy mínimos y que, por tanto, la amenaza es bastante más realista de lo que sugiere la percepción habitual [21, 19].

Sobre esa base, el objetivo principal se divide en dos vertientes. Por un lado, estudiar cómo responden los distintos clientes ante el ataque, desde los que entregan la contraseña en claro hasta los que lo rechazan por completo. Por otro, a partir de ese estudio, concienciar sobre una vulnerabilidad conocida y justificar las contramedidas que ya existen pero que rara vez se aplican de forma obligatoria. La tesis que se defiende es que la única protección realmente efectiva (la validación estricta del certificado del servidor, idealmente forzada mediante perfiles de configuración como **eduroam** CAT [18]) está disponible desde hace años y, aun así, sigue siendo opcional en la gran mayoría de instituciones.

3.2. Objetivos secundarios

Del objetivo principal se desprenden una serie de objetivos secundarios que conviene separar en dos grupos: los técnicos, que recogen lo que hay que construir y conseguir para que el experimento funcione, y los formativos, centrados en el aprendizaje personal que conlleva el trabajo.

3.2.1. Objetivos técnicos del proyecto

- Montar una red legítima y realista dentro del laboratorio: un punto de acceso WPA2-Enterprise emitido desde una máquina virtual Ubuntu Server con FreeRADIUS estándar y certificado propio, con el SSID **eduroam-tfg**, que representa la red institucional a la que la víctima se conecta de forma habitual y de la que aprende el SSID y el certificado.
- Construir la plataforma atacante portátil sobre una Raspberry Pi 5 partiendo de la herramienta PatataWiFiEnterprise [28], adaptándola a una arquitectura de dos radios físicas independientes (la radio interna de la Pi para la red de gestión y una antena Alfa AWUS036ACM para el *Evil Twin*) y actualizándola, ya que la versión original es de 2015 y arrastra dependencias obsoletas.
- Modernizar y reestructurar el *backend* de captura, que incluye sustituir el FreeRADIUS 2.x heredado por FreeRADIUS-WPE 3.2.5, la edición de captura de Joshua Wright mantenida en el repositorio de Kali. Así se evita compilar versiones antiguas de OpenSSL y se trabaja sobre una base actual y soportada.

- Capturar los dos tipos de credencial posibles: la contraseña en claro, forzando un *downgrade* a EAP-GTC dentro del túnel PEAP, y el *hash* NetNTLMv1, gracias al *fallback* automático a MSCHAPv2 cuando el cliente rechaza GTC. Las credenciales capturadas deben quedar en un formato directamente utilizable para su análisis *offline* (hashcat en modo 5500 o john).
- Automatizar todo el ciclo de vida para que la plataforma sea verdaderamente desatendida: tres servicios **systemd** encadenados (limpieza → gestión → ataque) que levantan ambas redes de forma automática unos treinta segundos después de arrancar, más una red de gestión WPA2-PSK (del inglés, *Pre-Shared Key*) independiente que permite administrar la Raspberry Pi de forma remota mientras el ataque está activo.
- Estudiar el comportamiento del cliente frente al *Evil Twin* documentando los tres escenarios observados: el cliente que acepta el certificado y GTC (contraseña en claro), el que acepta el certificado pero rechaza GTC y cae a MSCHAPv2 (*hash* crackeable *offline*) y el que tiene anclado el certificado legítimo mediante *CA pinning* y rechaza la conexión antes de abrir el túnel, único caso en el que el ataque no obtiene nada.

3.2.2. Objetivos formativos y de aprendizaje

- Profundizar en el funcionamiento de las redes Wi-Fi y en cómo ha evolucionado su seguridad, desde WEP hasta WPA3.
- Entender a fondo WPA2/WPA3-Enterprise, el estándar 802.1X y el papel del servidor RADIUS, así como sus diferencias con las redes domésticas WPA2-PSK.
- Comprender los distintos métodos EAP (PEAP, EAP-TTLS, EAP-TLS, GTC, MSCHAPv2) y el papel que juegan los certificados digitales y la validación del servidor como ancla de confianza de todo el sistema.
- Adquirir experiencia práctica en la auditoría de redes *enterprise*, manejando herramientas reales del ámbito (hostapd, FreeRADIUS-WPE, dnsmasq) y montando un entorno de pruebas de principio a fin.
- Aprender técnicas de *cracking offline* de credenciales (hashcat y john, ataques por diccionario, máscara y reglas) y entender por qué MSCHAPv2 sigue siendo recuperable pese a su antigüedad.
- Reforzar competencias transversales de administración de sistemas Linux: **systemd**, gestión de *drivers* inalámbricos, *scripting* y documentación reproducible de un proyecto técnico.

3.3. Metodología

Enunciados ya los objetivos del trabajo, conviene detenerse en cómo se ha llevado a cabo. Esta sección expone primero la propuesta de solución y los requisitos, funcionales y no funcionales, que el montaje debe cumplir para que el experimento sea válido, y aborda después la planificación del trabajo por fases junto con los criterios de validación con los que se comprueba que el ataque se reproduce como se espera.

3.3.1. Propuesta y requisitos

Para resolver el problema descrito, la propuesta arranca con un entorno de pruebas aislado y propio: una red WPA2-Enterprise privada del tipo **eduroam**, montada sobre una máquina virtual, que hace el papel de red legítima y permite experimentar sin atacar a nadie ni tocar infraestructura ajena. Ese laboratorio no ejecuta el ataque, simplemente es el blanco controlado sobre el que trabajar. La aportación central del trabajo es, a partir de él, desarrollar una plataforma de auditoría portátil sobre Raspberry Pi que despliegue un *Evil Twin* contra esa red, capture las credenciales del cliente que intenta conectarse y sirva a la vez para demostrar la vulnerabilidad y para concienciar sobre ella.

Para que el experimento sea válido, el montaje completo debe cumplir unos requisitos funcionales, que definen lo que tiene que conseguir:

- Montar la red privada que actuará como red legítima: un punto de acceso WPA2-Enterprise con el SSID **eduroam-tfg**, servido desde una máquina virtual con FreeRADIUS estándar y un certificado propio, que representa la red a la que el cliente se conecta de forma habitual.
- Emitir, desde la plataforma atacante, un punto de acceso con el mismo SSID que esa red legítima y levantar un servidor RADIUS bajo control del atacante que negocie el túnel PEAP con el cliente.
- Capturar las credenciales en sus dos formas, contraseña en claro mediante el *downgrade* a EAP-GTC y *hash* NetNTLMv1 mediante el *fallback* a MSCHAPv2, dejándolas listas para el análisis *offline*.
- Levantar tanto la red del ataque como la de gestión de forma automática y desatendida tras el encendido, sin intervención del operador.
- Permitir la administración remota del equipo mientras el ataque está en marcha, a través de una red de gestión independiente.

Y unos requisitos no funcionales, que marcan cómo debe comportarse:

- Portabilidad y autonomía, de modo que todo el sistema funcione alimentado por una batería externa y sin necesidad de un equipo supervisor.
- Accesible, apoyándose en hardware comercial y software libre de acceso público.
- Reproducibilidad, con una instalación repetible y documentada paso a paso.
- Confinamiento y seguridad, garantizando que este trabajo no afecta a ninguna red ni usuario ajeno, en coherencia con su carácter educativo y con el marco legal (Artículo 197 del Código Penal español).

3.3.2. Planificación y validación

El trabajo se ha abordado de forma incremental y experimental, y ha exigido un esfuerzo sostenido a lo largo de varios meses. Antes de tocar una sola línea de configuración hubo una etapa larga de documentación e investigación previa sobre WPA2-Enterprise, 802.1X, los métodos EAP, los certificados y los trabajos que ya habían demostrado este ataque [21, 19], sin la cual habría sido imposible tomar las decisiones de diseño con criterio. A esa fase de estudio le siguió una planificación cuidadosa, que se ha ido revisando y ajustando durante buena parte del curso a medida que las pruebas obligaban a replantear partes del montaje. El laboratorio actual es el resultado de numerosas iteraciones y de muchas horas de prueba y error.

Sobre esa base, el desarrollo se ha organizado en las siguientes fases:

1. Documentación y estado del arte. Estudio en profundidad de WPA2-Enterprise, 802.1X, los métodos EAP y los certificados, junto con la revisión de los trabajos previos que ya habían demostrado este ataque [21, 19] y de las herramientas de auditoría existentes. Fue la fase más larga y la que sentó las bases de todo lo demás.
2. Diseño del laboratorio. Definición de las dos partes del escenario (la red legítima y la plataforma atacante), elección del hardware y toma de las decisiones de diseño clave, como emplear dos radios físicas en lugar de *multi-BSSID* (por una limitación del *driver* del *chipset* MT7612U) o apoyarse en FreeRADIUS-WPE 3.x en vez de la versión 2.x obsoleta.
3. Implementación. Montaje de la red legítima sobre una máquina virtual, adaptación y modernización de PatataWiFi, configuración del RADIUS atacante con el *downgrade* a GTC y el *fallback* a MSCHAPv2, y automatización completa mediante los tres servicios `systemd`.

4. Validación experimental. Pruebas del ataque contra distintos clientes y configuraciones para comprobar que se reproducen los tres escenarios esperados y que las credenciales se capturan correctamente, seguidas del análisis *offline* de los *hashes* obtenidos.
5. Análisis y conclusiones. Caracterización de las contramedidas a partir de los resultados, discusión de la mitigación efectiva y redacción de la memoria.

Para saber si el trabajo avanzaba por buen camino se fijaron criterios de validación objetivos y comprobables sobre los propios registros del servidor RADIUS. Una captura de contraseña en claro se da por válida cuando aparece la línea **pap:** en el *log* de FreeRADIUS-WPE con usuario y contraseña, la captura de *hash* se confirma con la línea **mschap:** y su posterior crackeo *offline*, y la mitigación se considera demostrada cuando, ante un cliente con el certificado anclado, no se genera ninguna entrada en el *log* y el ataque queda neutralizado. Estos tres resultados, contrastados de forma repetible, son los que sostienen las conclusiones del trabajo.

Buena parte de ese tiempo se ha ido, además, en resolver los problemas que fueron surgiendo durante el montaje, como las colisiones de IP entre interfaces, las sesiones **tmux** huérfanas, el adaptador Alfa que tardaba en exponerse tras el arranque o los certificados caducados, entre otros, que se han resuelto y documentado uno a uno. El proceso de depuración forma parte también de la metodología y queda recogido para que el laboratorio sea reproducible por terceros, de hecho, todos los artefactos y los *scripts* de instalación se han publicado en un repositorio público [31] con el que reproducir el montaje paso a paso. La planificación temporal de todas estas fases, repartida a lo largo del curso, se resume en el cronograma del proyecto.

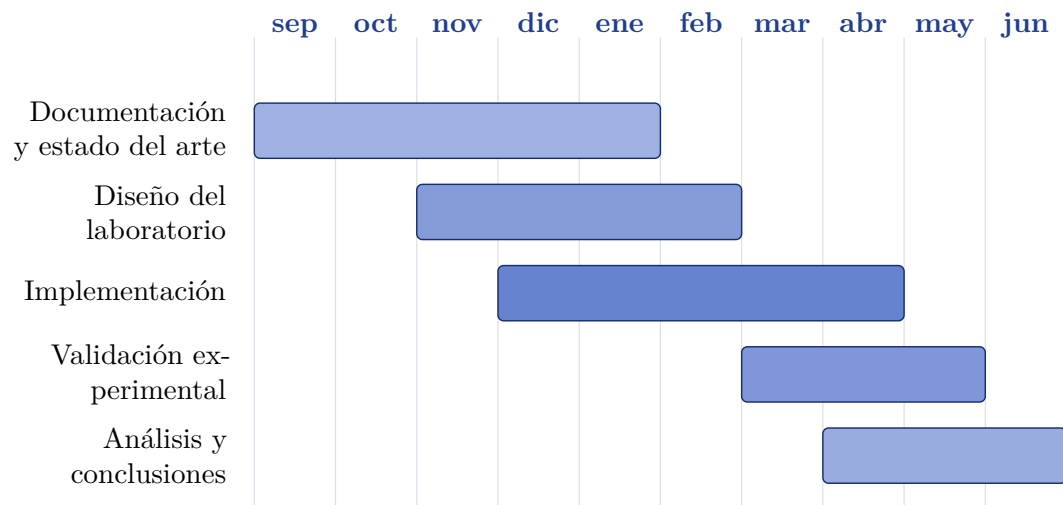


Figura 3.1: Cronograma de las fases del proyecto: distribución temporal de las cinco fases del trabajo a lo largo del curso 2025-2026. Las fases se solapan porque la planificación se revisó de forma incremental conforme avanzaba el desarrollo del laboratorio.

4

Implementación o descripción informática

Una vez justificados los objetivos y la metodología en los capítulos previos, este capítulo describe en detalle la construcción del laboratorio que da soporte al trabajo. El objetivo es documentar, paso a paso y de forma reproducible, cómo se ha montado un entorno que reproduce con fidelidad un ataque *Evil Twin* contra una red WPA2-Enterprise del tipo **eduroam**, partiendo del hardware disponible y de herramientas de software libre. Conviene insistir desde el principio en que el ataque que aquí se reproduce no constituye una vulnerabilidad nueva ni desconocida: se trata de una debilidad pública y documentada en la literatura, ligada a la ausencia de validación del certificado del servidor por parte del cliente, y que ya ha sido demostrada de forma experimental en entornos académicos reales [21, 19]. El propósito de su reproducción en este trabajo es estrictamente defensivo y de concienciación: evidenciar lo accesible que resulta el ataque para, sobre esa base, justificar las contramedidas existentes.

El capítulo se centra exclusivamente en la *construcción* del laboratorio, esto es, en el montaje y la configuración de cada uno de sus componentes y en las decisiones de diseño que los sustentan. La *ejecución* del ataque, las capturas reales de credenciales y la comparación del comportamiento de los distintos clientes víctima se abordan en el capítulo siguiente, dedicado a la validación experimental. A lo largo de la exposición, las decisiones de diseño se acompañan de su justificación, y los volcados completos de configuración se relegan a los apéndices para no entorpecer la lectura, manteniendo en el cuerpo únicamente los fragmentos imprescindibles para comprender cada apartado.

4.1. Visión general del laboratorio

El laboratorio se organiza en torno a tres elementos diferenciados: una red legítima, una plataforma atacante portátil y un dispositivo cliente que hace de víctima. Los dos primeros constituyen las dos partes que el autor ha construido y configurado por completo, mientras que el tercero es simplemente cualquier dispositivo que pueda autenticarse en la red legítima. Antes de entrar en el detalle de cada componente en las secciones siguientes, conviene presentar una visión de conjunto que permita situar todas las piezas y entender cómo encajan entre sí, tal como ilustra el diagrama de arquitectura general.

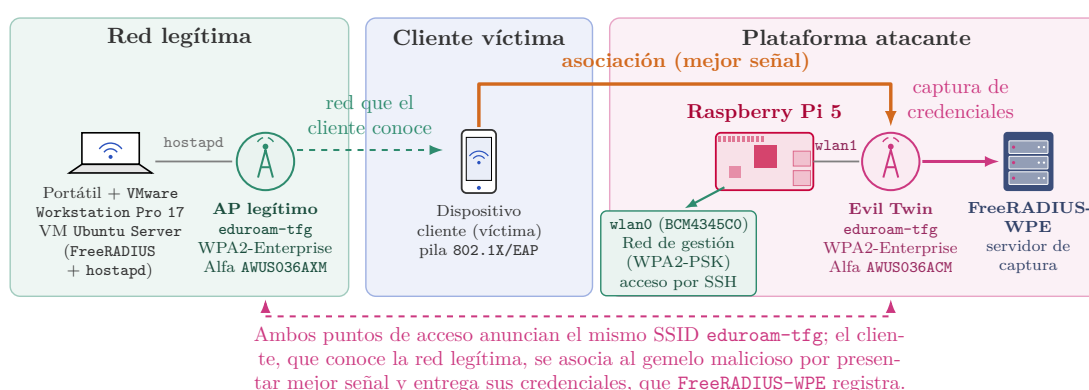


Figura 4.1: Arquitectura general del laboratorio de pruebas. A la izquierda, la **red legítima**: un portátil con VMware Workstation Pro 17 aloja la máquina virtual Ubuntu Server (FreeRADIUS + hostapd) que, a través de la antena Alfa AWUS036AXM, emite el punto de acceso eduroam-tfg (WPA2-Enterprise). En el centro, el **dispositivo cliente víctima**, con su pila 802.1X/EAP. A la derecha, la **plataforma atacante**: una Raspberry Pi 5 con dos radios (wlan0 (Broadcom BCM4345C0), reservada a la red de gestión (WPA2-PSK, acceso por SSH), y wlan1 (Alfa AWUS036ACM), que levanta el *Evil Twin* con el mismo SSID eduroam-tfg respaldada por FreeRADIUS-WPE. Aunque el cliente conoce la red legítima, acaba asociándose al gemelo malicioso por ofrecer mejor señal y entrega sus credenciales, que el servidor de captura registra. Nótese que los dos adaptadores Alfa son distintos: AWUS036AXM en el lado legítimo y AWUS036ACM en el atacante.

- La red legítima representa la red institucional sobre la que se quiere demostrar el ataque. Se materializa sobre el portátil del autor, donde se ejecuta VMware Workstation Pro 17 con una máquina virtual Ubuntu Server que actúa como punto de acceso WPA2-Enterprise completo: hostapd publica el SSID y FreeRADIUS desempeña el papel de servidor de autenticación 802.1X con un certificado y una autoridad certificadora propios. La emisión inalámbrica corre a cargo de un adaptador Alfa AWUS036AXM (*chipset* MediaTek MT7921AU, Wi-Fi 6) conectado a la máquina virtual por paso directo de

USB. Esta red simula la que el cliente víctima conoce y a la que se asocia de forma habitual, y de la que el dispositivo aprende tanto el nombre de la red como el método de autenticación. En este caso, sería el **eduroam** de su institución. Su construcción se detalla en la sección dedicada a la primera fase del laboratorio.

- La plataforma atacante es una Raspberry Pi 5 autónoma y portátil que despliega el *Evil Twin* propiamente dicho. Para evitar las limitaciones de emitir dos redes desde una única radio, la plataforma emplea dos radios físicas independientes: la radio interna de la Pi (Broadcom BCM4345C0, interfaz **wlan0**) sostiene una red de gestión WPA2-PSK que permite administrar el equipo de forma remota mientras el ataque está activo, y una antena externa Alfa AWUS036ACM (*chipset* MediaTek MT7612U, Wi-Fi 5, interfaz **wlan1**) emite el punto de acceso malicioso, con un servidor FreeRADIUS-WPE como motor de captura por detrás. Es importante no confundir ambos adaptadores Alfa: el modelo AXM pertenece a la red legítima y el modelo ACM, distinto, a la plataforma atacante. La construcción de esta plataforma se aborda en la sección correspondiente a la segunda fase.
- El cliente víctima es cualquier dispositivo que disponga de una pila 802.1X/EAP y se asocie al punto de acceso malicioso. El flujo del ataque, a alto nivel, es el siguiente: el cliente, que reconoce el nombre de la red, se asocia al *Evil Twin* atraído por su señal; a continuación se inicia la negociación PEAP y, si el cliente acepta el certificado que le presenta el servidor del atacante, se abre el túnel TLS; dentro de ese túnel el cliente entrega sus credenciales, que el servidor de captura registra para su posterior análisis. El comportamiento concreto del cliente ante este flujo, y los distintos resultados posibles, se estudian en el capítulo de resultados.

Conviene cerrar esta visión general subrayando el carácter aislado y ético del montaje, que condiciona todas las decisiones de diseño que siguen. El laboratorio es un entorno cerrado, físicamente separado de cualquier red en producción, de modo que en ningún momento se interactúa con la red **eduroam** real ni con usuarios ajenos al propio autor. El punto de acceso malicioso nunca emite el SSID **eduroam** a secas, sino la variante **eduroam-tfg**, con sufijo, precisamente para no suplantar el identificador de ninguna red institucional real. En coherencia con ello, en esta memoria no se reproducen credenciales reales ni *hashes* capturados, que se sustituyen sistemáticamente por marcadores del tipo <usuario>, <contraseña> o [SECRETO]. Estas medidas no son una simple formalidad: la realización de un ataque equivalente contra una red ajena, o sin el consentimiento explícito de su operador y de sus usuarios, está regulada expresamente en el Artículo 197 del Código Penal español relativo al descubrimiento y revelación de secretos, lo que refuerza la necesidad de mantener el experimento estrictamente confinado al entorno de pruebas del autor.

4.2. Entorno y tecnologías utilizadas

Situadas ya las piezas del laboratorio, antes de describir su montaje conviene precisar las tecnologías concretas que sustentan cada una de ellas y las razones que motivaron su elección. Esta sección describe y justifica las decisiones de hardware y software que sustentan ambas plataformas. La elección de cada componente se ha guiado por tres criterios: realismo respecto a un despliegue **eduroam** real, reproducibilidad de la cadena de montaje y disponibilidad efectiva del software sobre las plataformas actuales, donde varias de las herramientas históricas de este tipo de ataque han dejado de ser funcionales.

El cuadro general de componentes y versiones se recoge en la tabla resumen siguiente, que sirve de referencia para el resto del capítulo.

Componente	Versión	Rol en el laboratorio	Justificación
FreeRADIUS-WPE	3.2.5+dfsg-3kali1	Servidor de autenticación y captura (plataforma atacante)	Registra en claro las credenciales del túnel EAP; paquete moderno para aarch64 que evita la dependencia obsoleta libssl1.0 .
FreeRADIUS	3.x estándar	Servidor de autenticación (red legítima)	Reproduce un RADIUS institucional correcto (PEAP con MSCHAPv2) que nunca registra las credenciales.
hostapd	2.6 (compilado de wl-fi)	Punto de acceso del <i>Evil Twin</i> (plataforma atacante)	Estable con el <i>driver</i> mt76x2u ; la versión 2.10 provocaba desconexiones espurias.
hostapd	del sistema (repositorios)	Punto de acceso de la red de gestión (WPA2-PSK)	Suficiente para una red de gestión doméstica, sin extensiones.
Raspberry Pi OS	64 bits (bookworm / trixie , <i>kernel</i> 6.x)	Sistema operativo de la plataforma atacante	Base soportada y mantenida sobre aarch64 .
Ubuntu Server	<i>headless</i>	Sistema operativo de la red legítima	Sin NetworkManager ni entorno gráfico; aporta realismo de servidor RADIUS.
Alfa AWUS036AXM	MT7921AU / mt7921u (Wi-Fi 6)	Radio del AP (red legítima)	Adaptador USB con <i>passthrough</i> a la máquina virtual.
Alfa AWUS036ACM	MT7612U / mt76x2u (Wi-Fi 5)	Radio del <i>Evil Twin</i> , interfaz wlan1 (plataforma atacante)	Soporta modo AP y WPA-Enterprise; es un adaptador distinto del AXM.
dnsmasq	de los repositorios	Servidor DHCP y DNS (ambas plataformas)	Reparto de direcciones y resolución de nombres a los clientes.
systemd	del sistema	Orquestación del ciclo de vida (plataforma atacante)	Tres servicios encadenados; logra un arranque desatendido y reproducible.

Tabla 4.1: Componentes de hardware y software del laboratorio, con su versión, su rol y la justificación de su elección. La columna de versión recoge la rama o el *chipset/driver* concreto empleado en cada caso.

4.2.1. Hardware

Precisadas las tecnologías que sustentan el laboratorio, conviene partir del soporte físico sobre el que se asienta cada una de sus dos plataformas. Se describe tanto el hardware de la red legítima, montada sobre un portátil común con un adaptador Wi-Fi USB, como el de la plataforma atacante, una Raspberry Pi 5 equipada con dos radios Wi-Fi de funciones separadas.

Red legítima

La red legítima se ejecuta sobre el portátil del autor. En el siguiente apartado se detallan las herramientas de software que sustentan la red, pero a nivel de hardware cabe destacar que es un equipo común, con la única particularidad de contar con un adaptador Wi-Fi USB.

El papel de punto de acceso legítimo lo desempeña el adaptador Alfa AWUS036AXM, basado en el *chipset* MediaTek MT7921AU, conectado a la máquina virtual por paso USB y expuesto en el sistema invitado como la interfaz `wlx00c0cab7ef58`. Sobre este adaptador, `hostapd` levanta el SSID `eduroam-tfg` en WPA2-Enterprise.

Plataforma atacante

La plataforma atacante es una Raspberry Pi 5 de 8 GB de memoria (arquitectura `aarch64`), elegida por su uso común, su portabilidad y su capacidad de cómputo más que suficiente para compilar y ejecutar la pila de software del ataque de forma autónoma. La portabilidad resulta determinante: el realismo del ataque *Evil Twin* en espacios públicos reposa precisamente en que el atacante pueda acercar físicamente el equipo a la víctima, de modo que el suplicante de esta seleccione el punto de acceso malicioso por mejor intensidad de señal.

Un aspecto distintivo de esta plataforma es el empleo de dos radios físicas con funciones separadas, en lugar de multiplexar dos redes sobre una sola radio mediante BSSID (del inglés, *Basic Service Set Identifier*) múltiple. La radio interna de la Raspberry Pi 5, un Broadcom BCM4345C0 gestionado por el controlador `brcmfmac` y expuesto como `wlan0`, sostiene la red de gestión. La radio externa, un adaptador Alfa AWUS036ACM basado en el *chipset* MediaTek MT7612U (Wi-Fi 5) y gestionado por el controlador `mt76x2u`, se expone como `wlan1` y aloja el punto de acceso malicioso `eduroam-tfg`. Conviene subrayar que se emplean dos adaptadores Alfa *distintos*: el AWUS036AXM (MT7921AU, `mt7921u`) pertenece a la red legítima, mientras que la plataforma atacante emplea siempre el AWUS036ACM (MT7612U, `mt76x2u`).

La decisión de recurrir a dos radios físicas se justifica por una limitación concreta del controlador `mt76x2u` para el *chipset* MT7612U: no admite combinar WPA-Enterprise con BSSID múltiple sobre una misma radio [32]. Los intentos iniciales de levantar la red de gestión y el punto de acceso malicioso como dos BSS virtuales sobre el adaptador Alfa fracasaron, ya que el controlador exigía direcciones MAC (del inglés, *Media Access Control*) explícitas para el BSS virtual y reportaba el recurso como ocupado. Distribuir la gestión a la radio interna y el ataque a la radio externa resolvió de raíz el problema, además de aportar un beneficio operativo: la red de gestión y el punto de acceso malicioso operan en bandas y canales independientes, lo que evita interferencias entre la administración del equipo y el tráfico de ataque que se desea capturar. Además, esto permitiría que la plataforma atacante pueda ser configurada con otro tipo de adaptador Wi-Fi externo en caso de que el mencionado no estuviera disponible, siempre que el nuevo adaptador soporte modo AP y WPA-Enterprise.

4.2.2. Software

Descrito el soporte físico, se detalla a continuación el software sobre el que se asienta cada plataforma, componente a componente. Se parte del sistema operativo y las herramientas de base, se pasa por el punto de acceso `hostapd` y por el servidor de autenticación (donde divergen FreeRADIUS y FreeRADIUS-WPE), y se cierra con la infraestructura de clave pública generada con OpenSSL y con los servicios de red, la orquestación y la captura.

Sistema operativo y herramientas de base

La red legítima se ejecuta en el anfitrión Windows (Portátil del autor) y aloja una máquina virtual mediante VMware Workstation Pro 17. La elección de VMware Workstation Pro frente a alternativas como VirtualBox, Hyper-V o WSL2 responde a un único factor decisivo para este trabajo: el paso de dispositivos USB (*USB passthrough*) hacia la máquina virtual debe ser sólido y estable, ya que el punto de acceso legítimo se sirve de un adaptador Wi-Fi USB. Hyper-V se descartó por no ofrecer paso nativo de adaptadores Wi-Fi USB, y WSL2 por carecer de acceso real al hardware inalámbrico; además, VMware convive con Hyper-V a través de la *Windows Hypervisor Platform* (WHP), lo que evitó tener que desactivar la virtualización del anfitrión. La máquina virtual ejecuta Ubuntu Server en configuración *headless*, se optó deliberadamente por *Server* en lugar de *Desktop* por tres motivos: realismo (un servidor RADIUS de producción opera sin interfaz gráfica), menor consumo de recursos y, sobre todo, la ausencia de NetworkManager disputando el control de la interfaz inalámbrica, que en una instalación de escritorio interferiría con `hostapd`.

La plataforma atacante ejecuta Raspberry Pi OS de 64 bits (`bookworm/trixie`,

núcleo de la serie 6.x), sobre el que se apoya toda la pila del ataque. La administración del equipo se realiza por SSH (del inglés, *Secure Shell*) a través de la red de gestión, sin entorno gráfico. Sobre este sistema se emplean herramientas estándar cuyo papel no requiere mayor detalle: `iptables/nftables` para el filtrado y la traducción de direcciones, `macchanger` para la gestión de la dirección física del adaptador, y la utilidad `patch` para la aplicación idempotente de los parches descritos más adelante. Las tecnologías de propósito general (Linux, SSH) se dan por conocidas y no se documentan aquí.

Punto de acceso: `hostapd`

La emisión de todos los puntos de acceso recae sobre `hostapd`, tanto en la red legítima como en la plataforma atacante, pero en esta segunda, cabe destacar que es con dos binarios diferenciados. La red de gestión `PatataWiFi_mgmt` (WPA2-PSK) se levanta con el `hostapd` del sistema, instalado desde los repositorios de la distribución, que cumple sin necesidad de extensiones para una red de gestión doméstica. El punto de acceso malicioso, en cambio, se sirve de un `hostapd` 2.6 compilado desde el código fuente oficial de `wl.fi` [33], cuyo paquete se descarga verificando su resumen SHA-256 antes de la compilación, y al que se activan las opciones `CONFIG_LIBNL32` y `CONFIG_IEEE80211N` en su fichero de configuración de compilación.

La elección de la versión 2.6 en lugar de la 2.10 (la que arrastraba el proyecto del que parte esta plataforma) es una decisión deliberada y verificada en el laboratorio: la versión 2.10 producía desconexiones espurias con el controlador `mt76x2u`, mientras que la 2.6 se comporta de forma estable con ese mismo controlador. Se trata, por tanto, de una compatibilidad práctica probada empíricamente, no de una preferencia arbitraria.

Servidor de autenticación: FreeRADIUS frente a FreeRADIUS-WPE

El servidor de autenticación 802.1X es el componente donde más divergen las dos plataformas, y la divergencia es intencionada. En la red legítima se utiliza FreeRADIUS 3.x estándar, instalado desde los repositorios oficiales y configurado en `/etc/freeradius/3.0`, que reproduce fielmente el comportamiento de un servidor RADIUS institucional correcto: autentica por PEAP con MSCHAPv2 en el túnel interno y devuelve `Access-Accept` o `Access-Reject` sin registrar en ningún momento las credenciales del usuario.

En la plataforma atacante se emplea, por el contrario, FreeRADIUS-WPE (*Wireless Pwn Edition*) en su versión `3.2.5+dfsg-3kali1`, el paquete `freeradius-wpe` del repositorio `kali-rolling` [34]. La edición WPE incorpora un conjunto de parches que añaden el registro explícito de las credenciales

del túnel EAP en un fichero de captura [35], lo que constituye el núcleo del valor probatorio del ataque. Para que el repositorio Kali no interfiera con el sistema Debian de base, se añade con una prioridad de fijación (*pin*) de 50, de modo que sus paquetes solo se instalan bajo petición explícita y no actualizan automáticamente el resto del sistema.

La decisión de obtener FreeRADIUS-WPE 3.2.5 como paquete precompilado de Kali, en lugar de aplicar el parche WPE clásico sobre FreeRADIUS 2.1.12 (el camino que seguía el proyecto del que parte esta plataforma), responde a una incompatibilidad insalvable: la rama 2.1.12 exige `libssl1.0`, una biblioteca ya inexistente en `bookworm` sobre `aarch64`. Intentar resolver esa dependencia recuperando paquetes archivados, o portar el parche de 2010 sobre una versión moderna de OpenSSL, resultaba inviable o desproporcionado para los objetivos del trabajo. El paquete de Kali, mantenido y construido para `aarch64`, ofrece exactamente la misma funcionalidad de captura sobre una pila criptográfica actual, y es el camino habitual en la práctica profesional.

Infraestructura de clave pública: OpenSSL y los scripts de *bootstrap*

Ambas plataformas disponen de su propia infraestructura de clave pública (PKI, del inglés *Public Key Infrastructure*), generada con OpenSSL a través de los *scripts* de *bootstrap* que acompañan a FreeRADIUS. En la red legítima, la PKI define una autoridad certificadora propia con nombre común `TFG Loyola Root CA` y un certificado de servidor con nombre común `radius.tfg-loyola.local` (con caducidad el 16 de mayo de 2027), que simula la PKI institucional que un cliente correctamente configurado debería anclar. En la plataforma atacante, los *scripts* de *bootstrap* generan una PKI independiente cuyos certificados se redirigen mediante parche al material producido por el propio *bootstrap*. A diferencia de la red legítima, estos certificados no se personalizaban: conservan la identidad de ejemplo que los *scripts* de FreeRADIUS emiten por defecto, con una autoridad de nombre común `Example Certificate Authority` y un certificado de servidor con nombre común `Example Server Certificate`. El laboratorio es, en todo caso, un entorno de pruebas aislado, reproducido con fines exclusivamente educativos y de concienciación, sin suplantar ni interferir con ninguna red real.

Servicios de red, orquestación y captura

El reparto de direcciones a los clientes y la resolución de nombres corren a cargo de `dnsmasq`, que provee DHCP (del inglés, *Dynamic Host Configuration Protocol*) y DNS (del inglés, *Domain Name System*) tanto en la red legítima como en las dos redes de la plataforma atacante. En esta última, `dnsmasq` sirve el rango DHCP de la red de gestión `172.31.0.10–172.31.0.100` sobre la red `172.31.0.1/24`, con resolutores reenviados a `1.1.1.1` y `8.8.8.8`, mientras que

el punto de acceso malicioso opera sobre la red 10.0.0.1/24 en el canal 6.

La automatización del ciclo de vida del laboratorio se delega en `systemd`, mediante tres servicios propios de tipo `oneshot` (con `RemainAfterExit=yes`) encadenados por dependencias `After=/Requires=: tfg-cleanup` detiene procesos, baja las interfaces y recarga el controlador `mt76x2u`; `tfg-mgmt` levanta la red de gestión sobre `wlan0`; y `tfg-attack` espera a que aparezca `wlan1` y lanza el envoltorio que arranca el punto de acceso malicioso. Este último se apoya en `tmux` para multiplexar en una única sesión los cinco paneles de supervisión del ataque (`hostapd`, `dnsmasq`, el servidor RADIUS y el seguimiento en vivo de los registros de captura y autenticación). El conjunto se despliega mediante un instalador idempotente de once pasos que parte del proyecto *PatataWiFiEnterprise* [28] como base, lo moderniza y lo adapta a la pila descrita. La construcción detallada de estos servicios y del instalador se aborda en las secciones siguientes de este capítulo.

Finalmente, el análisis *offline* de las credenciales capturadas, que se desarrolla en el capítulo siguiente, se apoya en dos herramientas estándar de crackeo: `hashcat` [36], empleado en su modo 5500 (NetNTLMv1) para los *hashes* obtenidos por reversión a MSCHAPv2, y `john` (John the Ripper), capaz de procesar directamente las líneas en formato NetNTLMv1 del registro de captura. El crackeo se ejecuta fuera de la Raspberry Pi, en una máquina con aceleración por GPU (del inglés, *Graphics Processing Unit*), dado que la viabilidad práctica de este paso es precisamente lo que confiere impacto al ataque [30].

4.3. Construcción de la red legítima

Presentadas las tecnologías que sustentan el laboratorio, se aborda a continuación la primera de las dos fases de construcción: el levantamiento de la red legítima. La primera fase de la implementación consiste en levantar una red WPA2-Enterprise funcional que reproduzca, a escala de laboratorio, el modelo de confianza de un despliegue *eduroam* institucional. El objetivo de esta red es servir de referencia frente a la cual contrastar el punto de acceso malicioso de la Fase 2: solo disponiendo de un servidor de autenticación 802.1X real, con su propia infraestructura de clave pública y un método EAP representativo, es posible justificar de forma rigurosa por qué el ataque *Evil Twin* tiene éxito y qué configuración del cliente lo neutraliza. Por ello, todas las decisiones de diseño de esta sección persiguen el realismo funcional dentro de un entorno completamente aislado, empleando un dominio reservado y un direccionamiento privado que no colisiona con ninguna red real.

La arquitectura adoptada concentra el punto de acceso y el servidor de autenticación en una misma máquina virtual. En un despliegue institucional el autenticador (el punto de acceso) y el servidor de autenticación residen en equipos

distintos; aquí se colapsan en una sola VM por simplicidad, manteniendo no obstante la separación lógica de roles que exige el estándar 802.1X. Como ilustra el flujo de autenticación descrito más adelante, el suplicante del cliente dialoga con `hostapd`, que actúa como autenticador y delega la decisión de admisión en FreeRADIUS, el servidor de autenticación.

4.3.1. Preparación del anfitrión y la máquina virtual

Como fue mencionado anteriormente, el laboratorio se ejecuta sobre el portátil del autor (anfitrión Windows 11) en el que se instaló VMware Workstation Pro 17 como hipervisor de tipo 2.

Sobre el hipervisor se creó una máquina virtual con Ubuntu Server (*hostname* `tfg-ap-legitimo`), a la que se asignaron tres procesadores virtuales (CPU, del inglés *Central Processing Unit*), memoria suficiente para el *stack* de autenticación y un disco gestionado por LVM (del inglés, *Logical Volume Manager*). La VM se mantuvo *minimal*, sin Ubuntu Pro ni paquetes *snap* adicionales, reduciendo así la superficie de ataque y el ruido en los registros. La administración se realizó por SSH desde el anfitrión, lo que simplifica la captura de evidencias y el manejo de comandos extensos.

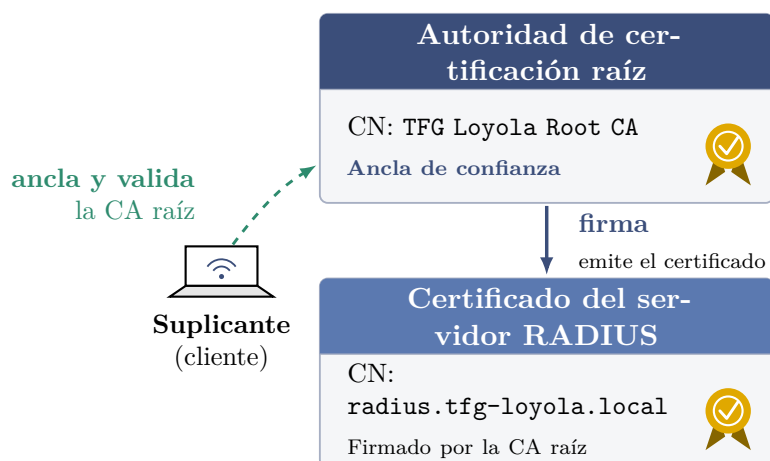
La interfaz de gestión de la VM (`ens33`, conectada al adaptador de red virtual de VMware) recibe una dirección del rango `192.168.28.0/24`. Esta red virtual está configurada en modo NAT (del inglés, *Network Address Translation*) para dar salida a Internet a los clientes de la red legítima. El *stack* del laboratorio se instaló mediante el gestor de paquetes de la distribución: `hostapd`, `freeradius` y sus utilidades, `openssl` y diversas herramientas de diagnóstico de red.

El elemento que confiere a la VM la capacidad de actuar como punto de acceso es el adaptador externo Alfa Networks AWUS036AXM (*chipset* MediaTek MT7921AU, gestionado por el *driver* `mt7921u`, Wi-Fi 6), redirigido a la máquina virtual mediante el *passthrough* USB de VMware. Una vez conectado, el sistema invitado reconoció el dispositivo y cargó el módulo del *kernel* correspondiente. Conviene anotar que el esquema de *predictable network interface names* de systemd renombró la interfaz de `wlan0` a `wlx00c0cab7ef58`. Se decidió conservar este nombre más largo en lugar de revertir el renombrado, por tratarse de un identificador persistente y reproducible incluso si se conectaran varios adaptadores; en consecuencia, `wlx00c0cab7ef58` es la interfaz que aparece a lo largo de toda la configuración del punto de acceso.

4.3.2. Servidor de autenticación

El núcleo de la red legítima es FreeRADIUS 3.x, instalado bajo `/etc/freeradius/3.0/`, que asume el papel de servidor de autenticación 802.1X. Su tarea consiste en validar las credenciales del usuario, gestionar el túnel TLS de PEAP y responder al autenticador con un **Access-Accept** o un **Access-Reject**.

Para reproducir la PKI institucional sobre la que se sustenta el modelo de confianza de **eduroam**, se generó una infraestructura de clave pública propia mediante los *scripts bootstrap* y el **Makefile** que FreeRADIUS incluye en su directorio de certificados. Previa personalización de los ficheros de descripción del sujeto (`ca.cnf` y `server.cnf`), se emitieron una autoridad de certificación raíz con `commonName` “TFG Loyola Root CA” y un certificado de servidor firmado por ella con `commonName` “radius.tfg-loyola.local”, este último con vencimiento el 16 de mayo de 2027. La cadena resultante se verificó con `openssl` para confirmar que el emisor del certificado de servidor es, efectivamente, la CA raíz del laboratorio.



El suplicante correctamente configurado ancla esta CA como única raíz de confianza y verifica que el nombre del certificado coincide con `radius.tfg-loyola.local`; así rechaza cualquier certificado que no esté firmado por ella. Eludir esa validación es, precisamente, el objetivo del ataque *Evil Twin*.

Figura 4.2: Cadena de certificados de la red legítima: la CA raíz “TFG Loyola Root CA” firma el certificado del servidor RADIUS `radius.tfg-loyola.local`. Un suplicante correctamente configurado debe anclar y validar esa CA (y comprobar el nombre del servidor) como ancla de confianza; esa validación es, justamente, la que el ataque *Evil Twin* de la fase siguiente busca eludir.

El papel de este certificado de servidor es central para todo el trabajo. En PEAP-MSCHAPv2 el cliente no presenta certificado propio, la única garantía de que está hablando con el servidor RADIUS legítimo, y no con un impostor,

es el certificado que el servidor exhibe al inicio del túnel TLS. Dicho certificado constituye la credencial que un atacante no puede reproducir sin la clave privada de la CA institucional, que es la que opera como verdadera ancla de confianza del sistema. Por ello, un suplicante correctamente configurado no debe limitarse a comprobar que la cadena de certificación valida correctamente hasta esa CA de confianza, sino también que el `commonName`/SAN (del inglés, *Subject Alternative Name*) del servidor coincide con el esperado (`radius.tfg-loyola.local`) y que la CA emisora es exactamente la institucional. Anclar estos tres elementos es precisamente lo que herramientas de aprovisionamiento como `eduroam` CAT [18] automatizan en los dispositivos de los usuarios, y es la condición que el ataque de la Fase 2 buscará eludir.

La configuración funcional de FreeRADIUS comprendió tres ajustes. En primer lugar, se dio de alta un usuario de prueba en el fichero `users`, con credencial almacenada en claro para el laboratorio (en esta memoria se referencia como `<usuario> / <contraseña>`, sin reproducir los valores reales). En un entorno de producción, esta comprobación de credenciales no se realiza contra un fichero plano, sino que se delega habitualmente en un *backend* externo (típicamente un directorio LDAP o una base de datos SQL (del inglés, *Structured Query Language*)) que el servidor consulta en cada solicitud de autenticación. En el laboratorio, sin embargo, se optó por el fichero `users` por simplicidad, dado que no altera en modo alguno la superficie de ataque estudiada. En segundo lugar, se declaró el cliente RADIUS (el NAS, del inglés *Network Access Server*) en `clients.conf`: dado que `hostapd` y FreeRADIUS comparten máquina, el cliente es la dirección de bucle local `127.0.0.1`, protegida por un secreto compartido robusto (`[SECRETO]`) conocido únicamente por el autenticador y el servidor, nunca por los clientes Wi-Fi. En tercer lugar, en `mods-enabled/eap` se fijó el método EAP por defecto a PEAP con MSCHAPv2 como método interno, que es el mecanismo histórico de `eduroam` y el vector sobre el que se construirá el ataque.

La configuración se validó arrancando FreeRADIUS en modo depuración (`freeradius -X`) hasta obtener el mensaje `Ready to process requests`, y comprobando la lógica de autenticación de extremo a extremo con `radtest`: una credencial válida produjo un `Access-Accept` con el `Message-Authenticator` esperado, mientras que una contraseña incorrecta y un usuario inexistente devolvieron sendos `Access-Reject`. Estas pruebas confirman que el servidor de autenticación opera correctamente antes de exponer la red por radio.

4.3.3. Punto de acceso con hostapd

El componente `hostapd` convierte el adaptador Alfa en un punto de acceso WPA2-Enterprise y desempeña el rol de autenticador en el esquema 802.1X. Su fichero de configuración (`/etc/hostapd/hostapd.conf`) define la interfaz física `wlx00c0cab7ef58` sobre el *driver* `nl80211`, publica el SSID `eduroam-tfg` y opera

en la banda de 2,4 GHz, canal 6. Se eligió 2,4 GHz por su compatibilidad universal con los suplicantes y porque las bandas de 5 GHz del *chipset* aparecían marcadas como (no IR) en el dominio regulatorio español, lo que impide iniciar transmisión en ellas sin validación previa de canales DFS (del inglés, *Dynamic Frequency Selection*).

La seguridad se configura en modo empresa y no en modo personal: la combinación `wpa=2` con `wpa_key_mgmt=WPA-EAP`, `rsn_pairwise=CCMP` e `ieee8021x=1` establece WPA2-Enterprise con cifrado por sesión y autenticación delegada. En lugar de una clave precompartida, `hostapd` reenvía las tramas EAP del suplicante hacia el servidor de autenticación; a tal efecto, las directivas `auth_server_addr` y `acct_server_addr` apuntan a 127.0.0.1 (puertos 1812 de autenticación y 1813 de *accounting*), con el secreto compartido [SECRETO] que enlaza el autenticador con FreeRADIUS. El siguiente fragmento recoge el bloque que define la identidad de la red y su modelo de seguridad; el contenido íntegro del fichero se reproduce en el apéndice.

```
interface=wlx00c0cab7ef58
driver=nl80211

ssid=eduroam-tfg
hw_mode=g
channel=6

auth_algs=1
wpa=2
wpa_key_mgmt=WPA-EAP
rsn_pairwise=CCMP
ieee8021x=1

auth_server_addr=127.0.0.1
auth_server_port=1812
auth_server_shared_secret=[SECRETO]
```

Código 4.1: Bloque principal de `hostapd.conf`

El conjunto materializa el flujo de autenticación 802.1X/EAP característico de **eduroam**: el suplicante del cliente establece un túnel TLS con el servidor (en el que este presenta su certificado), y dentro de dicho túnel cifrado viajan las credenciales MSCHAPv2 hasta el servidor virtual interno de FreeRADIUS (*inner-tunnel*). Solo si la autenticación tiene éxito el autenticador habilita el puerto y se completa el *handshake* de claves de WPA2.

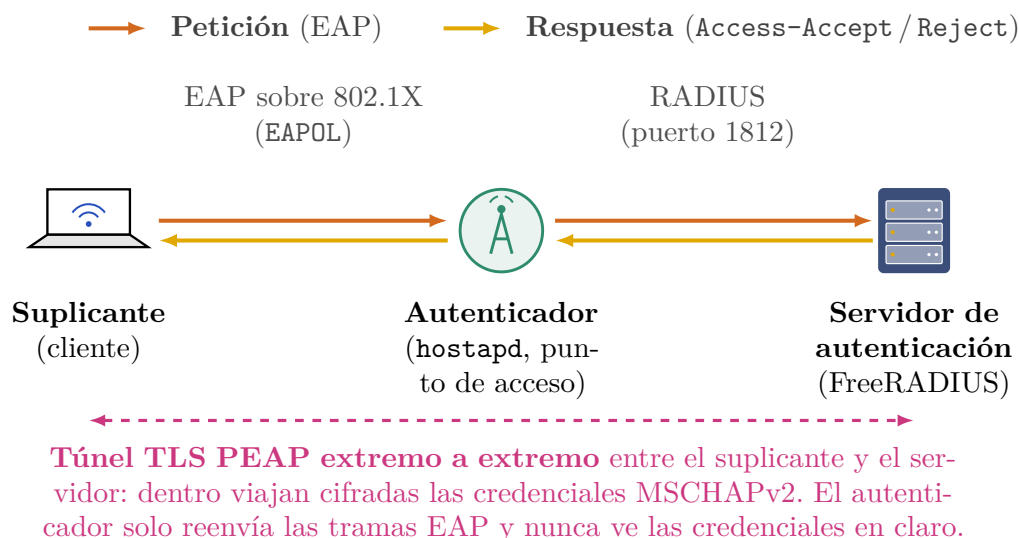


Figura 4.3: Flujo de autenticación 802.1X/EAP de la red legítima. El suplicante del cliente dialoga con el autenticador (`hostapd`), que delega la decisión en el servidor de autenticación (FreeRADIUS); dentro de un túnel TLS PEAP extremo a extremo entre suplicante y servidor viajan cifradas las credenciales MSCHAPv2, y solo si la autenticación tiene éxito el autenticador abre el puerto y se completa el *handshake* de claves de WPA2. El certificado del servidor es la única garantía de identidad de la que dispone el cliente.

La puesta en marcha de `hostapd` reveló un comportamiento inestable del *driver* `mt7921u` en los *kernels* recientes: si la interfaz inalámbrica quedaba en un estado de modo AP residual entre arranques, el *daemon* terminaba en *segmentation fault*. La solución, descrita en el apartado de automatización, consistió en restablecer la interfaz a modo gestionado antes de cada arranque del punto de acceso. Con la interfaz preparada, `hostapd` alcanza el estado `AP-ENABLED` y la red `eduroam-tfg` queda al aire.

4.3.4. Servicios de red: DHCP, NAT y reenvío IP

Para que un cliente autenticado pueda asociarse y, además, obtener direccionamiento y navegar, la red legítima incorpora una capa de servicios de red sobre la interfaz del punto de acceso. Se eligió el segmento privado `10.50.0.0/24` para no colisionar con la red doméstica del autor, con la red virtual de VMware (`192.168.28.0/24`), ni con el direccionamiento del campus. A la interfaz `wlx00c0cab7ef58` se le asigna la dirección de puerta de enlace `10.50.0.1`.

El reparto de direcciones y la resolución de nombres se delegan en `dnsmasq`, configurado para escuchar exclusivamente en la interfaz inalámbrica (con `bind-`

`interfaces`, evitando así un conflicto con `systemd-resolved` en el puerto 53). El servidor entrega direcciones del rango 10.50.0.10–10.50.0.100 mediante DHCP, anunciando como puerta de enlace y como servidor DNS la propia 10.50.0.1, y reenvía las consultas a servidores DNS públicos (1.1.1.1 y 8.8.8.8).

La salida a Internet se proporciona mediante traducción de direcciones (NAT) con `iptables`. Se habilitó el reenvío IP del *kernel* de forma permanente (`net.ipv4.ip_forward=1` en `/etc/sysctl.d/99-tfg-forward.conf`) y se definieron una regla de enmascaramiento (MASQUERADE) sobre la interfaz de salida `ens33` y reglas de FORWARD bidireccionales entre la red de clientes y dicha interfaz, persistidas con `iptables-persistent`. De este modo, el tráfico de los clientes de `eduroam-tfg` se traduce a la dirección de la VM al salir hacia Internet.

Dotar de conectividad real a la red legítima es una decisión de diseño con repercusión directa en la fidelidad del experimento: reproduce el comportamiento de un despliegue `eduroam` institucional, en el que un cliente que se autentica correctamente obtiene además salida a Internet, y permite validar de extremo a extremo el funcionamiento de la red legítima (asociación, autenticación 802.1X, direccionamiento y navegación), tal como se documenta en el apartado de validación.

4.3.5. Automatización con `systemd`

Para que la red legítima se comporte como un despliegue institucional que se levanta de forma autónoma tras el arranque, y para neutralizar la inestabilidad del *driver*, se orquestaron los componentes mediante unidades de `systemd` con dependencias explícitas, en lugar de lanzarlos a mano. El orden de arranque resultante es: preparación de la interfaz → servidor RADIUS operativo → punto de acceso → direccionamiento y servicios de red.

La unidad auxiliar `wlan-prepare.service` (de tipo `oneshot` con `RemainAfterExit`) se ejecuta antes que `hostapd` y restablece la antena a un estado limpio: desbloquea la radio con `rkill` y aplica el ciclo de reinicio de la interfaz (bajar → modo gestionado → subir → bajar) que evita el *segmentation fault* descrito anteriormente. Sobre el servicio de `hostapd` se aplicó un *drop-in override* que declara las dependencias `Wants/After` respecto a `freeradius.service` y `wlan-prepare.service`, garantizando que el punto de acceso solo arranque cuando el servidor de autenticación y la interfaz están listos; el mismo *override* añade `Restart=on-failure` con `RestartSec=5`, lo que confiere autorrecuperación frente a los fallos esporádicos del *driver*. Por último, la unidad `wlan-ip.service` asigna como puerta de enlace la dirección 10.50.0.1 a la interfaz y se vincula con `Bindsto` a `hostapd`, de manera que la dirección se reasigna de forma coherente si el punto de acceso se reinicia.

Tras un reinicio completo de la VM se verificó que los cinco servicios implicados

(FreeRADIUS, `wlan-prepare`, `hostapd`, `wlan-ip` y `dnsmasq`) arrancaban en el orden previsto y que la red `eduroam-tfg` quedaba operativa sin intervención manual. Durante el desarrollo se conservó, no obstante, la posibilidad de detener el servicio gestionado y ejecutar `hostapd` en primer plano con verbosidad máxima (`-dd`), modo que resulta imprescindible para revisar la negociación EAP al depurar.

4.3.6. Validación de la red legítima

La validación definitiva de la red legítima se realizó con un cliente real: un iPhone del autor que detectó la red `eduroam-tfg` e inició la asociación por PEAP-MSCHAPv2. Al carecer el dispositivo de un perfil de aprovisionamiento que anclase la CA esperada y el nombre del servidor, iOS mostró el *prompt* de confirmación del certificado, indicando el `commonName` del servidor (`radius.tfg-loyola.local`), su emisor (“TFG Loyola Root CA”) y la fecha de caducidad. Tras aceptar dicho *prompt*, la negociación EAP se completó correctamente (registrada por `hostapd` como *EAP type 25*, esto es, PEAP), el servidor devolvió `Access-Accept` y se cerró el *handshake* de claves de WPA2.

A continuación, el cliente obtuvo por DHCP una dirección del segmento 10.50.0.0/24 y, a través de las reglas de NAT, alcanzó Internet: los registros de `dnsmasq` reflejaron las consultas DNS reales del dispositivo y el contador de la regla `MASQUERADE` confirmó tráfico saliente traducido. Con ello quedó demostrado el funcionamiento de extremo a extremo de la red legítima: asociación segura, autenticación 802.1X, direccionamiento dinámico y salida a Internet.

Esta validación reviste, además, un valor conceptual que conecta con el resto del trabajo. La aceptación manual del *prompt* del certificado por parte del usuario es exactamente el punto de fallo humano que explota el ataque *Evil Twin* de la Fase 2: si el suplicante hubiera tenido anclada la CA y el nombre del servidor esperados, no habría existido *prompt* alguno que aceptar y un certificado distinto al legítimo habría sido rechazado de forma automática, esto, será explicado más en detalle posteriormente.

La red legítima queda así construida y verificada, y sirve de base de comparación para la plataforma atacante descrita en la sección siguiente.

4.4. Construcción de la plataforma atacante portátil

Frente a la red legítima de la fase anterior, que reproduce el despliegue real de un operador `eduroam`, esta segunda fase materializa la posición del adversario. El objetivo es disponer de una plataforma autónoma, alimentada por batería y repro-

ducible, capaz de levantar de forma automática un punto de acceso gemelo (*Evil Twin*) que suplante al SSID legítimo y de actuar como servidor de autenticación malicioso. La plataforma se construye sobre una Raspberry Pi 5 (8 GB, `aarch64`, Raspberry Pi OS *bookworm*) y un adaptador Wi-Fi USB Alfa AWUS036ACM basado en el *chipset* MediaTek MT7612U. Todo su ciclo de vida queda gobernado por servicios `systemd` encadenados, de modo que las dos redes quedan al aire en torno a treinta segundos tras el arranque sin intervención del operador.

El diseño no parte de cero. Toma como base el proyecto PatataWiFiEnterprise de Jesús Antón [28], una herramienta de impersonación de redes WPA-Enterprise pensada precisamente para este tipo de demostraciones, y la moderniza para que funcione sobre el hardware y las versiones de software disponibles en 2026. Las decisiones de adaptación se documentan a continuación, justificando en cada caso por qué la solución original resultaba inviable y qué la sustituye. La automatización completa, los *scripts* de orquestación y los volcados íntegros de configuración se recogen en el apéndice correspondiente y se publican, junto con los *scripts* de instalación, en el repositorio público del proyecto [31], preparado para que cualquiera pueda reproducir el laboratorio. Aquí se describe únicamente el porqué de cada componente.

4.4.1. PatataWiFiEnterprise y necesidad de modernización

PatataWiFiEnterprise [28] actúa como orquestador: no contiene en sí mismo las herramientas de ataque, sino que coordina tres piezas independientes (`hostapd` para levantar el punto de acceso, `dnsmasq` para repartir direcciones por DHCP y un servidor RADIUS para procesar la autenticación 802.1X/EAP y capturar las credenciales) y las multiplexa en una sesión `tmux` de varios paneles para su supervisión en tiempo real. Su *script* de instalación [29] compila y configura cada uno de estos componentes y registra el arranque automático mediante una entrada `crontab` de tipo `@reboot`. Este patrón de orquestación, los *scripts* de arranque y la idea de la red de gestión separada se heredan directamente del proyecto original.

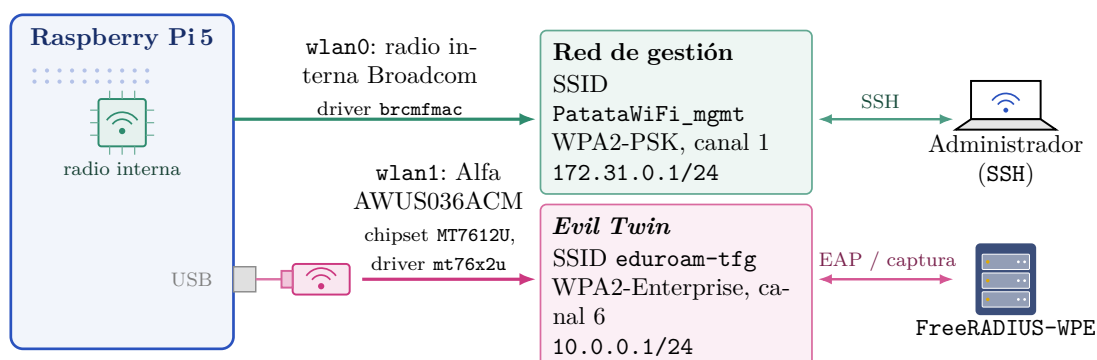
El problema es que la versión original fija un servidor de autenticación obsoleto. El instalador descarga y compila FreeRADIUS-WPE en su versión 2.1.12, una rama de 2011 que, en el momento de su compilación, depende de la biblioteca criptográfica `libssl1.0`. Dicha biblioteca fue retirada de las distribuciones modernas: en Raspberry Pi OS *bookworm* sobre `aarch64` ya no existe en los repositorios. El proyecto original sorteaba esto descargando manualmente paquetes `.deb` de `archive.debian.org` correspondientes a una versión muy anterior de Debian, una práctica frágil que introduce una biblioteca sin soporte de seguridad en el sistema y que no garantiza la compatibilidad de enlazado con el resto de paquetes actuales. Reproducir esa cadena de dependencias caducadas se descartó por inviable y por contradecir el principio de un laboratorio reproducible. Por

ello, la modernización sustituye ambos componentes históricos por dos piezas mantenidas: FreeRADIUS-WPE 3.2.5 y `hostapd` 2.6, descritos en los apartados siguientes.

4.4.2. Arquitectura de dos radios

La diferencia estructural más importante respecto al proyecto original es el abandono del esquema *multi-BSSID* sobre una única radio en favor de dos radios físicas independientes. El proyecto original levanta tanto la red de gestión como la red de ataque como dos BSS virtuales (`wlan1` y `wlan1_1`) sobre el mismo adaptador. Sobre el adaptador externo empleado en este laboratorio (un Alfa AWUS036ACM, con *chipset* MediaTek MT7612U y *driver* `mt76x2u` [32]), esa configuración resulta inestable: el *driver* es muy restrictivo con las direcciones MAC de los BSS virtuales y, sobre todo, no soporta de forma fiable la combinación de WPA-Enterprise con *multi-BSSID* en una sola radio. En las pruebas, el segundo BSS, el que requería la pila completa 802.1X/EAP, fallaba en la fase de asociación mientras que el BSS de gestión, mucho más ligero, sí funcionaba.

La solución adoptada reparte las dos funciones entre las dos radios físicas de que dispone la plataforma. La radio interna de la Raspberry Pi 5 (Broadcom BCM4345C0, *driver* `brcmfmac`), expuesta como `wlan0`, aloja la red de gestión WPA2-PSK en el canal 1 con direccionamiento `172.31.0.1/24`. El adaptador externo Alfa, expuesto como `wlan1`, queda reservado en exclusiva para el *Evil Twin* WPA2-Enterprise en el canal 6 con direccionamiento `10.0.0.1/24`, y es la única radio que ejecuta la pila EAP. De este modo, cada radio atiende un único tipo de red y se elimina por completo la limitación del *driver*, a costa de necesitar un segundo adaptador. La separación además resuelve un conflicto de direccionamiento que arrastraba la configuración original, en la que ambas redes compartían la misma dirección de *gateway*. La distribución de funciones entre ambas radios se resume en el diagrama de arquitectura.



Dos radios físicas independientes, cada una en su propia banda y canal (canal 1 y canal 6), sin interferencia mutua. Aislar el *Evil Twin* en wlan1 elude la limitación del driver `mt76x2u` con multi-BSSID en modo Enterprise.

Figura 4.4: Las dos radios físicas independientes de la plataforma atacante. La radio interna de la Raspberry Pi 5 (`wlan0`, Broadcom BCM4345C0, *driver* `brcmfmac`) sostiene la red de gestión WPA2-PSK `PatataWiFi_mgmt` (canal 1, 172.31.0.1/24), empleada para administrar el equipo por SSH; el adaptador externo Alfa AWUS036ACM (`wlan1`, *chipset* MT7612U, *driver* `mt76x2u`) emite en exclusiva el *Evil Twin* WPA2-Enterprise `eduroam-tfg` (canal 6, 10.0.0.1/24), con FreeRADIUS-WPE como motor de captura por detrás. Repartir cada SSID en una radio distinta elude la limitación del *driver* `mt76x2u` con *multi-BSSID* en modo Enterprise.

4.4.3. Backend de captura: FreeRADIUS-WPE 3.2.5

El servidor de autenticación es el núcleo del ataque: es la pieza que presenta el certificado del servidor durante la negociación TLS y la que registra las credenciales que el cliente entrega dentro del túnel. En lugar de compilar la rama histórica 2.1.12, este laboratorio emplea FreeRADIUS-WPE 3.2.5 [35], una revisión moderna de la edición *Wireless Pwn Edition* (que extiende FreeRADIUS para volcar a un *log* las credenciales observadas) empaquetada por el proyecto Kali como `freeradius-wpe` [34]. La instalación no añade el repositorio de Kali como fuente ordinaria, sino con una prioridad de *pin* deliberadamente baja (valor 50). Dado que APT solo instala automáticamente paquetes de repositorios con prioridad igual o superior a 500, este valor mantiene los paquetes de Kali invisibles para las actualizaciones del sistema y obliga a solicitarlos de forma explícita; así se obtiene el único paquete necesario sin convertir una instalación de Raspberry Pi OS en un híbrido inestable con Kali.

Sobre la configuración instalada por el paquete se aplican dos parches mínimos. El primero, sobre `radiusd.conf`, comenta las directivas `user` y `group` que harían que el *daemon* se ejecutase con la cuenta sin privilegios `freerad-wpe`; al correr

como `root`, el servidor puede gestionar las interfaces y los recursos que necesita en este entorno de laboratorio. El segundo, sobre el módulo `eap`, redirige las rutas del certificado de servidor, su clave privada y el certificado de CA desde los certificados *snakeoil* del sistema hacia los generados específicamente para el ataque mediante `make bootstrap`, descritos en el apartado siguiente.

4.4.4. El Evil Twin: `hostapd` 2.6 y certificado del atacante

El punto de acceso malicioso lo levanta `hostapd` compilado a partir de la versión 2.6 publicada en `w1.fi` [33], cuya integridad se verifica por suma `sha256` antes de compilar. Se descartó la versión 2.10 que empleaba el proyecto original porque, sobre el *driver* `mt76x2u` y los *kernel* de la serie 6.x, provocaba desconexiones espurias del adaptador Alfa que comprometían la estabilidad del punto de acceso. La compilación activa únicamente las opciones imprescindibles (el enlazado con `libnl` 3.2 y el soporte de 802.11n) mediante un parche sobre el fichero de configuración de compilación.

El punto de acceso se anuncia con el SSID `eduroam-tfg`, idéntico salvo el sufijo al SSID legítimo que se desea suplantar, y se configura como WPA2-Enterprise con cifrado CCMP (del inglés, *Counter Mode CBC-MAC Protocol*) y *management frame protection* (802.11w) activos, replicando los parámetros que un cliente esperaría de la red real. La eficacia del engaño descansa en el certificado que el servidor presenta durante la negociación TLS. Este certificado no se personalizó para el laboratorio: conserva la identidad de ejemplo que los *scripts* de *bootstrap* emiten por defecto, con CN “Example Certificate Authority” en la autoridad de certificación y CN “Example Server Certificate” en el certificado de servidor. Un cliente correctamente configurado debería rechazar este certificado por no estar firmado por la CA legítima de su institución, y ni siquiera una identidad tan genérica y manifiestamente ajena a la universidad lo delata ante un cliente mal configurado. El ataque solo prospera frente a clientes que no validan adecuadamente la identidad del servidor RADIUS, que es precisamente la mala configuración sobre la que este trabajo pretende concienciar. Un atacante experimentado y con intenciones maliciosas iría más lejos: replicaría en su certificado los datos del certificado legítimo (nombre común, emisor y demás campos del sujeto) para parecerse a él lo máximo posible y dificultar todavía más que la víctima sea consciente del engaño.

4.4.5. Downgrade a EAP-GTC y fallback a MSCHAPv2

Dentro del túnel PEAP, la configuración del servidor introduce un ataque de *downgrade* sobre el método de autenticación interno. Por defecto, el bloque PEAP de FreeRADIUS ofrece como método interno MSCHAPv2. El parche

`eap-gtc-downgrade.patch` altera ese valor por defecto para ofrecer en primer lugar EAP-GTC:

```
- default_eap_type = mschapv2
+ default_eap_type = gtc
```

Código 4.2: `eap-gtc-downgrade.patch`

La motivación es que cada método interno entrega la credencial en una forma distinta, el atacante, naturalmente, prefiere la más débil. Con EAP-GTC, el cliente transmite la contraseña en claro dentro del túnel TLS; si el cliente acepta el certificado del servidor y acepta negociar GTC, la contraseña queda registrada directamente, sin necesidad de ningún cómputo posterior, y el servidor (que conoce ahora la credencial) puede completar la asociación al punto de acceso gemelo. Es el desenlace más favorable para el atacante.

Muchos clientes, sin embargo, rechazan EAP-GTC al esperar MSCHAPv2 y responden con un mensaje **EAP-NAK** proponiendo este último método. Ante esa negativa, el servidor cae automáticamente a MSCHAPv2. En ese caso no se obtiene la contraseña en claro, sino el par reto-respuesta (*challenge/response*) del intercambio, del que se deriva un *hash* de tipo NetNTLMv1 susceptible de ataque por diccionario o fuerza bruta fuera de línea, aprovechando la conocida debilidad criptográfica de MS-CHAPv2 [37]. La combinación de ambos caminos garantiza que, frente a un cliente que acepta el certificado, el atacante obtenga material aprovechable en cualquiera de los dos casos: la credencial en claro si el cliente cede a GTC, o un *hash crackeable* si insiste en MSCHAPv2. El único cliente que frustra por completo el ataque es el que aplica una validación estricta de la CA y rechaza el certificado exterior, de modo que el túnel ni siquiera llega a abrirse y no se produce captura alguna. Las tres ramas posibles de este flujo se ilustran en el diagrama del método EAP.

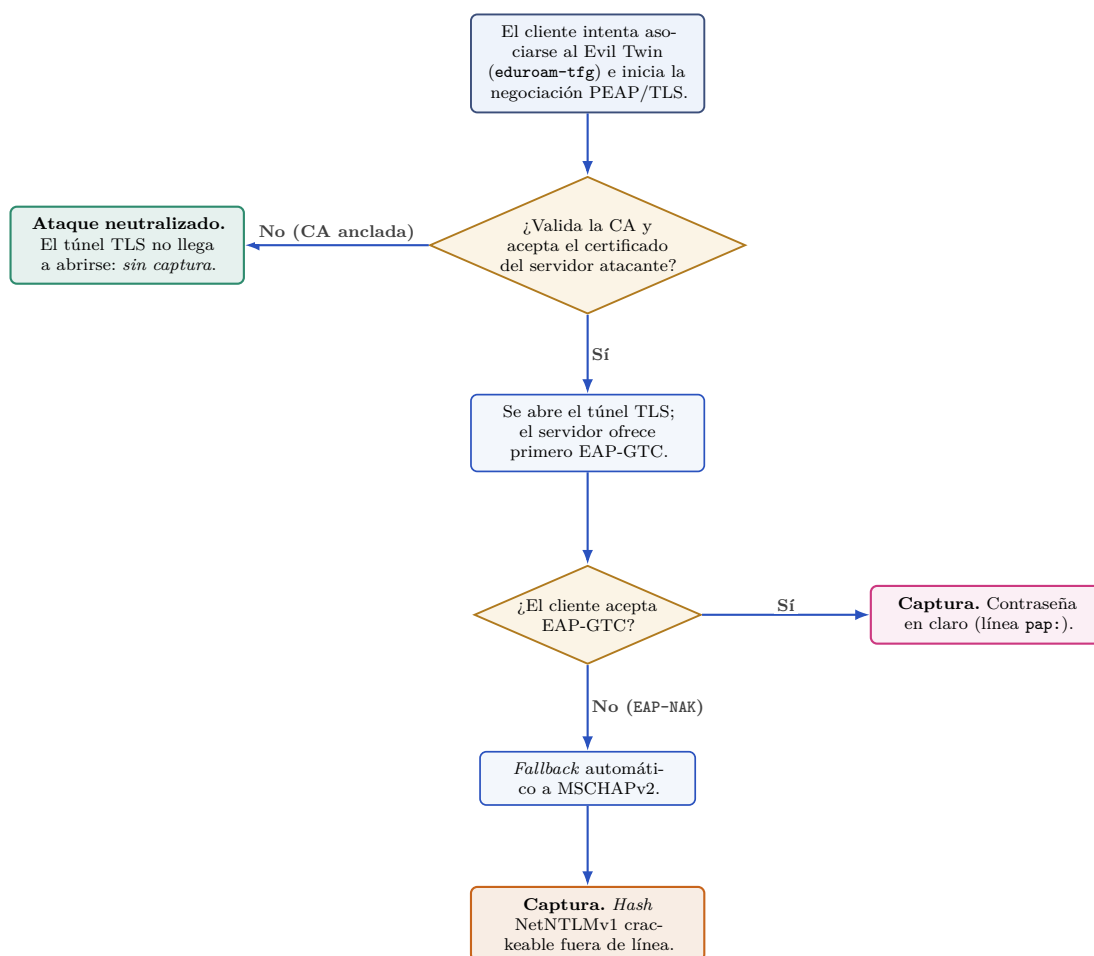


Figura 4.5: Flujo del método EAP en el *Evil Twin* con sus tres desenlaces posibles: (1) si el cliente acepta el certificado del atacante y responde a EAP-GTC, entrega la contraseña en claro (línea **pap:**); (2) si acepta el certificado pero rechaza EAP-GTC con EAP-NAK, el servidor retrocede a MSCHAPv2 y se captura un *hash* NetNTLMv1 crackeable fuera de línea (línea **mschap:**); (3) si el cliente ancla la CA legítima y rechaza el certificado del atacante, el túnel no llega a abrirse y no se produce captura alguna (único caso en que el ataque fracasa).

4.4.6. Red de gestión WPA2-PSK

La radio interna de la Raspberry Pi sostiene una red de gestión independiente del ataque, cuyo único cometido es permitir la administración remota de la plataforma mientras esta opera, sin necesidad de teclado, pantalla ni cable de red. Se trata del SSID *PatataWiFi_mgmt*, configurado como WPA2-PSK en el canal 1, con servidor DHCP propio (rango 172.31.0.10–172.31.0.100) y resolución DNS hacia servidores públicos. Desde un equipo conectado a esta red, el operador accede

por SSH a la Raspberry Pi para arrancar, detener o supervisar el laboratorio.

Esta red emplea WPA2-PSK, y no WPA2-Enterprise, porque la administración no requiere la complejidad de una infraestructura 802.1X y porque, al residir sobre la radio interna, queda al margen de las limitaciones del *driver* `mt76x2u` discutidas antes. La clave precompartida (`patatas333`) se hereda del proyecto original [28]; conviene subrayar que no constituye un secreto, sino un valor por defecto público del laboratorio y puede personalizarse antes de cualquier despliegue real. Hay que tener en cuenta que, aunque esta red de gestión no es el vector de ataque, su seguridad sigue siendo importante: un atacante que conozca las credenciales del sistema podría acceder a la plataforma atacante y obtener control total sobre ella.

4.5. Automatización y ciclo de vida desatendido

Descritos los componentes de la plataforma atacante, queda por exponer cómo se coordinan entre sí para que el laboratorio arranque por sí solo de forma fiable. Un laboratorio de seguridad solo resulta útil para la docencia y la defensa si puede reconstruirse y arrancarse de forma fiable, sin que el operador tenga que recordar una secuencia frágil de comandos manuales tras cada apagado. Por ese motivo, la última fase de la construcción de la plataforma atacante persigue dos objetivos complementarios: por un lado, que las dos redes (la de gestión y el *Evil Twin*) queden operativas de manera autónoma poco después del encendido de la Raspberry Pi 5, sin intervención humana; por otro, que todo el despliegue sea reproducible a partir del repositorio que ha sido creado, de modo que un tercero pueda regenerar el mismo entorno sobre hardware equivalente. La reproducibilidad aporta además una garantía metodológica, pues permite que el experimento descrito en esta memoria pueda ser auditado y repetido, y que las decisiones de diseño queden documentadas de forma trazable.

4.5.1. Orquestación con `systemd`

El proyecto del que parte la plataforma atacante, `PatataWiFiEnterprise` [28], contemplaba un autoarranque mediante una entrada de `crontab` con la directiva `@reboot`. Esta aproximación resulta insuficiente para el escenario de este trabajo, donde el orden de arranque y las dependencias entre componentes son críticos: el *Evil Twin* no puede levantarse hasta que la radio externa ha quedado en un estado limpio, y la red de gestión debe preceder al ataque para garantizar el acceso remoto a la plataforma en caso de fallo. Se optó, por tanto, por reemplazar el mecanismo de `crontab` por una orquestación basada en `systemd`, que ofrece un control fino del orden de arranque mediante dependencias explícitas y aísla cada etapa en su propia unidad con registro independiente.

El ciclo de vida se descompone en tres servicios encadenados (`tfg-cleanup`, `tfg-mgmt` y `tfg-attack`), en lugar de un único servicio monolítico. Esta separación de responsabilidades aporta varias ventajas. En primer lugar, cada etapa mantiene un registro propio (`/var/log/tfg-cleanup.log`, `/var/log/tfg-mgmt.log` y `/var/log/tfg-attack.log`), lo que facilita la trazabilidad y el diagnóstico. En segundo lugar, las dependencias declaradas con `Requires=` y `After=` garantizan que, si una etapa temprana falla, las posteriores no llegan a ejecutarse, evitando así estados inconsistentes de las interfaces. En tercer lugar, la modularidad permite reiniciar selectivamente el ataque (por ejemplo, tras modificar la configuración del servidor RADIUS) sin perturbar la red de gestión. La cadena completa y el flujo de arranque se resumen en la línea de tiempo de la plataforma.

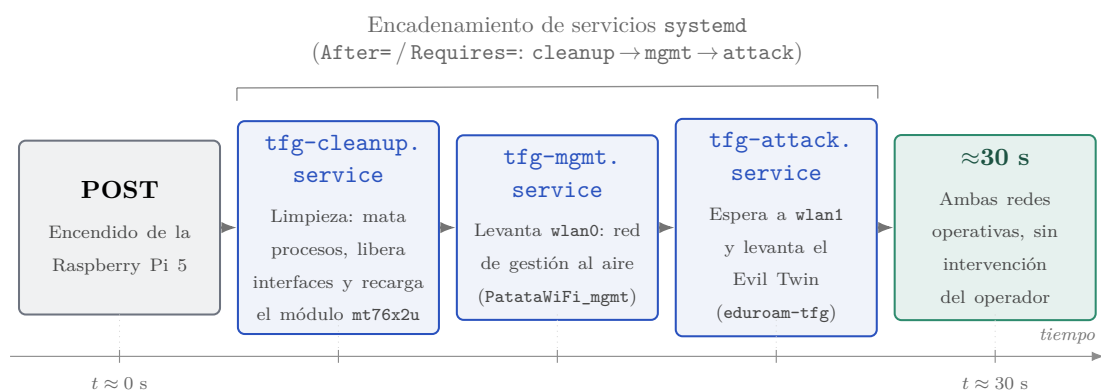


Figura 4.6: Cadena de arranque desatendido de la plataforma atacante: tras el POST de la Raspberry Pi 5 se ejecutan en cadena los tres servicios `systemd` (`tfg-cleanup` → `tfg-mgmt` → `tfg-attack`), encadenados por dependencias `After=`/`Requires=`, de modo que la red de gestión (`wlan0`) y el *Evil Twin* (`wlan1`) quedan al aire en torno a treinta segundos después del encendido, sin intervención del operador.

Los tres servicios comparten el mismo patrón de unidad: se declaran de tipo `Type=oneshot` con `RemainAfterExit=yes`. Esta combinación es la apropiada para *scripts* que realizan una secuencia de acciones, lanzan procesos en segundo plano (como `hostapd`, `dnsmasq` o el *wrapper* del ataque) y terminan: el tipo `oneshot` indica a `systemd` que el proceso principal concluye al final de su ejecución, mientras que `RemainAfterExit` hace que la unidad se considere activa de forma permanente una vez finalizado el *script*, reflejando que los servicios en segundo plano que ha levantado siguen vivos. El primer eslabón, además, se declara con `DefaultDependencies=no` y `After=network.target` para ejecutarse lo más temprano posible en el arranque y poder limpiar el estado del sistema antes de que el resto de componentes entren en juego. El reparto de tipos, dependencias y responsabilidades se detalla en el cuadro de servicios.

Servicio	Tipo	Depende de	Acción que realiza
tfg-cleanup	oneshot (RemainAfterExit=yes); además DefaultDependencies= no y After= network.target	Ninguna (primer eslabón)	Limpieza preventiva: mata procesos residuales (hostapd , radiusd , freeradius , dnsmasq), cierra sesiones tmux , libera y reinicia las dos interfaces, detiene NetworkManager y wpa_supplicant , y recarga el módulo mt76x2u (rmmod/modprobe).
tfg-mgmt	oneshot (RemainAfterExit=yes)	tfg-cleanup (After=/ Requires=)	Levanta la red de gestión sobre wlan0 (Broadcom BCM4345C0): pone la interfaz en modo managed , le asigna 172.31.0.1/24 y lanza hostapd (PatataWiFi_mgmt , WPA2-PSK, canal 1) junto con dnsmasq .
tfg-attack	oneshot (RemainAfterExit=yes)	tfg-mgmt (After=/ Requires=)	Materializa el <i>Evil Twin</i> sobre wlan1 : espera de forma activa a que aparezca la interfaz y lanza el <i>wrapper</i> hostapd-freeradius.sh (punto de acceso malicioso + FreeRADIUS-WPE) en una sesión tmux de cinco paneles.

Tabla 4.2: Los tres servicios **systemd** encadenados que orquestan el ciclo de vida de la plataforma atacante, con su tipo, sus dependencias y la acción que realiza cada uno. El encadenamiento **tfg-cleanup** → **tfg-mgmt** → **tfg-attack** garantiza un arranque ordenado y desatendido.

El primer servicio, **tfg-cleanup**, realiza una limpieza preventiva del sistema. Termina cualquier proceso residual de **hostapd**, **radiusd**, **freeradius**, **freeradius-wpe** y **dnsmasq**, cierra las posibles sesiones **tmux** previas, libera y reinicia las dos interfaces inalámbricas, y detiene **NetworkManager** y **wpa_supplicant**, que de otro modo interferirían con el modo punto de acceso. El paso más característico de esta etapa es la recarga del módulo del *kernel* **mt76x2u** mediante **rmmod** seguido de **modprobe**, que devuelve la radio externa Alfa AWUS036ACM (*chipset* MT7612U) a un estado conocido y reproducible, eliminando estados inconsistentes que podían quedar tras un apagado abrupto. Esta limpieza preventiva resultó esencial para evitar el diagnóstico repetido del laboratorio tras cada reinicio.

El segundo servicio, **tfg-mgmt**, levanta la red de gestión sobre la radio interna **wlan0** (Broadcom BCM4345C0). Vuelve a poner la interfaz en modo **managed** (precaución frente a un estado de “punto de acceso huérfano” detectado durante el desarrollo), le asigna la dirección **172.31.0.1/24** y lanza **hostapd** con la configura-

ción de gestión (SSID `PatataWiFi_mgmt`, WPA2-PSK, canal 1) junto con una instancia de `dnsmasq` que sirve direcciones en el rango `172.31.0.10–172.31.0.100`. Esta red proporciona un canal de administración fiable e independiente del ataque, de manera que la plataforma sigue siendo accesible aunque el *Evil Twin* no llegue a arrancar.

El tercer servicio, `tfg-attack`, materializa el *Evil Twin* sobre la radio externa `wlan1`. Antes de actuar, espera de forma activa (con un bucle de hasta quince intentos) a que la interfaz `wlan1` aparezca, ya que la radio Alfa se reenumera por USB tras la recarga del módulo realizada por `tfg-cleanup`. Una vez disponible, invoca el *wrapper* `hostapd-freeradius.sh` heredado de PatataWiFi, que orquesta el punto de acceso malicioso y el servidor de captura en una sesión `tmux` de cinco paneles: `hostapd`, `dnsmasq`, `radiusd` y los seguimientos en tiempo real de los registros `wpe.log` y `auth-detail`. Esta disposición multipanel facilita la supervisión simultánea de la asociación del cliente, la negociación EAP y la captura de credenciales durante las sesiones de validación descritas en el capítulo siguiente.

El resultado de esta orquestación es un laboratorio de arranque desatendido: tras el encendido de la Raspberry Pi 5, los tres servicios se ejecutan en cadena y ambas redes quedan al aire en torno a treinta segundos, sin ninguna intervención del operador.

4.5.2. Instalación y reversión automatizadas

La construcción manual del laboratorio implica una secuencia de operaciones cuyo orden es relevante: el servidor RADIUS debe instalarse y configurarse antes de generar los certificados, estos deben existir antes de redirigir hacia ellos la configuración EAP, y el *wrapper* de PatataWiFi solo puede lanzarse cuando todos los componentes anteriores están en su sitio. Para encapsular esta secuencia y hacerla repetible se desarrolló un *script* de instalación, `install.sh`, inspirado en el instalador del proyecto original [29] pero reescrito para el escenario de este trabajo. El *script* se ejecuta con privilegios de superusuario, valida en su preámbulo que se está ejecutando como `root` y sobre arquitectura `aarch64`, y se diseñó para ser idempotente: cada paso comprueba si la acción ya está realizada (parche ya aplicado, certificado ya generado, binario ya compilado) y, en tal caso, la omite, de modo que el *script* puede re-ejecutarse sin riesgo de dejar el sistema en un estado inconsistente. La idempotencia de la aplicación de parches se apoya en una comprobación previa con `patch` en modo `-reverse -dry-run`, como ilustra el fragmento siguiente.

```
apply_patch_idempotent() {
    local patch="$1" dir="$2" name
    name="$(basename "$patch")"
    if (cd "$dir" && patch -p1 --reverse --dry-run --silent < "$patch") 2>/dev/
        null; then
```

```
    note "patch ya aplicado: $name"
    return 0
fi
if ! (cd "$dir" && patch -p1 --forward --silent < "$patch"); then
    fail "patch fallo: $name"
fi
}
```

Código 4.3: Aplicación idempotente de un parche en `install.sh`

El instalador se estructura en once pasos numerados, que reproducen fielmente la secuencia de construcción descrita a lo largo de este capítulo. Los dos primeros preparan el sistema: instalan las dependencias base (herramientas de compilación, `hostapd`, `dnsmasq`, utilidades inalámbricas y `firmware` de MediaTek y Broadcom) y configuran el repositorio de Kali rolling, incorporando su clave GPG oficial y fijando una prioridad de `pin 50` para que sus paquetes no se instalen de forma automática. El tercer paso instala explícitamente desde ese repositorio `freeradius-wpe` [34], en su versión `3.2.5+dfsg-3kali1`. El cuarto aplica los parches sobre la configuración del servidor (el que comenta las directivas de usuario y grupo en `radiusd.conf`, el que redirige la configuración EAP hacia los certificados propios y el que activa el *downgrade* a EAP-GTC [35]) y, como `mods-enabled/eap` llega como enlace simbólico en el paquete, lo convierte previamente en fichero regular. El quinto genera mediante `make bootstrap` la autoridad certificadora y el certificado de servidor del atacante. El sexto clona el repositorio `PatataWiFiEnterprise` y copia su contenido al directorio de trabajo, y el séptimo aplica sobre él los parches que transforman su configuración de WPA-PSK a WPA-EAP con CCMP y gestión de tramas protegidas.

El octavo paso descarga el código fuente de `hostapd 2.6` desde `w1.fi` [33], verifica su huella SHA-256 frente a un valor codificado en el propio *script* (abortando si no coincide, lo que proporciona una garantía criptográfica de integridad superior a la de distribuir una copia del binario), aplica el parche que habilita `CONFIG_LIBNL32` e `IEEE80211N`, y lo compila. La elección de la versión 2.6 frente a la 2.10 responde a que esta última provocaba desconexiones espurias con el controlador `mt76x2u` [32] sobre los *kernels* de la serie 6.x. Los tres pasos finales completan el despliegue: el noveno instala la configuración de la red de gestión y crea el enlace simbólico que expone la configuración de FreeRADIUS-WPE al *wrapper*; el décimo copia los *scripts* `tfg-*.sh` a `/usr/local/bin` y las unidades a `/etc/systemd/system`; y el undécimo habilita los tres servicios para el arranque. El *script* concluye con una verificación final que confirma que los servicios han quedado habilitados y que los artefactos esperados (binario de `hostapd`, `radiusd`, certificados y *scripts*) están presentes, abortando con un código de error si detecta cualquier discrepancia. Tras un reinicio, el laboratorio queda operativo.

Por simetría, en el repositorio creado, se incluye también un *script* de reversión, `uninstall.sh`, deliberadamente conservador. De forma automática detiene

y deshabilita los tres servicios, elimina sus *scripts* y unidades, borra los registros del laboratorio, retira el directorio de gestión creado por la instalación y restaura la configuración original de `radiusd.conf` si encuentra el respaldo correspondiente. En cambio, para todo aquello que podría contener datos del usuario o ser utilizado por otros proyectos (la desinstalación del paquete `freeradius-wpe`, la retirada del repositorio de Kali y la eliminación del directorio de PatataWiFi, que alberga los registros de capturas), el *script* solicita confirmación interactiva, con la opción negativa por defecto. Esta cautela evita borrados accidentales de material relevante para el análisis posterior y respeta el principio de mínima sorpresa en una herramienta que opera con privilegios elevados.

4.6. Formato y almacenamiento de las credenciales capturadas

Construida y automatizada la plataforma atacante, conviene cerrar la descripción del laboratorio documentando el artefacto que da sentido a todo el montaje: el fichero en el que la edición WPE deposita el material de autenticación capturado. La pieza que distingue a FreeRADIUS-WPE de un servidor RADIUS convencional no es su lógica de autenticación, sino su instrumentación: el servidor está modificado para registrar, en texto plano sobre un fichero de bitácora, el material de autenticación que el cliente entrega dentro del túnel PEAP [35]. En consecuencia, la construcción del laboratorio no quedaría completa sin documentar el artefacto que materializa la captura, esto es, el fichero donde quedan depositadas las credenciales y el formato exacto que adoptan. Esta sección describe ese mecanismo a nivel de diseño; los resultados experimentales obtenidos al ejercitarlo se reservan para el Capítulo 5.

Todo el material capturado por la edición WPE se concentra en un único punto del sistema de ficheros de la plataforma atacante, la ruta `/var/log/freeradius-wpe/freeradius-server-wpe.log`. Conviene observar que este registro no rota automáticamente, de modo que crece de forma monótona a lo largo de la vida del laboratorio y debe vaciarse de forma explícita entre sesiones para evitar la mezcla de pruebas. La ausencia de rotación es coherente con el carácter experimental del entorno, en el que cada captura se examina manualmente, pero constituye un detalle relevante a la hora de interpretar el contenido del fichero.

El formato concreto de cada entrada depende del método de autenticación interno que el cliente termine empleando dentro del túnel, lo cual está a su vez determinado por el ataque de degradación descrito en la sección anterior. Se distinguen así dos formas de registro mutuamente excluyentes por sesión.

4.6.1. Caso EAP-GTC: credencial en claro

Cuando el cliente acepta el certificado del servidor atacante y, además, accede a continuar la autenticación interna mediante EAP-GTC, el *supplicant* transmite la contraseña corporativa en claro dentro del túnel TLS. FreeRADIUS-WPE la registra entonces en una entrada encabezada por la etiqueta **pap:**, seguida de la identidad del usuario y de la contraseña literal. Esquemáticamente, la entrada presenta la siguiente estructura:

```
pap: <marca temporal>
      username: <usuario>
      password: <contrasena>
```

Código 4.4: Estructura de la captura en el caso EAP-GTC.

En este escenario no se requiere ningún tratamiento posterior: la credencial queda directamente legible, sin coste computacional alguno, lo que ilustra de manera contundente el riesgo de aceptar un certificado de servidor no validado.

4.6.2. Caso de *downgrade* a MSCHAPv2: hash NetNTLMv1

Si el cliente acepta el certificado pero rechaza EAP-GTC mediante un mensaje EAP-NAK, el servidor retrocede automáticamente a MSCHAPv2 y la captura cambia de naturaleza. En lugar de la contraseña, queda registrado el intercambio reto-respuesta característico de MSCHAPv2 en una entrada encabezada por la etiqueta **mschap:**, que recoge la identidad del usuario, el reto del servidor (*challenge*) y la respuesta del cliente (*response*). Junto a esos campos, FreeRADIUS-WPE emite además una línea auxiliar etiquetada como **john NetNTLMv1:**, ya preparada en el formato que consume directamente la herramienta John the Ripper invocada con **john -format=netntlm**.

El registro adopta internamente una representación en hexadecimal cuyos campos van separados por dos puntos, con la forma **username::challenge:response:peerchallenge**. La tabla siguiente resume el significado y el tamaño de cada campo, información imprescindible para reformatear el *hash* hacia otras herramientas.

Campo	Tamaño	Descripción
username	variable	Identidad interna del usuario, capturada dentro del túnel PEAP.
challenge	8 bytes (16 hex)	Reto del servidor (<i>authenticator challenge</i>) del intercambio MSCHAPv2.
response	24 bytes (48 hex)	Respuesta del cliente, derivada del <i>hash</i> NT mediante tres operaciones DES.
peerchallenge	16 bytes (32 hex)	Reto generado por el propio cliente (<i>peer challenge</i>).

Tabla 4.3: Campos del *hash* NetNTLMv1 tal como FreeRADIUS-WPE lo registra en el caso de *downgrade* a MSCHAPv2, con la forma `username::challenge:response:peerchallenge`. Conocer el tamaño de cada campo es imprescindible para reformatear el *hash* hacia hashcat (modo 5500) o John the Ripper.

Para el ataque de fuerza bruta con hashcat se emplea el modo `-m 5500` (NetNTLMv1 / NetNTLMv1+ESS) [36], que espera los campos ordenados como `username:::response:challenge`. La conversión es puramente sintáctica: basta extraer las entradas de captura del fichero de bitácora y reordenar los campos al formato esperado por hashcat, sin alterar ningún valor. Al igual que en el caso anterior, ni los retos, ni las respuestas, ni los *hashes* observados se reproducen en este documento, que se limita a describir su estructura.

4.6.3. Recuperabilidad de MSCHAPv2 fuera de línea

Conviene precisar que el *downgrade* a MSCHAPv2 no constituye una mitigación, sino una degradación más sutil: aunque la contraseña ya no viaja en claro, la credencial sigue siendo recuperable mediante un ataque fuera de línea. La razón es estructural y fue expuesta de forma definitiva por Marlinspike y Hulton en la DEF CON 20 (2012) [37]. El reto-respuesta de MSCHAPv2 deriva de un *hash* NT de dieciséis bytes que se procesa, dividido en tres bloques, mediante tres operaciones DES (del inglés, *Data Encryption Standard*) independientes. El tercero de esos bloques solo aprovecha dos bytes de material de clave, lo que reduce drásticamente el espacio de búsqueda; pero, sobre todo, la seguridad efectiva del conjunto colapsa a la de una única operación DES de 56 bits. Un espacio de claves de ese tamaño es hoy enteramente abarcable por hardware especializado o por una búsqueda exhaustiva acelerada por GPU, de modo que el *hash* NetNTLMv1 capturado equivale, en términos prácticos, a recuperar la contraseña original siempre que esta sea débil o medianamente compleja. Es justamente esta reducción a operaciones DES de 56 bits la que justifica que el fichero de captura, aun cuando solo contenga *hashes* y no contraseñas en claro, deba tratarse como material sensible.

Lo expuesto hasta aquí documenta exclusivamente el formato y el almacenamiento de las credenciales como artefacto resultante de la construcción del laboratorio. La ejecución efectiva del ataque, las capturas reales obtenidas, el proceso de *cracking* fuera de línea del *hash* NetNTLMv1 y la comparación del comportamiento de los distintos clientes frente al certificado y al método de autenticación interno se presentan en el Capítulo 5.

4.7. Decisiones de diseño y problemas críticos de montaje

La construcción de la plataforma atacante no siguió una trayectoria lineal: la forma definitiva del laboratorio es, en buena medida, el residuo de una sucesión de obstáculos técnicos que fueron descartando alternativas aparentemente más sencillas y obligando a soluciones concretas. Documentar esas decisiones no es un ejercicio añadido, sino parte sustancial de la aportación de este trabajo: justifican por qué la arquitectura final adopta exactamente la configuración descrita en las secciones anteriores y no otra. Esta sección sintetiza los problemas que condicionaron el diseño y la solución que se adoptó en cada caso; el catálogo exhaustivo de errores, con la cadena de mensajes literal, la causa raíz y el procedimiento de reparación de cada uno, se relega al apéndice de problemas y soluciones, al que se remite para el detalle completo.

4.7.1. La imposibilidad de multi-BSSID

El planteamiento de partida, heredado del orquestador PatataWiFiEnterprise [28], contemplaba emitir las dos redes del laboratorio (la de gestión y el *Evil Twin*) como dos BSSID virtuales sobre una única radio física, recurso habitual en *hostapd* mediante la directiva `bss`. Esta vía resultó inviable con el adaptador atacante. El *driver* `mt76x2u` del *chipset* MediaTek MT7612U [32] dispone de un único registro de dirección hardware configurable, vinculado al BSS primario, y emula los BSS adicionales por software; en redes de tipo WPA2-PSK esa emulación basta, pero en WPA2-Enterprise el segundo BSS no procesa correctamente los marcos de gestión 802.1X. El síntoma observado fue inequívoco: los clientes detectaban la red `eduroam-tfg` en el escaneo, pero la asociación se interrumpía en la fase de autenticación 802.11 sin que el adaptador llegara a responder, mientras que la red de gestión PSK emitida en el BSS primario funcionaba con normalidad. La causa, contrastada con el seguimiento de incidencias del *driver*, confirma que se trata de una limitación conocida de la combinación *multi-BSSID* más modo Enterprise sobre esta familia de hardware.

La solución adoptada fue abandonar el *multi-BSSID* y separar las dos redes en

dos radios físicas independientes: la radio interna de la Raspberry Pi 5 (Broadcom BCM4345C0, *driver* `brcmfmac`) sobre la interfaz `wlan0` para la red de gestión, y el adaptador Alfa AWUS036ACM sobre `wlan1` para el *Evil Twin*. Más allá de eludir el defecto del *driver*, esta separación reproduce la arquitectura habitual de los despliegues profesionales de auditoría, en los que se dedica una radio por función, y permite asignar canales distintos (1 para la gestión, 6 para el ataque) evitando interferencias mutuas. La distinción es relevante y conviene matizarla para evitar malentendidos: la radio interna no soporta actuar como dos puntos de acceso simultáneos, pero sí como un único AP emitiendo una sola red, que es todo lo que la red de gestión necesita.

4.7.2. FreeRADIUS: del 2.1.12 al paquete WPE 3.2.5 de Kali

El proyecto del que se partió acompañaba sus plantillas de una versión de FreeRADIUS de la rama 2.1.12. Reutilizarla habría exigido la biblioteca `libssl1.0`, inexistente ya en Raspberry Pi OS *bookworm* sobre `aarch64`, por lo que esa versión quedó descartada de entrada. La alternativa de portar manualmente su configuración a la rama 3.x del FreeRADIUS del sistema se intentó y se abandonó: la migración archivo por archivo de una configuración de 2016 obligaba a una cadena interminable de adaptaciones sintácticas (renombrados de directivas, módulos referenciados sin su fichero de definición, directorios reorganizados) y, aun completándola, el binario estándar no registra las credenciales del túnel, que es precisamente lo que el laboratorio necesita capturar.

La decisión definitiva fue instalar el paquete `freeradius-wpe` en su versión 3.2.5+dfsg-3kali1 [34], que incorpora ya el parche *Wireless Pwn Edition* [35] sobre una base 3.x moderna y compatible con OpenSSL 3. El paquete se obtuvo del repositorio *kali-rolling* añadido como fuente APT pero con prioridad de fijación (*pin*) 50, de modo que sus paquetes solo se instalan bajo petición explícita y no contaminan ni actualizan el resto del sistema Debian de base. Sobre esa instalación se reemplazó la configuración rota del orquestador por un enlace simbólico a la del propio paquete, dejando que el *wrapper* de PatataWiFi se limite a invocar el binario, indiferente al contenido de la configuración.

4.7.3. hostapd 2.6 frente a 2.10

El proyecto original venía con `hostapd` 2.10. Esta versión provocaba desconexiones espurias con el *driver* `mt76x2u` sobre los *kernels* de la serie 6.x de la Raspberry Pi, comprometiendo la estabilidad del punto de acceso malicioso. Se optó por compilar desde las fuentes oficiales de `wl.f` [33] la versión 2.6, que no presentaba ese comportamiento, verificando la integridad del archivo descargado

mediante su suma SHA-256 antes de la compilación. La compilación se ajusta con un parche mínimo de configuración que habilita el soporte de `libnl 3.2` y de 802.11n. El `hostapd` de la red de gestión, en cambio, es el del propio sistema, pues sobre la radio interna no se daba el problema.

4.7.4. La colisión de direcciones IP entre las dos interfaces

Al separar las redes en dos radios afloró un conflicto que el *multi-BSSID* había ocultado: el *wrapper* de PatataWiFi asignaba a su interfaz la dirección `172.31.0.1`, la misma que se había fijado para la red de gestión en `wlan0`. Con ambas interfaces compartiendo dirección, el segundo servidor DHCP que intentaba arrancar no podía abrir su *socket* de escucha y abortaba con un error de dirección ya en uso. La solución consistió en desplazar el *Evil Twin* a una subred propia, asignando `10.0.0.1/24` a `wlan1` y reservando `172.31.0.1/24` para la gestión; este cambio quedó consolidado como un parche permanente sobre el *wrapper*, de modo que la separación de rangos es estructural y no una corrección manual repetida en cada arranque.

4.7.5. Sesiones `tmux` huérfanas y el script de arranque

Durante la depuración fueron recurrentes dos clases de incidencias ligadas a la orquestación. La primera eran las sesiones `tmux` residuales: el orquestador detecta si ya está en marcha por la presencia de su sesión `tmux`, de modo que una sesión que sobrevivía a un intento fallido impedía relanzarlo y dejaba además procesos e interfaces a medio configurar. La segunda fue un primer guion de arranque conjunto, concebido para levantar de un golpe la gestión y el ataque, que moría nada más empezar: invocaba internamente el *script* de limpieza, y este, al matar por patrón todos los procesos cuyo nombre contenía la cadena del proyecto, terminaba matando a su propio proceso padre. Como se ha documentado anteriormente, ambos problemas se resolvieron sustituyendo el arranque *ad hoc* por una arquitectura de tres servicios `systemd` encadenados (limpieza, gestión y ataque), todos de tipo `oneshot` con `RemainAfterExit`, vinculados por dependencias estrictas (`After/Requires`) de manera que cada eslabón solo arranca si el anterior ha terminado correctamente. El servicio de limpieza, que se ejecuta primero, asume explícitamente la responsabilidad de cerrar las sesiones `tmux` previas, terminar los procesos pendientes y devolver las interfaces a un estado conocido, eliminando de raíz los estados residuales que antes había que deshacer a mano.

4.7.6. El retardo en la aparición de wlan1 tras el arranque

La cadena de servicios destapó un problema de temporización propio del arranque en frío. El adaptador Alfa es un dispositivo USB cuyo *driver* se carga, y cuya interfaz `wlan1` aparece, con un retardo apreciable respecto al instante en que `systemd` alcanza la fase de arranque de los servicios de usuario. El servicio de ataque, al lanzarse, podía no encontrar todavía la interfaz y fallar. La solución fue dotar a ese servicio de una espera activa: antes de invocar al orquestador comprueba de forma reiterada la existencia de `wlan1` durante un número acotado de intentos, abortando con un mensaje claro solo si la interfaz no ha aparecido transcurrido ese margen. Esta espera, junto con el orden impuesto por las dependencias de `systemd`, es lo que garantiza que ambas redes queden operativas de forma fiable aproximadamente treinta segundos después del encendido.

4.7.7. radiusd ejecutándose como root

El servidor RADIUS del paquete instalado se configura por defecto para ceder privilegios a una cuenta de servicio dedicada (`freerad-wpe`). En este laboratorio el árbol de trabajo del orquestador reside bajo el directorio personal del superusuario, con permisos restrictivos, y el proceso necesita además manipular las interfaces de red; bajo la cuenta de servicio sin privilegios esas operaciones fallaban. La solución fue un parche que comenta las directivas de usuario y grupo en la configuración del servidor, dejándolo correr como `root`.

5

Resultados, validación o experimentos

Construido el laboratorio a lo largo del Capítulo 4, queda por comprobar de forma empírica que la plataforma cumple su cometido frente a dispositivos reales. Este capítulo recoge esa validación experimental. Se detalla primero el entorno sobre el que se han realizado las pruebas, esto es, la disposición física de los equipos, el mecanismo que conduce al dispositivo víctima hacia el punto de acceso gemelo, el procedimiento seguido en cada ensayo y los criterios objetivos con los que se determina su resultado. Se expone después, dispositivo a dispositivo, el comportamiento observado en cada uno de los equipos probados, desde teléfonos móviles y ordenadores portátiles hasta lectores de libros electrónicos. Por último, una tabla resumen reúne las características de cada dispositivo y su respuesta ante el ataque, y permite una lectura transversal de los resultados.

5.1. Entorno experimental

El entorno empleado en las pruebas es el laboratorio construido en el capítulo anterior, compuesto por las tres piezas ya descritas: la red legítima (la máquina virtual Ubuntu Server con FreeRADIUS y `hostapd` que, desde el portátil del autor, emite el SSID `eduroam-tfg` a través de la antena Alfa AWUS036AXM), la plataforma atacante (la Raspberry Pi 5 que levanta el *Evil Twin* sobre la antena Alfa AWUS036ACM, con FreeRADIUS-WPE como motor de captura) y el dispositivo víctima (cualquier cliente con pila 802.1X/EAP que llegue a asociarse al gemelo). La arquitectura de conjunto y el flujo general del ataque ya se recogieron en el diagrama de arquitectura general del laboratorio, lo que

interesa precisar ahora es cómo se disponen físicamente esas piezas durante un ensayo y de qué forma se opera para provocar el ataque y observar su desenlace.

5.1.1. Disposición física de los equipos

El escenario reproduce, a escala de laboratorio, la situación que viviría un usuario en un espacio público: un dispositivo que conoce la red de su institución y que, al desplazarse, va perdiendo la cobertura del punto de acceso legítimo a la vez que entra en el radio de uno malicioso. Para recrearla, los dos puntos de acceso se separan físicamente. El portátil que aloja la red legítima se sitúa en un extremo del espacio de pruebas y emite el SSID `eduroam-tfg` desde la antena Alfa AWUS036AXM. En el extremo opuesto se coloca la Raspberry Pi 5, que con la antena Alfa AWUS036ACM anuncia ese mismo SSID `eduroam-tfg`. Para el cliente, ambas emisiones resultan a primera vista la misma red, las únicas diferencias perceptibles son la intensidad con que cada una llega a su posición y el certificado que cada servidor RADIUS presenta durante la negociación.

La pieza que dispara el ataque es el mecanismo de itinerancia, o *roaming*, presente en cualquier cliente Wi-Fi. Cuando un dispositivo tiene memorizada una red, vigila de forma continua la intensidad de señal (RSSI) de los puntos de acceso que anuncian su SSID y tiende a asociarse al que ofrece mejor cobertura. De este modo, si la señal del punto de acceso al que está conectado decae por debajo de cierto umbral mientras otro que anuncia el mismo SSID llega con más fuerza, el cliente migra a este último sin intervención del usuario. Ese comportamiento, pensado para dar continuidad al servicio cuando alguien se mueve entre los distintos puntos de acceso de una misma red, es justamente el que el *Evil Twin* aprovecha en su favor.

Sobre esa base, cada prueba comienza asociando el dispositivo a la red legítima, junto al portátil, y alejándolo después de forma progresiva hacia la Raspberry Pi. A medida que aumenta la distancia con el portátil, la señal de la red legítima se debilita hasta que el *Evil Twin*, más próximo, pasa a ser el punto de acceso con mejor cobertura para ese SSID. En ese instante el suplicante del dispositivo intenta reasociarse al gemelo de forma automática y arranca la negociación que el ataque pretende explotar. Esta separación física reproduce un aspecto esencial del ataque real: el atacante puede acercar su equipo a la víctima (de ahí el carácter portátil y autónomo de la plataforma, alimentada por una batería externa) para que el suplicante de esta lo seleccione por mejor intensidad de señal.

5.1.2. Procedimiento de prueba

El ensayo con cada dispositivo sigue siempre la misma secuencia, de modo que los resultados sean comparables entre equipos. En primer lugar, se da de

alta el dispositivo en la red legítima con unas credenciales de laboratorio, para que memorice el SSID `eduroam-tfg` junto con su método de autenticación y el certificado o la autoridad de certificación de la red. Estas credenciales son ficticias y de uso exclusivo en el entorno aislado, en ningún caso se emplean credenciales reales. A continuación se provoca el *roaming* alejando el dispositivo del portátil, según se ha descrito. Por último, se observa la reacción del suplicante desde los paneles de supervisión de la propia Raspberry Pi, accesibles por SSH a través de la red de gestión: la sesión `tmux` del orquestador muestra en vivo la actividad de `hostapd`, la del servidor FreeRADIUS-WPE y, sobre todo, la del *log* de captura, donde se materializa o no el robo de credenciales.

Conviene precisar el alcance de estas pruebas. En todos los dispositivos se ha trabajado, siempre que su gestor de red lo permite, con la configuración que adopta la gran mayoría de los usuarios, es decir, sin anclar la autoridad de certificación legítima de la institución. El motivo es que el anclaje del certificado (CA *pinning*) neutraliza el ataque por completo y con independencia del dispositivo: si el cliente exige que el certificado del servidor esté firmado por la CA esperada, el túnel no llega a abrirse y no queda comportamiento alguno que observar ni que comparar entre equipos. Ensayar ese caso dispositivo a dispositivo carecería de sentido, porque el resultado sería siempre idéntico. Lo que de verdad interesa es cómo responde cada dispositivo cuando el usuario no toma esa precaución, que es el escenario realista y el único que distingue a unos equipos de otros. Ese caso protegido se retoma por separado, más adelante, como la contramedida que confirma la tesis del trabajo.

5.1.3. Criterios de validación y desenlaces posibles

El resultado de cada prueba se determina con criterios objetivos y reproducibles, definidos sobre los propios registros del servidor RADIUS atacante y ya anticipados en la metodología del trabajo. La captura de una contraseña en claro se da por válida cuando en el *log* de FreeRADIUS-WPE aparece una línea `pap:` con el usuario y la contraseña del cliente. La captura de un *hash* se confirma cuando aparece una línea con el par desafío-respuesta del intercambio, del que se deriva un *hash* de tipo NetNTLMv1 recuperable fuera de línea. Y la mitigación se considera demostrada cuando, ante un cliente que valida estrictamente el certificado, no se genera ninguna entrada en el *log* y el túnel ni siquiera llega a abrirse.

Esos criterios se corresponden con los tres desenlaces posibles del ataque, ya descritos al detallar el flujo del método EAP en el capítulo anterior, que sirven aquí de marco para interpretar el comportamiento de cada dispositivo en las secciones siguientes:

1. El cliente acepta el certificado del servidor atacante y, una vez abierto el túnel, acepta negociar EAP-GTC. Entrega entonces la contraseña en claro,

que queda registrada en la línea **pap**: sin necesidad de cómputo posterior. Es el desenlace más favorable para el atacante.

2. El cliente acepta el certificado, pero rechaza EAP-GTC con un mensaje **EAP-NAK** y exige MSCHAPv2. El servidor retrocede a ese método y captura un *hash* NetNTLMv1; no se obtiene la contraseña de forma inmediata, aunque sí material crackeable fuera de línea mediante hashcat (modo 5500) o john.
3. El cliente tiene anclada la autoridad de certificación legítima de su institución (CA *pinning*) y rechaza el certificado del atacante. El túnel TLS no llega a abrirse y el ataque no obtiene nada. Es el único caso en que la víctima queda protegida y, a la vez, la confirmación más clara de que la validación estricta del certificado es la defensa que de verdad funciona.

Como ya se ha indicado, las pruebas se realizan sin que el usuario ancle la CA, de modo que el tercer desenlace solo aparece cuando es el propio dispositivo el que impone esa validación; en los demás casos, los equipos ensayados recaen en uno de los dos primeros: la entrega de la contraseña en claro o el *downgrade* a MSCHAPv2. Con este marco fijado, las secciones siguientes recorren los dispositivos probados uno a uno, indicando para cada uno cuál de los desenlaces se produjo.

5.2. Comportamiento de los dispositivos

Descrito el entorno y el marco con el que se interpretan los resultados, esta sección recoge el comportamiento observado en cada uno de los dispositivos sometidos al ataque. Los equipos se agrupan por ecosistema, ya que los que comparten sistema operativo y pila de red reaccionan de forma prácticamente idéntica; para cada grupo se describe la secuencia completa que se observa al provocar el *roaming* hacia el *Evil Twin* y el desenlace en que termina.

5.2.1. Dispositivos Apple (iOS y macOS)

Dentro del ecosistema de Apple se han probado cuatro equipos: un iPhone 16 Pro con iOS 27.0 (beta de desarrollador), un iPhone 11 con iOS 26.5, un iPhone 12 Pro Max con iOS 26.4.2 y un MacBook Pro con chip M1 Max y macOS Tahoe 26.5.1. Pese a las diferencias de modelo, generación y versión del sistema operativo, los cuatro exhiben el mismo comportamiento frente al *Evil Twin*.

El ensayo parte del dispositivo asociado a la red legítima, en la que, tras aceptar su certificado la primera vez, navega con normalidad. Al desplazarlo hacia la zona de cobertura del *Evil Twin* y debilitarse la señal de la red legítima, el dispositivo

detecta el punto de acceso gemelo e intenta reasociarse a él, pero no completa la conexión por sí solo. En unas ocasiones la tentativa termina con un error y la red queda listada en el apartado de “Redes conocidas”, en otras, el dispositivo se queda con el indicador de carga girando, como si la conexión estuviera en curso, sin llegar a establecerse. En ninguno de los dos casos el suplicante entrega las credenciales de forma automática: para que el proceso avance es necesario que el usuario seleccione manualmente la red, indicando así de forma explícita su intención de autenticarse.

Es en ese momento, y solo entonces, cuando el dispositivo presenta al usuario el certificado del servidor atacante y le solicita que lo acepte. El sistema detecta que la red presenta un certificado distinto del que se aceptó en su día, lo advierte al usuario y deja en sus manos la decisión de aceptarlo o no. Esa salvaguarda, sin embargo, depende por completo del criterio de quien la recibe, y un usuario sin conocimientos para distinguir un certificado legítimo de uno fraudulento tiende a aceptarlo para recuperar la conexión. En cuanto el certificado se acepta, el túnel TLS se abre y los cuatro equipos admiten el *downgrade* a EAP-GTC, de modo que la contraseña se transmite en claro dentro del túnel y queda registrada en la línea **pap**: del *log* de captura.

5.2.2. Dispositivos Android

En el ecosistema Android se ha probado un teléfono Poco M8 5G con Android 16 y la capa de personalización HyperOS (compilación 3.0.301.0.WPQEUXM.C08). El teléfono se conecta sin problemas a la red legítima, pero al desplazarlo hacia el *Evil Twin* nunca llega a asociarse a él: en ningún momento ofrece la opción de aceptar el certificado del servidor atacante, mantiene la conexión en curso de forma indefinida y termina devolviendo errores de red, como si detectara una anomalía y descartara el intento sin más.

Este comportamiento se explica por el mecanismo de confianza en el primer uso (TOFU, del inglés *Trust On First Use*) que Android incorpora desde su versión 13 [38]. Cuando un dispositivo se conecta por primera vez a una red WPA2-Enterprise para la que no se ha configurado una autoridad de certificación, Android retiene el certificado que el servidor le presenta y fija (*pinning*) algunos de sus atributos para validar las conexiones posteriores. Así, al asociarse a la red legítima, el teléfono ancla el certificado del servidor RADIUS auténtico. Cuando más tarde aparece el *Evil Twin*, que anuncia el mismo SSID pero presenta un certificado distinto, el dispositivo comprueba que no coincide con el que tenía anclado y rechaza la conexión por su cuenta, sin volver a consultar al usuario. De ahí que se quede cargando y acabe en error sin llegar a ofrecer la aceptación del nuevo certificado.

A efectos del ataque, el resultado es que el Poco M8 no entrega credenciales y no se registra ninguna entrada en el *log* de captura. Coincide, por tanto, con el

tercer desenlace, el de la mitigación, aunque lo alcanza por una vía propia: es el sistema operativo el que aplica el anclaje del certificado de forma automática en la primera conexión, sin que intervenga la configuración manual del usuario.

Conviene matizar, no obstante, que este resultado no es extrapolable a todo el ecosistema Android, por dos motivos. El primero es la personalización de los fabricantes: la validación obligatoria del certificado y el propio mecanismo TOFU son el comportamiento que Android trae de serie desde la versión 13, pero el subsistema Wi-Fi es uno de los componentes que los fabricantes pueden ajustar mediante *overlays* de configuración (por ejemplo, la opción `config_wifiAllowInsecureEnterpriseConfiguration`), de manera que algunas capas de personalización siguen permitiendo crear configuraciones inseguras que omiten la validación del servidor, tal como ya se apuntaba en el estado del arte de este trabajo. El segundo motivo, y más determinante, es la versión de Android instalada.

Este segundo punto merece tratarse aparte. El Poco M8 ejecuta Android 16, pero las protecciones que neutralizan el ataque son relativamente recientes, de modo que un dispositivo con una versión anterior se comportaría de forma muy distinta. No ha sido posible comprobarlo de forma empírica por no disponer de equipos con esas versiones, de modo que todo lo que sigue se apoya en la documentación investigada y no en pruebas propias, aunque la evolución del sistema en este punto está bien documentada. Hasta Android 10, la configuración de una red WPA2-Enterprise ofrecía la opción de no validar el certificado del servidor (rotulada como “No validar” o equivalente), que numerosas instituciones recomendaban en sus guías y que dejaba al dispositivo completamente expuesto al *Evil Twin* [21, 19]. Android 11 retiró esa opción del menú de configuración para evitar el robo de credenciales [39], y Android 12 fue un paso más allá al exigir de forma obligatoria la validación del certificado del servidor en las redes *enterprise*, en línea con la especificación de la Wi-Fi Alliance [40]. Sobre esa base, Android 13 incorporó por último el mecanismo TOFU ya descrito [38]. Con esa cautela, y siempre como expectativa derivada de esa documentación y no de una prueba directa, un dispositivo con Android 10 o anterior configurado para no validar el certificado (o cualquier versión en la que esa opción siguiera disponible) caería previsiblemente en uno de los dos desenlaces de captura, aunque por una vía distinta de la observada en los equipos de Apple: mientras que estos exigían que el usuario aceptase a mano el certificado del gemelo antes de conectarse, un Android con la validación desactivada se asociaría al *Evil Twin* de forma automática y silenciosa, sin mostrar advertencia ni pedir confirmación, lo que lo haría aún más peligroso. De hecho, el diálogo de aceptación manual del certificado no llegó a Android hasta el propio mecanismo TOFU. Desde Android 12, la validación obligatoria del certificado aproximaría su comportamiento al observado en el Poco M8.

Conviene cerrar este apartado reconociendo su principal limitación. Del eco-

sistema Android solo ha podido probarse un dispositivo, el Poco M8 5G, al no disponer de más equipos ni de otras versiones del sistema. Por ese motivo, cuanto se ha expuesto sobre el comportamiento de las versiones anteriores a la 13 es una conclusión obtenida a partir de la investigación documental, y no una observación realizada en el laboratorio. Para darla por confirmada haría falta repetir el experimento sobre un conjunto más amplio de teléfonos y de versiones de Android, comprobando empíricamente cada caso, lo que queda planteado como una línea de trabajo pendiente.

5.2.3. Portátiles con Windows 11

En el lado de Windows se han probado dos ordenadores portátiles, un Alienware x17 R1 y un Dell XPS 13 9350, ambos con Windows 11 Business. De nuevo, los dos se comportan igual entre sí, aunque el resultado al que llevan se aparta del observado en los dispositivos de Apple.

Al igual que en los dispositivos de Apple, el equipo parte de una conexión normal a la red legítima, con su certificado ya aceptado y acceso pleno a internet. Al desplazarlo hacia el *Evil Twin*, Windows tampoco se reasocia por sí solo: muestra un aviso con el texto “Se requiere una acción” junto a la red, que indica que la conexión necesita la intervención del usuario. Al pulsar sobre ese aviso y, después, sobre el botón de conexión, que es la forma en que Windows pide que se acepte el certificado del servidor, el equipo da por bueno el certificado del atacante y completa la asociación al punto de acceso gemelo.

La diferencia con los dispositivos de Apple aparece en el método de autenticación interno. Los dos portátiles con Windows rechazan el *downgrade* a EAP-GTC y obligan a continuar con MSCHAPv2, de modo que la contraseña no llega a transmitirse en claro. Lo que queda registrado en el *log* de captura es el intercambio desafío-respuesta de MSCHAPv2, del que se deriva un *hash* de tipo NetNTLMv1. El ataque tiene éxito igualmente, pero la credencial obtenida queda cifrada y debe someterse después a un proceso de *cracking* fuera de línea, con hashcat en modo 5500 o con john, para recuperar la contraseña original.

5.2.4. Lectores electrónicos Kindle

Se han incluido en las pruebas dos lectores de libros electrónicos Kindle, un tipo de dispositivo muy distinto de los teléfonos y portátiles anteriores. El primero, con la versión de *firmware* 4.1.4, ni siquiera llegó a entrar en el escenario de ataque: su gestor de red no admite la conexión a redes *enterprise*, de modo que no hay forma de asociarlo a una red 802.1X ni, por tanto, al *Evil Twin*. La ausencia de captura se debe aquí únicamente a esa limitación técnica del *firmware*, que deja al dispositivo fuera del alcance del ataque.

El segundo Kindle, con la versión de *firmware* 5.13.6, sí admite redes WPA2-Enterprise, pero su forma de gestionarlas frustra el ataque. Durante la configuración de la red exige que se indique manualmente la autoridad de certificación con la que ha de validarse el servidor, y no contempla aceptar un certificado arbitrario en el momento de conectarse. Cuando el equipo entra en la cobertura del *Evil Twin*, rechaza el certificado del atacante por no coincidir con la CA configurada y el túnel TLS no llega a abrirse, de modo que no se registra ninguna credencial en el *log* de captura. Este Kindle recae en el tercer desenlace, el de la mitigación efectiva, al igual que el Poco M8 aunque por un mecanismo distinto: aquí es el propio gestor de red el que obliga a validar la CA durante la configuración, sin ofrecer la vía insegura de aceptar a mano un certificado arbitrario que sí admiten los equipos de Apple y los portátiles con Windows.

5.3. Síntesis de los resultados

Recopilado el comportamiento de cada equipo, la tabla siguiente reúne los resultados de todos los dispositivos probados, con su sistema operativo, su versión y el desenlace del ataque, y permite una lectura de conjunto.

Dispositivo	Sistema operativo	Versión	Comportamiento ante el ataque
iPhone 16 Pro	iOS	27.0 (beta de desarrollador)	Contraseña en claro: acepta el <i>downgrade</i> a EAP-GTC.
iPhone 11	iOS	26.5	
iPhone 12 Pro Max	iOS	26.4.2	
MacBook Pro (M1 Max)	macOS	Tahoe 26.5.1	
Poco M8 5G	Android (HyperOS)	Android 16 / HyperOS 3.0.301.0. WPQEUXM.C08	Mitigado: ancla el certificado de forma automática (TOFU).
Alienware x17 R1	Windows 11	Business	Captura de <i>hash</i> NetNTLMv1: retrocede a MSCHAPv2.
Dell XPS 13 9350	Windows 11	Business	
Kindle 4th Generation	Kindle	<i>Firmware</i> 4.1.4	Fuera de alcance: no admite redes WPA2-Enterprise.
Kindle 7th Generation	Kindle	<i>Firmware</i> 5.13.6	Mitigado: exige validar la CA manualmente.

Tabla 5.1: Resumen de los dispositivos sometidos al ataque, con su sistema operativo, su versión y el desenlace observado. Los equipos de Apple entregan la contraseña en claro y los portátiles con Windows un *hash* NetNTLMv1 crackeable; el Poco M8 y el Kindle con *firmware* 5.13.6 mitigan el ataque al validar el certificado del servidor, y el Kindle con *firmware* 4.1.4 queda fuera de alcance por no admitir redes WPA2-Enterprise.

De esa visión de conjunto se desprende el patrón que vertebra todo el trabajo. Los dispositivos que aceptan el certificado del atacante terminan entregando la credencial, ya sea en claro (los equipos de Apple) o como un *hash* crackeable (los portátiles con Windows), y la diferencia entre ambos desenlaces depende solo del método interno que cada uno admite. Los equipos que resisten son los que validan estrictamente el certificado del servidor, y en los dos casos observados esa validación la impone el propio dispositivo: el Kindle con *firmware* 5.13.6 la exige durante la configuración y el Poco M8 la aplica de forma automática mediante TOFU. El experimento confirma así que la frontera entre caer y resistir se reduce a una única condición, que se valide o no la identidad del servidor, con independencia de la gama o el precio del equipo.

6

Conclusiones y trabajos futuros

Una vez construido el laboratorio y puesta a prueba la plataforma frente a distintos clientes, conviene cerrar la memoria volviendo la vista atrás sobre lo que se planteó al principio. Este último capítulo recoge una reflexión honesta sobre hasta qué punto se han cumplido los objetivos definidos al comienzo del trabajo y cómo se han materializado, la valoración personal del autor sobre lo conseguido y, por último, las líneas por las que el proyecto podría seguir creciendo, ordenadas según el horizonte temporal en que tendrían sentido.

6.1. Grado de cumplimiento de los objetivos

Para valorar lo conseguido conviene retomar el objetivo principal tal como se enunció: demostrar de forma empírica, sobre un laboratorio completamente aislado, que una red WPA2-Enterprise del tipo **eduroam** puede ser comprometida mediante un ataque *Evil Twin* cuando el cliente no valida correctamente el certificado del servidor RADIUS. Ese objetivo se ha cumplido. Se ha montado una red legítima y, frente a ella, una plataforma atacante portátil capaz de levantar un punto de acceso gemelo con el mismo SSID, negociar el túnel PEAP y quedarse con las credenciales del cliente que pica el anzuelo. Lo que al principio era una debilidad descrita en la literatura [21, 19] se ha convertido en un montaje real, reproducible y funcionando de principio a fin sobre hardware de apenas unas decenas de euros, con todos sus artefactos publicados en un repositorio público [31].

Esa demostración, además, se ha materializado exactamente sobre el tipo de

plataforma que se buscaba: portátil, autónoma y de bajo coste. Todo el ataque corre sobre una Raspberry Pi 5 con una antena Alfa AWUS036ACM, se alimenta de una simple batería externa y no necesita un equipo supervisor ni intervención del operador una vez encendido. Con ello se confirma la tesis de partida, esto es, que el coste y la dificultad de montar este ataque son hoy mínimos y que, por tanto, la amenaza es bastante más realista de lo que sugiere la percepción habitual.

A un nivel más concreto, los objetivos secundarios de carácter técnico también se han ido cumpliendo uno a uno:

- La red legítima quedó montada sobre una máquina virtual Ubuntu Server con FreeRADIUS estándar y certificado propio, emitiendo el SSID `eduroam-tfg`, tal como se planteó.
- La plataforma atacante se construyó sobre la Raspberry Pi 5 partiendo de PatataWiFiEnterprise [28] y se reorganizó en torno a dos radios físicas independientes (la radio interna de la Pi para la red de gestión y la antena Alfa para el *Evil Twin*), modernizando una herramienta que arrastraba dependencias de 2015.
- El *backend* de captura se actualizó sustituyendo el FreeRADIUS 2.x heredado por FreeRADIUS-WPE 3.2.5, con lo que se evita compilar versiones antiguas de OpenSSL y se trabaja sobre una base actual y soportada.
- Se capturan los dos tipos de credencial previstos, tanto la contraseña en claro, forzando el *downgrade* a EAP-GTC dentro del túnel PEAP, como el *hash* NetNTLMv1, gracias al *fallback* automático a MSCHAPv2, ambos en un formato directamente atacable con hashcat (modo 5500) o john.
- Todo el ciclo de vida quedó automatizado mediante los tres servicios `systemd` encadenados (limpieza → gestión → ataque), que levantan las dos redes de forma desatendida unos treinta segundos después del arranque, con una red de gestión WPA2-PSK independiente para administrar el equipo mientras el ataque está en marcha.

Conviene añadir que la forma concreta que adoptó la plataforma (las dos radios físicas, las versiones de FreeRADIUS-WPE y `hostapd`, el direccionamiento de cada subred o la cadena de tres servicios `systemd`) vino dictada en cada caso por una limitación real del hardware, del software de base o del entorno de trabajo, y responde por tanto a decisiones razonadas surgidas durante la propia construcción del laboratorio.

Sobre ese montaje se han podido reproducir y documentar los tres escenarios de cliente que se buscaban: el que acepta el certificado y GTC y entrega la contraseña en claro, el que rechaza GTC y cae a MSCHAPv2 dejando un *hash* crackeable

offline, y el que tiene el certificado anclado mediante CA *pinning* y rechaza la conexión antes incluso de abrir el túnel, único caso en el que el ataque no obtiene nada. Precisamente ese último escenario es la mejor confirmación de la tesis que se defendía, ya que la única defensa que funciona de verdad (la validación estricta del certificado del servidor, idealmente forzada mediante perfiles como **eduroam** CAT [18]) está disponible desde hace años y, aun así, sigue siendo opcional en la mayoría de instituciones.

En cuanto a los objetivos formativos, también se dan por cumplidos. El recorrido ha obligado a entender de primera mano cómo funcionan las redes Wi-Fi y cómo ha evolucionado su seguridad, a manejar a fondo WPA2/WPA3-Enterprise, el estándar 802.1X y el papel del servidor RADIUS, a comprender los distintos métodos EAP y el lugar central que ocupan los certificados como ancla de confianza de todo el sistema, y a adquirir experiencia práctica tanto en la auditoría de redes *enterprise* con herramientas reales (hostapd, FreeRADIUS-WPE, dnsmasq) como en el *cracking offline* de credenciales y en la administración de sistemas Linux. Todo ese aprendizaje, que al principio era un objetivo en sí mismo, ha acabado siendo la base sobre la que se ha sostenido la parte técnica del trabajo.

6.2. Valoración personal

Por encima del detalle técnico, la valoración que se hace es que se trata de un proyecto completo. La plataforma no se ha quedado a medias ni es una prueba de concepto que solo funcione sobre el papel: enciende, levanta las dos redes por sí sola y ejecuta el ataque de principio a fin, capturando credenciales reales sobre el laboratorio. Con lo que existe ahora mismo, el ataque ya es posible y está al alcance de cualquiera, de modo que todo lo que pueda añadirse a partir de aquí no cambia esa realidad, simplemente la hace más cómoda. Las líneas de trabajo futuro que se proponen a continuación van precisamente en esa dirección, ya que facilitan la operación y mejoran el acabado del dispositivo, pero no habilitan nada que hoy no se pueda hacer ya, porque el ataque está disponible.

Quizás lo que más impresiona, una vez recorrido todo el camino, es que toda la solidez criptográfica de WPA2/WPA3-Enterprise, que sobre el papel es impecable, acaba descansando sobre una única decisión que casi siempre queda en manos del usuario final: confiar o no en el certificado que le presenta la red. Que un trabajo de fin de grado, con hardware comercial y software libre, sea capaz de evidenciar esa fragilidad de forma tan directa es, para el autor, el resultado más valioso del proyecto y la mejor justificación de su carácter de concienciación.

6.3. Concienciación y recomendaciones para las instituciones

Por encima del balance técnico, el trabajo deja una conclusión que conviene subrayar, ya que es la que sostiene su vocación de concienciación. Lo que se ha reproducido en el laboratorio es una vulnerabilidad real y de gran alcance, y no un caso de estudio aislado. **eduroam** es hoy una de las infraestructuras de itinerancia más extendidas del mundo académico: superó los ocho mil millones de autenticaciones a lo largo de 2024 [17] y está presente en la práctica totalidad de las universidades españolas a través de RedIRIS [13]. Sobre esa escala, que un dispositivo de fácil acceso pueda capturar credenciales válidas siempre que el cliente no valide el certificado del servidor RADIUS eleva un detalle de configuración a la categoría de problema de seguridad de primer orden. Los estudios sobre despliegues reales de **eduroam** confirman, además, que la configuración insegura del lado del cliente sigue siendo frecuente [14].

Lo que convierte esto en una cuestión de concienciación es que la solución existe desde hace años y está perfectamente documentada. La defensa que de verdad funciona, anclar en el cliente el certificado y la autoridad de certificación que debe presentar el servidor legítimo, está al alcance de cualquier institución mediante herramientas oficiales y gratuitas. **eduroam** CAT [18] genera instaladores que dejan el dispositivo configurado con el certificado correcto ya anclado, y la aplicación **geteduroam** [41] hace lo propio sobre dispositivos móviles de forma guiada. Cuando el cliente queda así configurado, el ataque reproducido en este trabajo fracasa antes incluso de abrir el túnel TLS, tal y como se comprobó en el escenario con anclaje del certificado. El obstáculo, por tanto, no es tecnológico.

El obstáculo está en que esa configuración segura sigue siendo, en la mayoría de los casos, opcional y queda en manos del usuario final. Muchas instituciones continúan publicando guías de conexión manual en las que se pide introducir a mano el SSID, el método EAP y el dominio, omitiendo el anclaje del certificado o delegándolo en quien rara vez está en condiciones de valorar esa decisión. Mientras la vía insegura siga disponible y resulte además la más cómoda, una parte de los usuarios la seguirá tomando. La labor pendiente es, por tanto, de concienciación y de política institucional antes que de ingeniería.

A la luz de lo observado en el laboratorio, las recomendaciones para las instituciones que ofrecen **eduroam** pueden resumirse en los siguientes puntos:

- Hacer obligatoria la configuración mediante herramientas oficiales. El alta de cualquier dispositivo debería pasar siempre por **eduroam** CAT [18] o **geteduroam** [41], de modo que el certificado quede anclado sin intervención del usuario, retirando o desaconsejando de forma activa las guías de configuración manual que omiten ese paso.

- Convertir la opción segura en la opción por defecto. La configuración correcta debe ser la más sencilla de seguir y la única que se promociona, ya que cualquier alternativa manual que el usuario perciba como más rápida acabará empleándose.
- Acompañar el despliegue con campañas de concienciación. Conviene explicar a la comunidad, en un lenguaje accesible, por qué aceptar sin más un certificado desconocido equivale a entregar las credenciales y por qué el aviso que muestra el sistema operativo en la primera conexión no es un mero trámite.
- Establecer una revisión periódica. La configuración de los clientes y de los servidores RADIUS debería auditarse con regularidad, comprobando que los perfiles distribuidos siguen anclando el certificado correcto y atendiendo a los avisos técnicos que publica la propia federación [42, 43].

En definitiva, la seguridad de **eduroam** no depende tanto de la criptografía del protocolo, que es sólida, como de una decisión de configuración que hoy sigue recayendo demasiado a menudo en el usuario. La solución lleva años disponible, lo que falta es hacerla obligatoria, mantenerla en el tiempo mediante revisiones periódicas y explicar a la comunidad por qué importa.

6.4. Trabajos futuros

Aunque se da el trabajo por cerrado en cuanto a sus objetivos, el proyecto deja varias puertas abiertas que merecería la pena explorar y que conviene ordenar según el horizonte temporal en que tendrían sentido.

A corto plazo, la mejora más inmediata sería dotar a la plataforma de una interfaz web de control. Hoy, administrar el ataque obliga a conectarse por SSH a la red de gestión y a trabajar contra la sesión **tmux** del orquestador, lo cual es perfectamente funcional pero poco cómodo. La idea es levantar una pequeña aplicación web, accesible desde el navegador sin más que conectarse a la red de gestión WPA2-PSK de la propia Raspberry Pi, desde la que poder controlar los parámetros del ataque (el SSID objetivo, los métodos EAP que se fuerzan, o el arranque y la parada de las redes) y consultar en tiempo real las credenciales que se van capturando, todo ello sin necesidad de tocar la línea de comandos.

A medio plazo, el siguiente paso natural es convertir el conjunto en un dispositivo verdaderamente portátil y autocontenido. En su estado actual la plataforma cumple con su cometido, pero físicamente no es más que una Raspberry Pi con su antena Alfa conectada, a la que se le acopla una batería externa: no tiene caja, ni fuente de alimentación propia, ni batería integrada, de modo que para transportarla hay que llevar las tres piezas sueltas. Sería interesante diseñar una carcasa

cerrada que integrara la placa, la antena y una batería interna, con refrigeración activa mediante ventilador para aguantar el ataque funcionando durante horas, de manera que todo el sistema quedara dentro de una única caja lista para encender y meter en una mochila.

A largo plazo, y ya con más recursos, el trabajo podría crecer en profundidad y en alcance. Por un lado, ampliando el estudio del comportamiento del cliente a un abanico mucho mayor de dispositivos, versiones de sistema operativo y suplicantes, para tener un mapa más completo de cuáles entregan la contraseña, cuáles caen a MSCHAPv2 y cuáles resisten gracias a un perfil bien configurado. Por otro, dándole la vuelta al planteamiento y usando la misma plataforma con un fin defensivo, ya sea para detectar la presencia de gemelos malvados en un entorno o para servir de banco de pruebas con el que validar que los perfiles de **eduroam** CAT [18] y el anclaje de certificado protegen de verdad a los usuarios de una institución. En último término, todo el material generado podría reaprovecharse como recurso educativo y de concienciación, que es, al fin y al cabo, el propósito que ha movido el trabajo desde el principio.

Bibliografía

- [1] Purple, “The definitive timeline of wifi: From alohanet to wifi 7 and beyond | technical guides,” 2024. [Online]. Available: <https://www.purple.ai/en-gb/guides/the-definitive-timeline-of-wifi-from-alohonet-to-wifi-7-and-beyond>
- [2] Cisco News The EMEA Network, “Wi-fi is 20 years old – here’s 20 milestones in wi-fi’s history,” 2019. [Online]. Available: <https://news-blogs.cisco.com/emea/2019/09/30/20-years-of-wi-fi/>
- [3] D. Ferro and B. Rink, “Understanding technology options for deploying wi-fi,” Ubee Interactive / Society of Cable Telecommunications Engineers, accedido en 2026. [Online]. Available: <https://euro.ecom.cmu.edu/resources/elibrary/auto/Understanding.pdf>
- [4] Wikipedia contributors, “Ieee 802.11 - wikipedia,” accedido en 2026. [Online]. Available: https://en.wikipedia.org/wiki/IEEE_802.11
- [5] A. Myles, “20 years of wireless with the wi-fi alliance,” Cisco Blogs, 2019. [Online]. Available: <https://blogs.cisco.com/networking/20-years-of-wireless-with-the-wi-fi-alliance>
- [6] M. P. Espinoza Martínez, F. R. Orozco Lara, A. B. Vásquez Díaz, M. D. Hernández Ponce, and W. A. Rodríguez López, “Análisis de una red inalámbrica bajo el estándar wi-fi 6 gestionado con sdn: caso de estudio labomedica,” *GADE: Revista Científica*, vol. 5, no. 3, pp. 220–238, 2025. [Online]. Available: <https://doi.org/10.63549/rg.v5i3.706>
- [7] Wikipedia contributors, “Wi-fi 7 - wikipedia,” accedido en 2026. [Online]. Available: https://en.wikipedia.org/wiki/Wi-Fi_7
- [8] IEEE 802.11 Working Group, “IEEE 802.11bn (task group bn) status update,” 2025, accedido en 2026. [Online]. Available: https://www.ieee802.org/11/Reports/tgbn_update.htm
- [9] B. Robinson, “How secure is wi-fi really?” World Wide Technology (WWT), 2019. [Online]. Available: <https://www.wwt.com/article/how-secure-is-wifi-really>
- [10] M. Amhiyen, “Wpa2/3-enterprise: Secure wi-fi with 802.1x authentication,” Cloudi-Fi, 2025. [Online]. Available: <https://www.cloudi-fi.com/blog/wpa2-enterprise-802-1x>
- [11] Super User, “802.1x: What exactly is it regarding wpa and eap?” 2012. [Online]. Available: <https://superuser.com/q/373453>
- [12] GÉANT Association, “How does eduroam work?” 2024, accedido en 2026. [Online]. Available: <https://eduroam.org/how/>
- [13] RedIRIS, “eduroam.es - iniciativa de movilidad en la red académica española,” 2024, actualizado el 16/12/2024. [Online]. Available: <https://www.rediris.es/servicios/movilidad/eduroam/index.html.es>
- [14] L. M. Batista, H. Gomes, and J. R. Almeida, “A study of security issues of eduroam networks in portugal,” in *13th Symposium on Languages, Applications and Technologies (SLATE 2024)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024.

- [15] K. Meyer, “eduroam® and wi-fi certified wpa3™ security,” eduroam.org, June 2018. [Online]. Available: <https://eduroam.org/eduroam-and-wpa3/>
- [16] eduroam South Africa, “Faq for participating institutions/administrators,” 2024. [Online]. Available: <https://eduroam.ac.za/faq/admin/>
- [17] K. Meyer, “eduroam hits a new record – 8.4 billion authentications in 2024,” eduroam.org, February 2025. [Online]. Available: <https://www.eduroam.org/eduroam-hits-a-new-record-8-4-billion-authentications-in-2024/>
- [18] eduroam, “eduroam cat – configuration assistant tool,” 2025. [Online]. Available: <https://cat.eduroam.org/>
- [19] G. Bellicini, “Evilroam: Analysis and deployment of evil-twins-based attacks to eduroam network users’ security,” 2021. [Online]. Available: https://ans.unibs.it/assets/documents/thesis/bellicini_tesi.pdf
- [20] Y. Daldoul, “A robust certificate management system to prevent evil twin attacks in ieee 802.11 networks,” 2023. [Online]. Available: <https://arxiv.org/pdf/2302.00338>
- [21] I. Palamà, A. Amici, F. Gringoli, and G. Bianchi, ““careful with that roam, edu”: Experimental analysis of eduroam credential stealing attacks,” in *2022 17th Wireless On-Demand Network Systems and Services Conference (WONS)*, 2022, pp. 1–7. [Online]. Available: <https://dl.ifip.org/db/conf/wons/wons2022/wons22-final65.pdf>
- [22] C. Yang, Y. Song, and G. Gu, “Active user-side evil twin access point detection using statistical techniques,” *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 5, pp. 1638–1651, 2012.
- [23] Z. Tang, Y. Zhao, L. Yang, S. Qi, D. Fang, X. Chen, X. Gong, and Z. Wang, “Exploiting wireless received signal strength indicators to detect evil-twin attacks in smart homes,” *Mobile Information Systems*, vol. 2017, pp. 1–14, 2017.
- [24] F. Lanze, A. Panchenko, B. Braatz, and T. Engel, “Letting the puss in boots sweat: Detecting fake access points using dependency of clock skews on temperature,” in *Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security (ASIA CCS)*, 2014, pp. 3–14.
- [25] CANARIE, “Caf – evil-twin eaphammer attack mitigation for eduroam,” 2021. [Online]. Available: <https://www.canarie.ca/document/evil-twin-eaphammer-attack-mitigation-for-eduroam/>
- [26] eduroam, “Configuration assistant tool (cat),” 2025. [Online]. Available: <https://eduroam.org/configuration-assistant-tool-cat/>
- [27] AARNet, “What is the eduroam configuration assistant tool (cat)?” 2024. [Online]. Available: <https://support.aarnet.edu.au/hc/en-us/articles/13362857245967-What-is-the-eduroam-Configuration-Assistant-Tool-CAT>
- [28] J. Anton, “Patatawifienterprise: Tool for wpa enterprise hacking,” 2024. [Online]. Available: <https://github.com/jesux/PatataWiFiEnterprise>
- [29] —, “patatawifi-install.sh,” 2024. [Online]. Available: <https://github.com/jesux/PatataWiFiEnterprise/blob/master/patatawifi-install.sh>
- [30] Q. Vacher, “Wireless-(in)fidelity: Pentesting wi-fi in 2025,” Synacktiv, Jan. 2026, publicado el 14 de enero de 2026. [Online]. Available: <https://www.synacktiv.com/en/publications/wireless-infidelity-pentesting-wi-fi-in-2025>
- [31] A. C. Fleury, “Patatawifi-tfg: laboratorio reproducible de auditoría wpa2-enterprise mediante evil twin sobre eduroam,” 2026, repositorio público con los *scripts* de instalación, los parches y los artefactos de configuración del laboratorio. [Online]. Available: <https://github.com/XelaJr/PatataWifi-TFG>

- [32] Felix Fietkau and the Linux mt76 project, “mt76: Linux kernel driver for mediatek 802.11ac/ax USB and PCIe WLAN chipsets,” 2024. [Online]. Available: <https://github.com/openwrt/mt76>
- [33] J. Malinen, “hostapd: IEEE 802.11 AP, IEEE 802.1x/wpa/wpa2/eap/radius authenticator,” 2016, versión 2.6. [Online]. Available: <https://w1.fi/hostapd/>
- [34] Kali Linux, “freeradius-wpe (kali tools),” 2025. [Online]. Available: <https://www.kali.org/tools/freeradius-wpe/>
- [35] J. Wright and B. Antoniewicz, “Freeradius-wpe: Wireless pwn edition,” 2024, edición de captura de credenciales de FreeRADIUS; empaquetada en Kali Linux como el paquete **freeradius-wpe** 3.2.5+dfsg-3kali1. [Online]. Available: <https://github.com/brad-anton/freeradius-wpe>
- [36] J. Steube and hashcat project, “hashcat: advanced password recovery (modo 5500, netntlmv1/v1+ess),” 2024. [Online]. Available: <https://hashcat.net/hashcat/>
- [37] M. Marlinspike and D. Hulton, “Defeating PPTP VPNs and WPA2 enterprise with MS-CHAPv2,” in *DEF CON 20*, 2012. [Online]. Available: <https://www.youtube.com/watch?v=gkPvZDcrLFk>
- [38] Android Open Source Project, “Trust on first use (TOFU) for Wi-Fi Enterprise networks,” 2024. [Online]. Available: <https://source.android.com/docs/core/connect/wifi-tofu>
- [39] M. Rahman, “Android 11 will no longer let you connect to some enterprise Wi-Fi networks,” 2021. [Online]. Available: <https://www.xda-developers.com/android-11-break-enterprise-wifi-connection/>
- [40] V. Raj, “How to validate server certificates on Android 12,” 2024. [Online]. Available: <https://www.securew2.com/blog/server-certificate-validation-with-android-12-devices>
- [41] GÉANT Association, “geteduroam – get connected quickly and safely,” 2024. [Online]. Available: <https://eduroam.org/geteduroam-get-connected-quickly-and-safely/>
- [42] eduroam ES, “Technical advisories,” 2022. [Online]. Available: <https://www.eduroam.es/technical-advisories/>
- [43] —, “eduroam deployment,” 2022. [Online]. Available: <https://www.eduroam.es/technical-advisories/eduroam-deployment>

Apéndice



Configuraciones y artefactos del laboratorio

Este apéndice reproduce de forma íntegra los ficheros de configuración, los *scripts* de orquestación, las unidades `systemd` y los parches que materializan el laboratorio descrito en el Capítulo 4. Los contenidos se transcriben *verbatim* a partir de los artefactos reales del entorno, salvo las credenciales y secretos de la red legítima, que se han sustituido por los marcadores [SECRETO] por imperativo ético. Se conservan, en cambio, los valores `testing123` y `patatas333`: ambos son parámetros públicos de laboratorio heredados del proyecto *upstream* y no constituyen secreto alguno. El material se agrupa en dos secciones, una por cada fase de la arquitectura. Todos estos ficheros, junto con los *scripts* de instalación y desinstalación que automatizan el despliegue, están disponibles en el repositorio público del proyecto [31].

A.1. Fase 1: red legítima

Los listados de esta sección corresponden al punto de acceso WPA2-Enterprise legítimo levantado sobre la máquina virtual Ubuntu Server con FreeRADIUS 3.x, y reproducen tanto la configuración del `daemon hostapd` como la del servidor RADIUS y la capa de red.

El siguiente listado recoge la configuración completa de `hostapd` que define el punto de acceso institucional `eduroam-tfg` en modo WPA2-Enterprise, delegando la autenticación 802.1X en el FreeRADIUS local. El secreto compartido se ha redactado.

```
# =====  
# TFG Alejandro - AP legitimo WPA2-Enterprise  
# Simula un AP institucional eduroam-like  
# =====  
  
# --- Interfaz fisica ---
```

```
interface=wlx00c0cab7ef58
driver=nl80211

# --- Identidad de la red ---
ssid=eduroam-tfg
country_code=ES
ieee80211d=1

# --- Capa fisica ---
hw_mode=g
channel=6

# --- Seguridad: WPA2-Enterprise (802.1X + EAP) ---
auth_algs=1
wpa=2
wpa_key_mgmt=WPA-EAP
rsn_pairwise=CCMP
ieee8021x=1

# --- Conexion al servidor RADIUS (FreeRADIUS en localhost) ---
auth_server_addr=127.0.0.1
auth_server_port=1812
auth_server_shared_secret=[SECRETO]

acct_server_addr=127.0.0.1
acct_server_port=1813
acct_server_shared_secret=[SECRETO]

# --- Identificacion del NAS ---
nas_identifier=tfg-ap-legitimo
own_ip_addr=127.0.0.1

# --- Logging para depuracion / capturas ---
logger_syslog=-1
logger_syslog_level=2
logger_stdout=-1
logger_stdout_level=2

# --- Buenas practicas ---
ignore_broadcast_ssid=0
macaddr_acl=0
```

Código A.1: /etc/hostapd/hostapd.conf (AP legítimo WPA2-Enterprise)

El módulo EAP de FreeRADIUS se ajusta para que el tipo EAP por defecto sea PEAP (en lugar del valor de fábrica md5) y para que el material criptográfico apunte a la PKI propia generada por los *scripts* de *bootstrap*. El fragmento relevante de `mods-available/eap` (enlazado desde `mods-enabled/eap`) es el siguiente.

```
default_eap_type = peap
private_key_file = /etc/freeradius/3.0/certs/server.key
certificate_file = /etc/freeradius/3.0/certs/server.pem
ca_file = /etc/freeradius/3.0/certs/ca.pem
```

Código A.2: Fragmento de `/etc/freeradius/3.0/mods-available/eap`

La declaración del cliente RADIUS reutiliza el bloque `client localhost` preexistente (en lugar de declarar un NAS nuevo) para evitar la colisión del cargador de FreeRADIUS por coincidencia de `ipaddr` y `nas_type`. Se endurece el secreto compartido (aquí redactado) y se activa `require_message_authenticator` como mitigación de la vulnerabilidad BlastRADIUS.

```
client localhost {
    ipaddr = 127.0.0.1
    proto = *
    secret = [SECRETO]
    require_message_authenticator = yes
    # ... el resto sin tocar
}
```

Código A.3: Fragmento de `/etc/freeradius/3.0/clients.conf`

El usuario de prueba se define en el fichero `users`, con su credencial en claro redactada. El espaciado original emplea tabuladores entre el identificador y los atributos.

```
alejandro      Cleartext-Password := "[SECRETO]"
                Reply-Message := "Bienvenido al TFG"
```

Código A.4: Entrada de `/etc/freeradius/3.0/users` (usuario de prueba)

La distribución de direccionamiento y resolución de nombres a los clientes de la red 10.50.0.0/24 corre a cargo de `dnsmasq`, cuya configuración completa se reproduce a continuación.

```
# =====
# TFG Alejandro - dnsmasq para AP eduroam-tfg
# =====

# Solo escuchar en la interfaz Wi-Fi (no en ens33 ni en lo)
interface=wlx00c0cab7ef58
bind-interfaces

# No usar /etc/resolv.conf del sistema (evita loops con systemd-resolved)
no-resolv

# DNS upstream - usar Cloudflare y Google publicos
server=1.1.1.1
server=8.8.8.8

# DHCP pool: 10.50.0.10 - 10.50.0.100, lease de 12h
dhcp-range=10.50.0.10,10.50.0.100,255.255.255.0,12h

# Opciones DHCP para los clientes
dhcp-option=3,10.50.0.1          # Gateway por defecto
dhcp-option=6,10.50.0.1          # DNS = nosotros mismos

# Domain name (cosmetico, aparece en algunos clientes)
domain=tfg-loyola.local
```

```
# Logging
log-dhcp
log-queries
```

Código A.5: `/etc/dnsmasq.conf` (DHCP + DNS de la red legítima)

La preparación de la antena antes de `hostapd` se delega en una unidad `systemd` de tipo `oneshot` que realiza un ciclo de reinicio de la interfaz para sortear la inestabilidad del *driver*. El siguiente listado reproduce dicha unidad.

```
[Unit]
Description=Preparar AWUS036AXM antes de hostapd
Wants=sys-subsystem-net-devices-wlx00c0cab7ef58.device
After=sys-subsystem-net-devices-wlx00c0cab7ef58.device
Before=hostapd.service
ConditionPathExists=/sys/class/net/wlx00c0cab7ef58

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/sbin/rfkill unblock all
ExecStart=/usr/sbin/ip link set wlan0c0cab7ef58 down
ExecStart=/usr/sbin/iw dev wlan0c0cab7ef58 set type managed
ExecStart=/usr/sbin/ip link set wlan0c0cab7ef58 up
ExecStart=/usr/sbin/ip link set wlan0c0cab7ef58 down

[Install]
WantedBy=multi-user.target
```

Código A.6: `/etc/systemd/system/wlan-prepare.service`

El servicio `hostapd` del sistema se complementa con un *drop-in* de *override* que añade las dependencias de arranque (FreeRADIUS y la preparación de la antena) y habilita el reinicio automático ante fallos.

```
[Unit]
Wants=freeradius.service wlan-prepare.service
After=freeradius.service wlan-prepare.service

[Service]
Restart=on-failure
RestartSec=5
```

Código A.7: `/etc/systemd/system/hostapd.service.d/override.conf`

La asignación de la dirección IP de puerta de enlace a la interfaz del AP se realiza con una segunda unidad `oneshot`, ligada al ciclo de vida de `hostapd` mediante `BindTo`, de modo que la IP solo exista mientras el punto de acceso esté activo.

```
[Unit]
Description=Asignar IP estatica a la interfaz AP
After=hostapd.service
```

```
Bindsto=hostapd.service
ConditionPathExists=/sys/class/net/wlx00c0cab7ef58

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/sbin/ip addr add 10.50.0.1/24 dev wlx00c0cab7ef58
ExecStart=/usr/sbin/ip link set wlx00c0cab7ef58 up
ExecStop=/usr/sbin/ip addr flush dev wlx00c0cab7ef58

[Install]
WantedBy=multi-user.target
```

Código A.8: /etc/systemd/system/wlan-ip.service

Por último, el reenvío de paquetes IPv4 necesario para que los clientes naveguen por NAT se habilita de forma persistente mediante el siguiente fichero `sysctl`.

```
net.ipv4.ip_forward=1
```

Código A.9: /etc/sysctl.d/99-tfg-forward.conf

A.2. Fase 2: plataforma atacante

Los artefactos de esta sección proceden del repositorio público del laboratorio atacante [31], fuente autoritativa del Capítulo 4 y preparado para reproducir el montaje completo. Cubren la configuración de la red de gestión, los tres *scripts* de orquestación y sus unidades `systemd` encadenadas, así como los parches aplicados sobre FreeRADIUS-WPE y sobre el *upstream* PatataWiFi.

La red de gestión WPA2-PSK PatataWiFi_mgmt, levantada sobre la radio interna wlan0, se configura con el siguiente fichero. La clave patatas333 es un valor público de laboratorio heredado del *upstream*.

```
interface=wlan0
driver=nl80211
ctrl_interface=/var/run/hostapd-mgmt
ctrl_interface_group=0
ssid=PatataWiFi_mgmt
country_code=ES
hw_mode=g
channel=1
ieee80211n=1
wmm_enabled=1
auth_algs=1
ignore_broadcast_ssid=0
wpa=2
wpa_key_mgmt=WPA-PSK
wpa_pairwise=CCMP
rsn_pairwise=CCMP
wpa_passphrase=patatas333
```

Código A.10: hostapd-mgmt/mgmt.conf (red de gestión)

El primer *script* de la cadena, `tfg-cleanup.sh`, realiza una limpieza total previa al arranque: termina los procesos relacionados, baja las interfaces y recarga el *driver* `mt76x2u` para sanear el estado del *kernel*.

```
#!/bin/bash
# tfg-cleanup.sh - Limpieza total antes de arrancar el laboratorio

LOG=/var/log/tfg-cleanup.log
echo "=== $(date) === Inicio cleanup ===" >> $LOG

# Matar todos los procesos relacionados
for proc in hostapd radiusd freeradius freeradius-wpe dnsmasq; do
    pkill -9 -x $proc 2>/dev/null
done
pkill -9 -f patatawifi 2>/dev/null
tmux kill-server 2>/dev/null

sleep 2

# Reset wlan0 (interna, gestion)
ip link set wlan0 down 2>/dev/null
ip addr flush dev wlan0 2>/dev/null

# Reset wlan1 (Alfa, ataque)
ip link set wlan1 down 2>/dev/null
iw dev wlan1 del 2>/dev/null

# Recargar driver MT7612U para limpiar estado kernel
rmmod mt76x2u 2>/dev/null
sleep 2
modprobe mt76x2u
sleep 3

# Detener NetworkManager y wpa_supplicant (interfieren con hostapd)
systemctl stop NetworkManager 2>/dev/null
systemctl stop wpa_supplicant 2>/dev/null

# Eliminar PIDs huérfanos
rm -f /tmp/dnsmasq-mgmt.pid /tmp/hostapd-mgmt.log /tmp/dnsmasq-mgmt.log 2>/dev/null
rm -rf /var/run/hostapd /var/run/hostapd-mgmt 2>/dev/null

echo "=== $(date) === Cleanup OK ===" >> $LOG
exit 0
```

Código A.11: scripts/tfg-cleanup.sh

El segundo *script*, `tfg-mgmt.sh`, levanta la red de gestión en `wlan0`: asigna la IP de puerta de enlace, arranca `hostapd` con `mgmt.conf` y lanza `dnsmasq` para el servicio DHCP y DNS.

```
#!/bin/bash
LOG=/var/log/tfg-mgmt.log
echo "=== $(date) === Arrancando ===" >> $LOG
ip link set wlan0 down 2>/dev/null
iw dev wlan0 set type managed 2>/dev/null
sleep 1
ip link set wlan0 up
sleep 2
ip addr flush dev wlan0
ip addr add 172.31.0.1/24 dev wlan0
hostapd -B /root/patatawifi/hostapd-mgmt/mgmt.conf >> $LOG 2>&1
sleep 2
dnsmasq --interface=wlan0 --bind-interfaces --dhcp-range
    =172.31.0.10,172.31.0.100,12h --dhcp-option=3,172.31.0.1 --dhcp-option
    =6,1.1.1.1,8.8.8.8 --no-resolv --pid-file=/tmp/dnsmasq-mgmt.pid --log-
    facility=/tmp/dnsmasq-mgmt.log >> $LOG 2>&1
sleep 2
echo "=== $(date) === OK ===" >> $LOG
exit 0
```

Código A.12: scripts/tfg-mgmt.sh

El tercer *script*, `tfg-attack.sh`, espera a que aparezca la interfaz `wlan1` de la Alfa AWUS036ACM y lanza el *wrapper* `hostapd-freeradius.sh` que abre la sesión `tmux` de cinco paneles donde conviven `hostapd`, `dnsmasq`, `radiusd` y los seguimientos en tiempo real de los registros `wpe.log` y `auth-detail`.

```
#!/bin/bash
LOG=/var/log/tfg-attack.log
echo "=== $(date) === Arrancando Evil Twin ===" >> $LOG
for i in {1..15}; do
    if ip link show wlan1 &>/dev/null; then
        break
    fi
    sleep 1
done
if ! ip link show wlan1 &>/dev/null; then
    echo "ERROR: wlan1 no detectado" >> $LOG
    exit 1
fi
cd /root/patatawifi
./hostapd-freeradius.sh >> $LOG 2>&1 &
sleep 10
if ! pgrep -f "hostapd PatataWifi_Hostapd" > /dev/null; then
    echo "ERROR: PatataWiFi no arrancho" >> $LOG
    exit 1
fi
echo "=== $(date) === OK ===" >> $LOG
exit 0
```

Código A.13: scripts/tfg-attack.sh

Las tres unidades `systemd` de tipo `oneshot` encadenan los *scripts* anteriores mediante

directivas `After/Requires`, de modo que la secuencia `cleanup` → `mgmt` → `attack` se ejecute de forma ordenada tras el arranque. La primera unidad inicia la cadena.

```
[Unit]
Description=TFG - Cleanup previo al arranque del laboratorio
After=network.target
DefaultDependencies=no

[Service]
Type=oneshot
ExecStart=/usr/local/bin/tfg-cleanup.sh
RemainAfterExit=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Código A.14: `systemd/tfg-cleanup.service`

La segunda unidad depende de la anterior y levanta la red de gestión.

```
[Unit]
Description=TFG Red gestion wlan0
After=tfg-cleanup.service
Requires=tfg-cleanup.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/tfg-mgmt.sh
RemainAfterExit=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Código A.15: `systemd/tfg-mgmt.service`

La tercera unidad cierra la cadena lanzando el *Evil Twin* sobre `wlan1`.

```
[Unit]
Description=TFG - Evil Twin eduroam-tfg (wlan1)
After=tfg-mgmt.service
Requires=tfg-mgmt.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/tfg-attack.sh
RemainAfterExit=yes
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

Código A.16: systemd/tfg-attack.service

Los parches que siguen modernizan y reorientan el comportamiento de FreeRADIUS-WPE y del *upstream* PatataWiFi. El primero comenta las directivas `user/group` para que `radiusd` corra como `root` y pueda gestionar las interfaces.

```
--- a/radiusd.conf
+++ b/radiusd.conf
@@ -544,8 +544,8 @@
     # member. This can allow for some finer-grained access
     # controls.
     #
-   user = freerad-wpe
-   group = freerad-wpe
+   #user = freerad-wpe
+   #group = freerad-wpe

     # Core dumps are a bad thing. This should only be set to
     # 'yes' if you're debugging a problem with the server.
```

Código A.17: freeradius-wpe/radiusd.conf.patch

El segundo parche redirige las rutas del material TLS hacia los certificados generados por `make bootstrap`, sustituyendo los certificados *snakeoil* del sistema.

```
--- a/mods-enabled/eap
+++ b/mods-enabled/eap
@@ -198,7 +198,7 @@
     #
     tls-config tls-common {
         private_key_password = whatever
-       private_key_file = /etc/ssl/private/ssl-cert-snakeoil.key
+       private_key_file = ${certdir}/server.key

         # If Private key & Certificate are located in
         # the same file, then private_key_file &
@@ -234,7 +234,7 @@
         # give advice which will work everywhere. Instead,
         # we give general guidelines.
         #
-       certificate_file = /etc/ssl/certs/ssl-cert-snakeoil.pem
+       certificate_file = ${certdir}/server.pem

         # Trusted Root CA list
         #
@@ -247,7 +247,7 @@
         # In that case, this CA file should contain
         # *one* CA certificate.
         #
-       ca_file = /etc/ssl/certs/ca-certificates.crt
```



```
-
    beacon_int=100
    dtim_period=2
    max_num_sta=255
    -rts_threshold=2347
    -fragm_threshold=2346
    macaddr_acl=0
    -auth_algs=3
    +auth_algs=1
    ignore_broadcast_ssid=0
    wmm_enabled=1
    -wmm_ac_bk_cwmin=4
    -wmm_ac_bk_cwmax=10
    -wmm_ac_bk_aifs=7
    -wmm_ac_bk_txop_limit=0
    -wmm_ac_bk_acm=0
    -wmm_ac_be_aifs=3
    -wmm_ac_be_cwmin=4
    -wmm_ac_be_cwmax=10
    -wmm_ac_be_txop_limit=0
    -wmm_ac_be_acm=0
    -wmm_ac_vi_aifs=2
    -wmm_ac_vi_cwmin=3
    -wmm_ac_vi_cwmax=4
    -wmm_ac_vi_txop_limit=94
    -wmm_ac_vi_acm=0
    -wmm_ac_vo_aifs=2
    -wmm_ac_vo_cwmin=2
    -wmm_ac_vo_cwmax=3
    -wmm_ac_vo_txop_limit=47
    -wmm_ac_vo_acm=0
    ieee80211n=1
-
-
+## WPA2-Enterprise
+ieee8021x=1
    eapol_key_index_workaround=0
    eap_server=0
    own_ip_addr=127.0.0.1
+auth_server_addr=127.0.0.1
+auth_server_port=1812
+auth_server_shared_secret=testing123
    wpa=2
    -#wpa_key_mgmt=WPA-EAP
    -wpa_key_mgmt=WPA-PSK
    -wpa_passphrase=[MGMT_PASSWORD]
    -wpa_pairwise=TKIP CCMP
    +wpa_key_mgmt=WPA-EAP
    +wpa_pairwise=CCMP
    rsn_pairwise=CCMP
-
+ieee80211w=1
```

Código A.20: patatawifi-patches/patatawifi.conf.patch

El quinto parche ajusta las redes virtuales del *wrapper*: fija una BSSID, establece CCMP como cifrado y activa la protección de tramas de gestión (PMF, del inglés *Protected Management Frames*).

```
--- a/hostapd/patatawifi-virtual.conf
+++ b/hostapd/patatawifi-virtual.conf
@@ -1,6 +1,7 @@

#### Multiple BSSID ####
bss=[VIRTUALINTERFACE]
+bssid=06:00:00:00:00:01
ssid=[SSID]
country_code=ES

@@ -15,5 +16,6 @@

wpa=2
wpa_key_mgmt=WPA-EAP
-wpa_pairwise=TKIP CCMP
+wpa_pairwise=CCMP
rsn_pairwise=CCMP
+ieee80211w=1
```

Código A.21: patatawifi-patches/patatawifi-virtual.conf.patch

El último parche habilita las opciones de compilación necesarias para construir **hostapd** 2.6: el soporte de libnl 3.2 y el modo IEEE 802.11n de alto rendimiento.

```
--- a/hostapd/.config
+++ b/hostapd/.config
@@ -31,7 +31,7 @@
#CONFIG_LIBNL20=y

# Use libnl 3.2 libraries (if this is selected, CONFIG_LIBNL20 is ignored)
-#CONFIG_LIBNL32=y
+CONFIG_LIBNL32=y

# Driver interface for FreeBSD net80211 layer (e.g., Atheros driver)
@@ -148,7 +148,7 @@
#CONFIG_DRIVER_RADIUS_ACL=y

# IEEE 802.11n (High Throughput) support
-#CONFIG_IEEE80211N=y
+CONFIG_IEEE80211N=y

# Wireless Network Management (IEEE Std 802.11v-2011)
# Note: This is experimental and not complete implementation.
```

Código A.22: patatawifi-patches/hostapd-2.6-config.patch

B

Referencia de comandos de operación

Este apéndice recopila, a modo de referencia rápida, los comandos de operación, verificación y monitorización empleados durante la explotación del laboratorio descrito en el cuerpo del Capítulo 4. Se reproducen agrupados por tarea, con una breve introducción que sitúa cada bloque dentro del flujo de trabajo habitual. Salvo indicación contraria, todos los comandos se ejecutan en la plataforma atacante (Raspberry Pi 5) y requieren privilegios de superusuario, ya sea con `sudo` o como `root`. Cualquier dato que en la práctica revelaría una credencial, un identificador de usuario o un *hash* capturado se sustituye aquí por un marcador entre ángulos (por ejemplo `<usuario>`) o por `[SECRETO]`, de acuerdo con el criterio ético adoptado en toda la memoria; los volcados completos y la ejecución del ataque se documentan en el Capítulo 5.

B.1. Comprobación del estado de los servicios

La cadena de arranque del laboratorio se materializa en tres servicios `systemd` encadenados (`tfg-cleanup` → `tfg-mgmt` → `tfg-attack`). La forma más directa de confirmar que el laboratorio está operativo tras el arranque es consultar el estado de los tres de una sola vez; lo esperable es una triple respuesta `active`.

```
# Estado resumido de los tres servicios (esperado: active / active / active)
sudo systemctl is-active tfg-cleanup tfg-mgmt tfg-attack

# Confirmar que arrancan automáticamente en el siguiente reboot
sudo systemctl is-enabled tfg-cleanup tfg-mgmt tfg-attack

# Detalle de un servicio concreto (p. ej. el del Evil Twin)
sudo systemctl status tfg-attack.service --no-pager

# Diario unificado de los tres servicios desde el último arranque
sudo journalctl -u tfg-cleanup -u tfg-mgmt -u tfg-attack -b
```

Código B.1: Comprobación del estado de la cadena de servicios

Para depuración manual de la secuencia sin necesidad de reiniciar la máquina, los servicios pueden detenerse y relanzarse en orden, respetando las dependencias `Requires=/After=` declaradas en las unidades.

```
sudo systemctl stop tfg-attack tfg-mgmt tfg-cleanup
sudo systemctl start tfg-attack # arrastra mgmt y cleanup por dependencias
```

Código B.2: Relanzamiento manual de la cadena de servicios

B.2. Verificación de los SSID activos

Una vez levantada la cadena de servicios, conviene confirmar que ambas radios están emitiendo el SSID correcto en el canal previsto: la red de gestión `PatataWiFi_mgmt` en `wlan0` (canal 1) y el *Evil Twin* `eduroam-tfg` en `wlan1` (canal 6). El subcomando `iw dev <interfaz> info` expone el SSID, el modo de operación y el canal de cada interfaz.

```
# Red de gestion: esperado ssid PatataWiFi_mgmt, type AP, channel 1
sudo iw dev wlan0 info | grep -E "ssid|type|channel"

# Evil Twin: esperado ssid eduroam-tfg, type AP, channel 6
sudo iw dev wlan1 info | grep -E "ssid|type|channel"

# Direccion IP asignada a cada interfaz (172.31.0.1 y 10.0.0.1)
ip -4 addr show wlan0 | grep "inet "
ip -4 addr show wlan1 | grep "inet "

# Procesos de soporte vivos (hostapd, radiusd, dnsmasq)
ps aux | grep -E "hostapd|radius|dnsmasq" | grep -v grep
```

Código B.3: Verificación de las dos redes al aire

B.3. Monitorización del log de captura

El servidor FreeRADIUS-WPE vuelca el material de autenticación interceptado en `/var/log/freeradius-wpe/freeradius-server-wpe.log`. La supervisión en vivo se realiza con `tail -F`, que sigue el fichero aunque sea rotado o recreado. Para aislar cada uno de los dos resultados posibles del ataque resultan útiles dos filtros: las líneas `pap:` corresponden a la captura de contraseña en claro (cliente que acepta EAP-GTC) y las líneas `mschap:` al *fallback* a MSCHAPv2 (*hash* NetNTLMv1 crackable *offline*).

```
LOG=/var/log/freeradius-wpe/freeradius-server-wpe.log

# Seguimiento en vivo del log completo
sudo tail -F "$LOG"
```

```
# Filtrar solo las capturas en claro (caso EAP-GTC): bloque pap:
#   username: <usuario> / password: <contrasena> [SECRETO]
sudo grep -A2 '^pap:' "$LOG"

# Filtrar solo el fallback MSCHAPv2: bloque mschap: con challenge/response
#   y la linea john NetNTLMv1 lista para crack offline [SECRETO]
sudo grep -A4 '^mschap:' "$LOG"
```

Código B.4: Seguimiento del log de captura de FreeRADIUS-WPE

El *log* crece sin rotación automática; cuando interese arrancar una sesión de captura limpia puede vaciarse sin borrarlo, conservando el descriptor de fichero abierto por el servidor.

```
sudo truncate -s 0 /var/log/freeradius-wpe/freeradius-server-wpe.log
```

Código B.5: Vaciado del log de captura entre sesiones

B.4. Acceso remoto a la red de gestión

La radio interna de la Raspberry Pi sostiene una red WPA2-PSK independiente (PatataWiFi_mgmt, 172.31.0.1/24) cuya única finalidad es la administración fuera de banda de la plataforma: permite operar el laboratorio por SSH sin depender de la interfaz cableada ni interferir con la radio que ejecuta el ataque. Tras asociarse a dicha red con la clave de laboratorio, se accede a la consola de la Pi apuntando a la dirección de su *gateway*.

```
# Desde un cliente asociado a PatataWiFi_mgmt
ssh <usuario>@172.31.0.1
```

Código B.6: Acceso SSH por la red de gestion fuera de banda

B.5. Acceso a la sesión tmux del orquestador

El *wrapper* heredado de PatataWiFiEnterprise multiplexa los procesos del *Evil Twin* en una sesión *tmux* de cinco paneles (*hostapd*, *dnsmasq*, *radiusd* y sendos seguimientos de *wpe.log* y de *auth-detail*). Adjuntarse a esa sesión es la vía más cómoda para observar simultáneamente todos los componentes del ataque en tiempo real.

```
# Adjuntar a la sesion del orquestador (5 paneles)
sudo tmux attach -t PatataWifi_Hostapd

# Listar sesiones activas / desconectarse sin matar la sesion
sudo tmux ls                # listar
# (dentro de tmux) prefijo + d para hacer detach
```

Código B.7: Conexion a la sesion tmux del Evil Twin

B.6. Recarga del driver de la radio atacante

El *chipset* MediaTek MT7612U del adaptador Alfa AWUS036ACM, gobernado por el *driver* `mt76x2u`, puede quedar en un estado inconsistente tras un apagado abrupto o un ciclo de pruebas. El servicio `tfg-cleanup` ya descarga y recarga el módulo en cada arranque; cuando la incidencia surge en mitad de una sesión, la recarga manual del *driver* devuelve la interfaz `wlan1` a un estado limpio.

```
sudo rmmod mt76x2u
sleep 2
sudo modprobe mt76x2u

# Confirmar que el driver se ha registrado de nuevo
sudo dmesg | grep -E "mt76|wlan1"
#   esperado: usbcore: registered new interface driver mt76x2u
#             mt76x2u: ASIC revision: 76120044
```

Código B.8: Recarga manual del driver `mt76x2u`

B.7. Regeneración de los certificados del servidor RADIUS

Los certificados que FreeRADIUS-WPE presenta en el túnel TLS exterior se generan con el objetivo `bootstrap` del `Makefile` incluido en el directorio de certificados de la instalación. Caducan aproximadamente al año, por lo que deben regenerarse periódicamente; el mismo procedimiento se emplea cuando se modifican los nombres distinguidos en los ficheros `ca.cnf` o `server.cnf`. Tras regenerarlos conviene reiniciar la cadena de ataque (o la máquina) para que el servidor recargue el nuevo material.

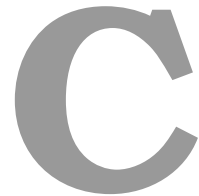
```
# (Opcional) editar los nombres distinguidos antes de regenerar
#   sudo nano /etc/freeradius-wpe/3.0/certs/ca.cnf
#   sudo nano /etc/freeradius-wpe/3.0/certs/server.cnf

# Regenerar CA y certificado de servidor
sudo make -C /etc/freeradius-wpe/3.0/certs bootstrap

# Comprobar que el nuevo material se ha generado
ls -l /etc/freeradius-wpe/3.0/certs/{ca.pem,server.pem}

# Aplicar el cambio reiniciando la cadena de ataque
sudo systemctl restart tfg-attack
```

Código B.9: Regeneración de los certificados con `make bootstrap`



Herramientas utilizadas en el proyecto

Este apéndice recoge las herramientas empleadas a lo largo del trabajo, tanto las que sirvieron para construir y operar el laboratorio como las utilizadas para redactar y revisar esta memoria. Muchas de ellas ya se han descrito en su contexto técnico en el Capítulo 4, por lo que aquí se enumeran de forma resumida. El apéndice incluye además, en su última sección, la declaración explícita del uso de inteligencia artificial que exige la normativa de la Universidad Loyola Andalucía.

C.1. Herramientas de construcción y operación del laboratorio

La red legítima se levantó sobre una máquina virtual Ubuntu Server alojada en VMware Workstation Pro 17, instalado a su vez sobre el portátil del autor con Windows 11. En ese sistema invitado se configuraron FreeRADIUS como servidor de autenticación 802.1X, **hostapd** como punto de acceso y OpenSSL para generar la infraestructura de clave pública con el certificado propio de la red.

La plataforma atacante se construyó sobre una Raspberry Pi 5 con Raspberry Pi OS *bookworm*. En ella se emplearon FreeRADIUS-WPE 3.2.5 como *backend* de captura, **hostapd** 2.6 para el *Evil Twin*, **dnsmasq** para el reparto de direcciones por DHCP, **systemd** para encadenar los servicios de arranque y **tmux** para multiplexar y supervisar los procesos del ataque. La administración del equipo se realizó de forma remota por SSH a través de la red de gestión, empleando PuTTY como cliente de consola y WinSCP para la transferencia de ficheros desde el portátil con Windows 11. Para el análisis *offline* de las credenciales capturadas se utilizaron hashcat y john.

C.2. Herramientas de redacción y control de versiones

La memoria se redactó en LaTeX, compilándola en Overleaf y editándola también en local con Visual Studio Code, que sirvió igualmente para escribir y revisar los *scripts* de configuración del laboratorio. El control de versiones se llevó con git y GitHub, que permitieron mantener el histórico de cambios y facilitar la supervisión del trabajo por parte de los tutores. En GitHub residen tanto el repositorio de la propia memoria como el repositorio público con los artefactos del laboratorio [31].

C.3. Uso de inteligencia artificial

En cumplimiento del Artículo 18 de la normativa de Trabajo Fin de Grado de la Universidad Loyola Andalucía, se declara de manera explícita el uso de herramientas de inteligencia artificial (IA) durante la realización del trabajo, indicando para cada una su función y el impacto que ha tenido. En todos los casos su empleo se ha mantenido dentro de las tres formas que la normativa considera válidas, esto es, el apoyo a la investigación, la asistencia en la elaboración de la memoria y el apoyo en la generación de soluciones técnicas.

Las herramientas utilizadas y la función que cumplieron fueron las siguientes:

- NotebookLM. Empleada como apoyo a la investigación, para organizar y consultar la documentación técnica y los artículos del estado del arte, así como para resolver dudas sobre el material recopilado. La selección de las fuentes y la interpretación de su contenido se hicieron de forma autónoma.
- Google Gemini y ChatGPT. Utilizadas como asistentes generales para aclarar conceptos de WPA2-Enterprise, 802.1X y los métodos EAP, contrastar explicaciones y orientar la búsqueda de información durante la fase de documentación.
- Claude y Claude Code. Empleadas como apoyo en la elaboración de la memoria y en la resolución de problemas técnicos. En la memoria ayudaron a estructurar apartados, a corregir la redacción y a sugerir formas de expresar ideas ya definidas por el autor. En la parte técnica sirvieron para depurar errores de configuración y para entender el comportamiento de herramientas como `hostapd`, `FreeRADIUS-WPE` o `systemd` cuando el montaje no respondía como se esperaba.

El impacto de estas herramientas fue el de acelerar la comprensión de los conceptos y la superación de las dificultades y los errores que fueron surgiendo durante el trabajo, actuando siempre como un apoyo y no como sustituto del trabajo propio.